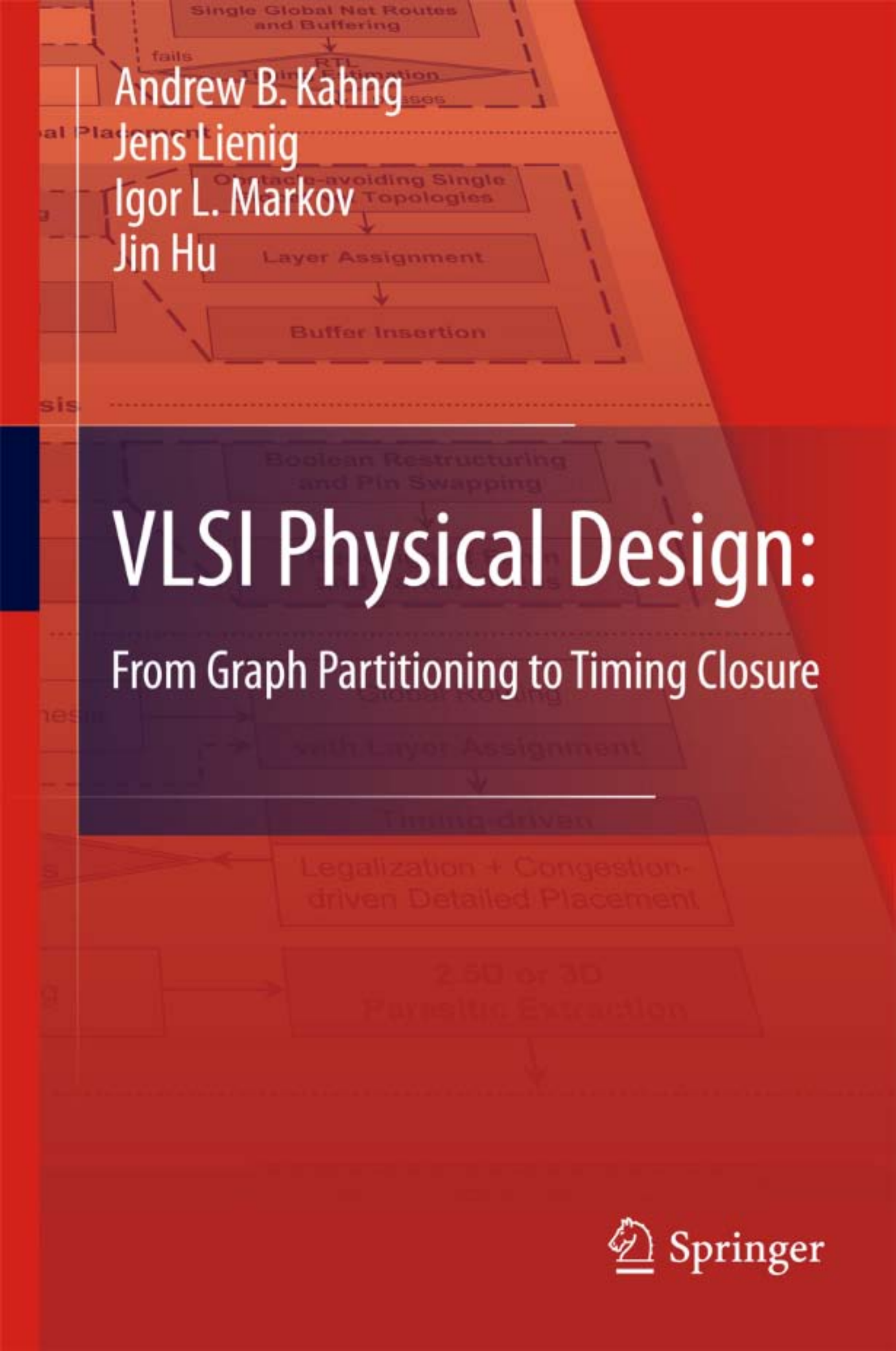


Andrew B. Kahng
Jens Lienig
Igor L. Markov
Jin Hu

VLSI Physical Design:

From Graph Partitioning to Timing Closure

Springer

Andrew B. Kahng • Jens Lienig
Igor L. Markov • Jin Hu

VLSI Physical Design: From Graph Partitioning to Timing Closure

Springer

Andrew B. Kahng
University of California at San Diego
Departments of CSE and ECE
Mailcode #0404
La Jolla, California 92093
USA
abk@ucsd.edu

Jens Lienig
Dresden University of Technology
Electrical Engineering and
Information Technology
Helmholtzstr. 10
01069 Dresden
Germany
jens@ieee.org

Igor L. Markov
University of Michigan
Electrical Engineering and
Computer Science
2260 Hayward St.
Ann Arbor, Michigan 48109
USA
imarkov@eecs.umich.edu

Jin Hu
University of Michigan
Electrical Engineering and
Computer Science
2260 Hayward St.
Ann Arbor, Michigan 48109
USA
jinhu@eecs.umich.edu

ISBN 978-90-481-9590-9 e-ISBN 978-90-481-9591-6
DOI 10.1007/978-90-481-9591-6
Springer Dordrecht Heidelberg London New York

© Springer Science+Business Media B.V. 2011
No part of this work may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, microfilming, recording or otherwise, without written permission from the Publisher, with the exception of any material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work.

Cover design: eStudio Calamar S.L.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Foreword

Physical design of integrated circuits remains one of the most interesting and challenging arenas in the field of Electronic Design Automation. The ability to integrate more and more devices on our silicon chips requires the algorithms to continuously scale up. Nowadays we can integrate 2e9 transistors on a single 45nm-technology chip. This number will continue to scale for the next couple of technology generations, requiring more transistors to be automatically placed on a chip and connected together. In addition, more and more of the delay is contributed by the wires that interconnect the devices on the chip. This has a profound effect on how physical design flows need to be put together. In the 1990s, it was safe to assume that timing goals of the design could be reached once the devices were placed well on the chip. Today, one does not know whether the timing constraints can be satisfied until the final routing has completed.

As far back as 10 or 15 years ago, people believed that most physical design problems had been solved. But, the continued increase in the number of transistors on the chip, as well as the increased coupling between the physical, timing and logic domains warrant a fresh look at the basic algorithmic foundations of chip implementation. That is exactly what this book provides. It covers the basic algorithms underlying all physical design steps and also shows how they are applied to current instances of the design problems. For example, Chapter 7 provides a great deal of information on special types of routing for specific design situations.

Several other books provide in-depth descriptions of core physical design algorithms and the underlying mathematics, but this book goes a step further. The authors very much realize that the era of individual point algorithms with single objectives is over. Throughout the book they emphasize the multi-objective nature of modern design problems and they bring all the pieces of a physical design flow together in Chapter 8. A complete flow chart, from design partitioning and floorplanning all the way to electrical rule checking, describes all phases of the modern chip implementation flow. Each step is described in the context of the overall flow with references to the preceding chapters for the details.

This book will be appreciated by students and professionals alike. It starts from the basics and provides sufficient background material to get the reader up to speed on the real issues. Each of the chapters by itself provides sufficient introduction and depth to be very valuable. This is especially important in the present era, where experts in one area must understand the effects of their algorithms on the remainder of the design flow. An expert in routing will derive great benefit from reading the chapters on planning and placement. An expert in Design For Manufacturability (DFM) who seeks a better understanding of routing algorithms, and of how these algorithms can be affected by choices made in setting DFM requirements, will benefit tremendously from the chapters on global and detailed routing.

The book is completed by a detailed set of solutions to the exercises that accompany each chapter. The exercises force the student to truly understand the basic physical design algorithms and apply them to small but insightful problem instances.

This book will serve the EDA and design community well. It will be a foundational text and reference for the next generation of professionals who will be called on to continue the advancement of our chip design tools.

Dr. Leon Stok
Vice President, Electronic Design Automation
IBM Systems and Technology Group
Hopewell Junction, NY

Preface

VLSI physical design of integrated circuits underwent explosive development in the 1980s and 1990s. Many basic techniques were suggested by researchers and implemented in commercial tools, but only described in brief conference publications geared for experts in the field. In the 2000s, academic and industry researchers focused on comparative evaluation of basic techniques, their extension to large-scale optimization, and the assembly of point optimizations into multi-objective design flows. Our book covers these aspects of physical design in a consistent way, starting with basic concepts in Chapter 1 and gradually increasing the depth to reach advanced concepts, such as physical synthesis. Readers seeking additional details, will find a number of references discussed in each chapter, including specialized monographs and recent conference publications.

Chapter 2 covers netlist partitioning. It first discusses typical problem formulations and proceeds to classic algorithms for balanced graph and hypergraph partitioning. The last section covers an important application – system partitioning among multiple FPGAs, used in the context of high-speed emulation in functional validation.

Chapter 3 is dedicated to chip planning, which includes floorplanning, power-ground planning and I/O assignment. A broad range of topics and techniques are covered, ranging from graph-theoretical aspects of block-packing to optimization by simulated annealing and package-aware I/O planning.

Chapter 4 addresses VLSI placement and covers a number of practical problem formulations. It distinguishes between global and detailed placement, and first covers several algorithmic frameworks traditionally used for global placement. Detailed placement algorithms are covered in a separate section. Current state of the art in placement is reviewed, with suggestions to readers who might want to implement their own software tools for large-scale placement.

Chapters 5 and 6 discuss global and detailed routing, which have received significant attention in research literature due to their interaction with manufacturability and chip-yield optimizations. Topics covered include representing layout with graph models and performing routing, for single and multiple nets, in these models. State-of-the-art global routers are discussed, as well as yield optimizations performed in detailed routing to address specific types of manufacturing faults.

Chapter 7 deals with several specialized types of routing which do not conform with the global-detailed paradigm followed by Chapters 5 and 6. These include non-Manhattan area routing, commonly used in PCBs, and clock-tree routing required for every synchronous digital circuit. In addition to algorithmic aspects, we explore the impact of process variability on clock-tree routing and means of decreasing this impact.

Chapter 8 focuses on timing closure, and its perspective is particularly unique. It offers a comprehensive coverage of timing analysis and relevant optimizations in placement, routing and netlist restructuring. Section 8.6 assembles all these techniques, along with those covered in earlier chapters, into an extensive design flow, illustrated in detail with a flow chart and discussed step-by-step with several figures and many references.

This book does not assume prior exposure to physical design or other areas of EDA. It introduces the reader to the EDA industry and basic EDA concepts, covers key graph concepts and algorithm analysis, carefully defines terms and specifies basic algorithms with pseudocode. Many illustrations are given throughout the book, and every chapter includes a set of exercises, solutions to which are given in one of the appendices. Unlike most other sources on physical design, we made an effort to avoid impractical and unnecessarily complicated algorithms. In many cases we offer comparisons between several leading algorithmic techniques and refer the reader to publications with additional empirical results.

Some chapters are based on material in the book *Layoutsynthese elektronischer Schaltungen – Grundlegende Algorithmen für die Entwurfsautomatisierung*, which was published by Springer in 2006.

We are grateful to our colleagues and students who proofread earlier versions of this book and suggested a number of improvements (in alphabetical order): Matthew Guthaus, Kwangok Jeong, Johann Knechtel, Andreas Krinke, Nancy MacDonald, Jarrod Roy, Yen-Kuan Wu and Hailong Yao.

Images for global placement and clock routing in Chapter 8 were provided by Myung-Chul Kim and Dong-Jin Lee. Cell libraries in Appendix B were provided by Bob Bullock, Dan Clein and Bill Lye from PMC Sierra; the layout and schematics in Appendix B were generated by Matthias Thiele. The work on this book was partially supported by the National Science Foundation (NSF) through the CAREER award 0448189 as well as by Texas Instruments and Sun Microsystems.

We hope that you will find the book interesting to read and useful in your professional endeavors.

Sincerely,

Andrew, Jens, Igor and Jin

Table of Contents

1 Introduction 3
1.1 Electronic Design Automation (EDA) 4
1.2 VLSI Design Flow 7
1.3 VLSI Design Styles 11
1.4 Layout Layers and Design Rules 16
1.5 Physical Design Optimizations 18
1.6 Algorithms and Complexity 20
1.7 Graph Theory Terminology 24
1.8 Common EDA Terminology 26
Chapter 1 References 30

2 Netlist and System Partitioning 33
2.1 Introduction 33
2.2 Terminology 34
2.3 Optimization Goals 35
2.4 Partitioning Algorithms 36
2.4.1 Kernighan-Lin (KL) Algorithm 36
2.4.2 Extensions of the Kernighan-Lin Algorithm 41
2.4.3 Fiduccia-Mattheyses (FM) Algorithm 41
2.5 A Framework for Multilevel Partitioning 47
2.5.1 Clustering 48
2.5.2 Multilevel Partitioning 48
2.6 System Partitioning onto Multiple FPGAs 50
Chapter 2 Exercises 53
Chapter 2 References 54

3 Chip Planning 57
3.1 Introduction to Floorplanning 58
3.2 Optimization Goals in Floorplanning 59
3.3 Terminology 61
3.4 Floorplan Representations 63
3.4.1 Floorplan to a Constraint-Graph Pair 63
3.4.2 Floorplan to a Sequence Pair 64
3.4.3 Sequence Pair to a Floorplan 65
3.5 Floorplanning Algorithms 68
3.5.1 Floorplan Sizing 69
3.5.2 Cluster Growth 73
3.5.3 Simulated Annealing 77
3.5.4 Integrated Floorplanning Algorithms 81
3.6 Pin Assignment 82
3.7 Power and Ground Routing 86
3.7.1 Design of a Power-Ground Distribution Network 87
3.7.2 Planar Routing 87
3.7.3 Mesh Routing 89
Chapter 3 Exercises 91
Chapter 3 References 92

4 Global and Detailed Placement 95
4.1 Introduction 95
4.2 Optimization Objectives 96
4.3 Global Placement 103
4.3.1 Min-Cut Placement 104
4.3.2 Analytic Placement 110
4.3.3 Simulated Annealing 117
4.3.4 Modern Placement Algorithms 120
4.4 Legalization and Detailed Placement 122
Chapter 4 Exercises 125
Chapter 4 References 126

5 Global Routing 131
5.1 Introduction 131
5.2 Terminology and Definitions 133
5.3 Optimization Goals 136
5.4 Representations of Routing Regions 138
5.5 The Global Routing Flow 140
5.6 Single-Net Routing 141
5.6.1 Rectilinear Routing 141
5.6.2 Global Routing in a Connectivity Graph 146
5.6.3 Finding Shortest Paths with Dijkstra's Algorithm 149
5.6.4 Finding Shortest Paths with A* Search 154
5.7 Full-Netlist Routing 155
5.7.1 Routing by Integer Linear Programming 155
5.7.2 Rip-Up and Reroute (RRR) 158
5.8 Modern Global Routing 160
5.8.1 Pattern Routing 161
5.8.2 Negotiated Congestion Routing 162
Chapter 5 Exercises 164
Chapter 5 References 165

6 Detailed Routing 169
6.1 Terminology 169
6.2 Horizontal and Vertical Constraint Graphs 172
6.2.1 Horizontal Constraint Graphs 172
6.2.2 Vertical Constraint Graphs 173
6.3 Channel Routing Algorithms 175
6.3.1 Left-Edge Algorithm 175
6.3.2 Dogleg Routing 178
6.4 Switchbox Routing 180
6.4.1 Terminology 180
6.4.2 Switchbox Routing Algorithms 181
6.5 Over-the-Cell Routing Algorithms 182
6.5.1 OTC Routing Methodology 183
6.5.2 OTC Routing Algorithms 184
6.6 Modern Challenges in Detailed Routing 185
Chapter 6 Exercises 187
Chapter 6 References 188

7 Specialized Routing 191
7.1 Introduction to Area Routing 191
7.2 Net Ordering in Area Routing 193
7.3 Non-Manhattan Routing 195
7.3.1 Octilinear Steiner Trees 195
7.3.2 Octilinear Maze Search 197
7.4 Basic Concepts in Clock Networks 197
7.4.1 Terminology 198
7.4.2 Problem Formulations for Clock-Tree Routing 201
7.5 Modern Clock Tree Synthesis 203
7.5.1 Constructing Trees with Zero Global Skew 203
7.5.2 Clock Tree Buffering in the Presence of Variation 212
Chapter 7 Exercises 215
Chapter 7 References 217

8 Timing Closure 221
8.1 Introduction 221
8.2 Timing Analysis and Performance Constraints 223
8.2.1 Static Timing Analysis 224
8.2.2 Delay Budgeting with the Zero-Slack Algorithm 229
8.3 Timing-Driven Placement 233
8.3.1 Net-Based Techniques 234
8.3.2 Embedding STA into Linear Programs for Placement 237
8.4 Timing-Driven Routing 239
8.4.1 The Bounded-Radius, Bounded-Cost Algorithm 240
8.4.2 Prim-Dijkstra Tradeoff 241
8.4.3 Minimization of Source-to-Sink Delay 242
8.5 Physical Synthesis 244
8.5.1 Gate Sizing 244
8.5.2 Buffering 245
8.5.3 Netlist Restructuring 246
8.6 Performance-Driven Design Flow 250
8.7 Conclusions 258
Chapter 8 Exercises 260
Chapter 8 References 262

A Solutions to Chapter Exercises 267
Chapter 2: Netlist and System Partitioning 267
Chapter 3: Chip Planning 270
Chapter 4: Global and Detailed Placement 273
Chapter 5: Global Routing 276
Chapter 6: Detailed Routing 280
Chapter 7: Specialized Routing 284
Chapter 8: Timing Closure 292

B Example CMOS Cell Layouts 299

Chapter 1

Introduction

1

1 Introduction 3
1.1 Electronic Design Automation (EDA) 4
1.2 VLSI Design Flow 7
1.3 VLSI Design Styles 11
1.4 Layout Layers and Design Rules 16
1.5 Physical Design Optimizations 18
1.6 Algorithms and Complexity 20
1.7 Graph Theory Terminology 24
1.8 Common EDA Terminology 26
Chapter 1 References 30

1

1 Introduction

The design and optimization of *integrated circuits* (*ICs*) are essential to the production of new semiconductor chips. Modern chip design has become so complex that it is largely performed by specialized software, which is frequently updated to reflect improvements in semiconductor technologies and increasing design complexities. A *user* of this software needs a high-level understanding of the implemented algorithms. On the other hand, a *developer* of this software must have a strong computer-science background, including a keen understanding of how various algorithms operate and interact, and what their performance bottlenecks are.

This book introduces and evaluates algorithms used during *physical design* to produce a geometric chip layout from an abstract circuit design. Rather than list every relevant technique, however, it presents the essential and fundamental algorithms used within each physical design stage.

- *Partitioning* (Chap. 2) and *chip planning* (Chap. 3) of design functionality during the initial stages of physical design
- Geometric *placement* (Chap. 4) and *routing* (Chaps. 5-6) of circuit components
- Specialized routing and *clock tree synthesis* for synchronous circuits (Chap. 7)
- Meeting specific technology and performance requirements, i.e., *timing closure*, such that the final fabricated layout satisfies system objectives (Chap. 8)

Other design steps, such as *circuit design*, *logic synthesis*, *transistor-level layout* and *verification*, are not discussed in detail, but are covered in such references as [1.1].

This book emphasizes *digital* circuit design for *very large-scale integration* (*VLSI*); the degree of automation for digital circuits is significantly higher than for *analog* circuits. In particular, the focus is on algorithms for digital ICs, such as system partitioning for *field-programmable gate arrays* (*FPGAs*) or *clock network synthesis* for *application-specific integrated circuits* (*ASICs*). Similar design techniques can be applied to other implementation contexts such as *multi-chip modules* (*MCMs*) and *printed circuit boards* (*PCBs*).

The following broad questions, of interest to both students and designers, are addressed in the upcoming chapters.

- How is functionally correct layout produced from a netlist?
- How does software for VLSI physical design work?
- How do we develop and improve software for VLSI physical design?

More information about this book is at http://vlsicad.eecs.umich.edu/KLMH/.

A. B. Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure,*
DOI 10.1007/978-90-481-9591-6_1, © Springer Science+Business Media B.V. 2011

1.1 1.1 Electronic Design Automation (EDA)

The *Electronic Design Automation* (*EDA*) industry develops software to support engineers in the creation of new integrated-circuit (IC) designs. Due to the high complexity of modern designs, EDA touches almost every aspect of the IC design flow, from high-level system design to fabrication. EDA addresses designers' needs at multiple levels of electronic system hierarchy, including integrated circuits (ICs), multi-chip modules (MCMs), and printed circuit boards (PCBs).

Progress in semiconductor technology, based on *Moore's Law* (Fig. 1.1), has led to integrated circuits (1) comprised of hundreds of millions of transistors, (2) assembled into packages, each having multiple chips and thousands of pins, and (3) mounted onto *high-density interconnect* (*HDI*) circuit boards with dozens of wiring layers. This design process is highly complex and heavily depends on automated tools. That is, computer software is used to mostly automate design steps such as logic design, simulation, physical design, and verification.

EDA was first used in the 1960s in the form of simple programs to automate placement of a very small number of blocks on a circuit board. Over the next few years, the advent of the integrated circuit created a need for software that could reduce the total number of gates. Current software tools must additionally consider electrical effects such as signal delays and capacitive coupling between adjacent wires. In the modern VLSI design flow, nearly all steps use software to automate optimizations.

In the 1970s, semiconductor companies developed in-house EDA software, specialized programs to address their proprietary design styles. In the 1980s and 1990s, independent software vendors created new tools for more widespread use. This gave rise to an independent EDA industry, which now enjoys annual revenues of approximately five billion dollars and employs around twenty thousand people. Many EDA companies have headquarters in Santa Clara county, in the state of California. This area has been aptly dubbed the *Silicon Valley*.

Several annual conferences showcase the progress of the EDA industry and academia. The most notable one is the *Design Automation Conference* (*DAC*), which holds an industry trade show as well as an academic symposium. The *International Conference on Computer-Aided Design* (*ICCAD*) places emphasis on academic research, with papers that relate to specialized algorithm development. PCB developers attend *PCB Design Conference West* in September. Overseas, Europe and Asia host the *Design, Automation and Test in Europe* (*DATE*) conference and the *Asia and South Pacific Design Automation Conference* (*ASP-DAC*), respectively. The world-wide engineering association *Institute of Electrical and Electronic Engineers* (*IEEE*) publishes the monthly *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (*TCAD*), while the *Association for Computing Machinery* (*ACM*) publishes *ACM Transactions on Design Automation of Electronic Systems* (*TODAES*).

Impact of EDA. According to Moore's Law (Fig. 1.1), the number of transistors on a chip is increasing at an exponential rate. Historically, this corresponds to an annual *compounded* increase of 58% in the number of transistors per chip.

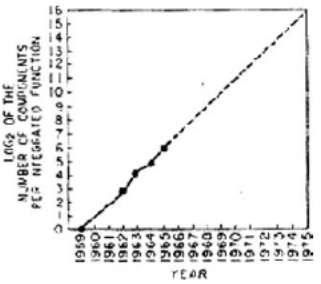


Fig. 1.1 Moore's Law and the original graph from Gordon Moore's article [1.10] predicting the increase in number of transistors. In 1965, Gordon Moore (Fairchild) stated that the number of transistors on an IC would double every year. Ten years later, he revised his statement, asserting that doubling would occur every 18 months. Since then, this "rule" has been famously known as Moore's Law.

However, chip designs produced by prominent semiconductor companies suggest a different trend. The annual *productivity*, measured by the number of transistors, of designers, and (fixed-size) design teams has an annual compounded growth of only around 21% per year, leading to a *design productivity gap* [1.5]. Since the number of transistors is highly context-specific – analog versus digital or memory versus logic – this statistic, due to SEMATECH in the mid-1990s, refers to the design productivity for *standardized transistors*.

Fig 1.2, reproduced from the *International Technology Roadmap for Semiconductors* (*ITRS*) [1.5], demonstrates that cost-feasible IC products require innovation in EDA technology. Given the availability of efficient design technologies to semiconductor design teams, the hardware design cost of a typical portable system-on-chip, e.g., a baseband processor chip for a cell phone, remains manageable at $15.7 million (2009 estimate). With associated software design costs, the overall chip design project cost is $45.3 million. Without the design technology innovations between 1993 and 2007, and the resulting design productivity improvements, the design cost of a chip would have been $1,800 million, well over a *billion* dollars.

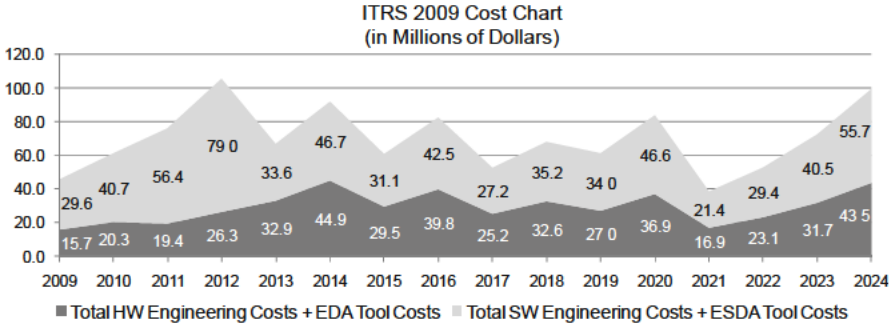


Fig. 1.2 Recent editions of the semiconductor technology roadmap project total hardware (HW) engineering costs + EDA tool costs (dark gray) and total software (SW) engineering costs + electronic system design automation (ESDA) tool costs (light gray). This shows the impact of EDA technologies on overall IC design productivity and hence IC design cost [1.5].

History of EDA. After tools for schematic entry of integrated circuits were developed, the first EDA tool, a placer that optimized the physical locations of devices on a circuit board, was created in the late 1960s. Shortly thereafter, programs were written to aid circuit layout and visualization. The first integrated-circuit *computer-aided design* (*CAD*) systems addressed the physical design process and were written in the 1970s. During that era, most CAD tools were proprietary – major companies such as IBM and AT&T Bell Laboratories relied on software tools designed for internal use only. However, beginning in the 1980s, independent software developers started to write tools that could serve the needs of multiple semiconductor product companies. The electronic design automation (EDA) market grew rapidly in the 1990s, as many design teams adopted commercial tools instead of developing their own in-house versions. The largest EDA software vendors today are, in alphabetical order, *Cadence Design Systems*, *Mentor Graphics*, and *Synopsys*.

EDA tools have always been geared toward automating the entire design process and linking the design steps into a complete design flow. However, such integration is challenging, since some design steps need additional degrees of freedom, and scalability requires tackling some steps independently. On the other hand, the continued decrease of transistor and wire dimensions has blurred the boundaries and abstractions that separate successive design steps – physical effects such as signal delays and coupling capacitances need to be accurately accounted for earlier in the design cycle. Thus, the design process is moving from a sequence of atomic (independent) steps toward a deeper level of integration. Tab. 1.1 summarizes a timeline of key developments in circuit and physical design.

Tab. 1.1 Timeline of EDA progress with respect to circuit and physical design.

Time Period	Circuit and Physical Design Process Advancements
1950-1965	Manual design only.
1965-1975	Layout editors, e.g., place and route tools, first developed for printed circuit boards.
1975-1985	More advanced tools for ICs and PCBs, with more sophisticated algorithms.
1985-1990	First performance-driven tools and parallel optimization algorithms for layout; better understanding of underlying theory (graph theory, solution complexity, etc.).
1990-2000	First over-the-cell routing, first 3D and multilayer placement and routing techniques developed. Automated circuit synthesis and routability-oriented design become dominant. Start of parallelizing workloads. Emergence of physical synthesis.
2000-now	Design for Manufacturability (DFM), optical proximity correction (OPC), and other techniques emerge at the design-manufacturing interface. Increased reusability of blocks, including intellectual property (IP) blocks.

1.2 VLSI Design Flow

The process of designing a very large-scale integrated (VLSI) circuit is highly complex. It can be separated into distinct steps (Fig. 1.3). Earlier steps are high-level; later design steps are at lower levels of abstraction. At the end of the process, before fabrication, tools and algorithms operate on detailed information about each circuit element's geometric shape and electrical properties.

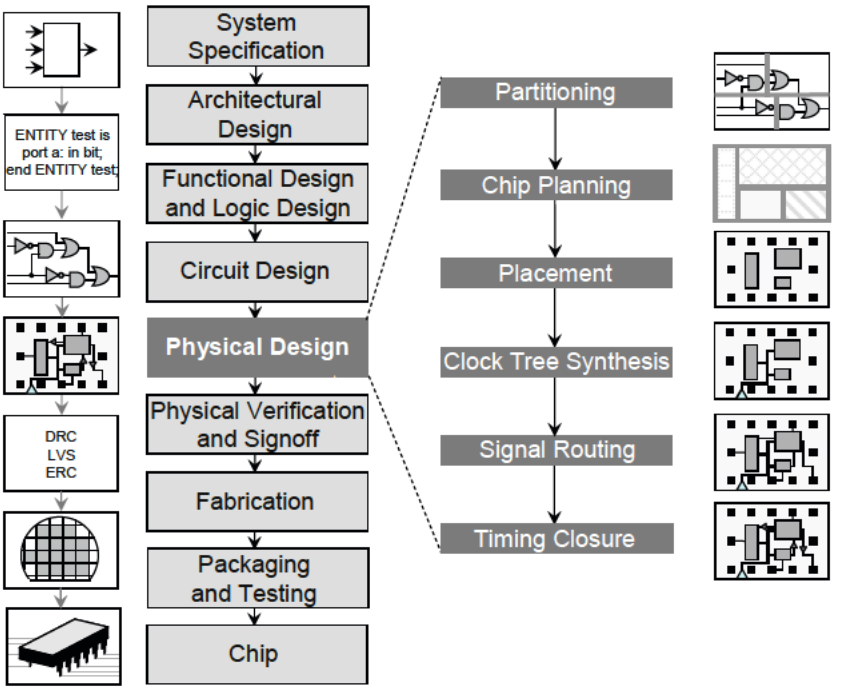


Fig. 1.3 The major steps in the VLSI circuit design flow, with a focus on the physical design steps: partitioning (Chap. 2), chip planning (Chap. 3), placement (Chap. 4), clock tree synthesis (Chap. 7), routing (Chaps. 5-6), and timing closure (Chap. 8).

The steps of the VLSI design flow in Fig. 1.3 are discussed in detail below. For further reading on physical design algorithms, see [1.1]. Books [1.6], [1.9], [1.11] and [1.12] cover other specialized topics that are not addressed here.

System specification. Chip architects, circuit designers, product marketers, operations managers, and layout and library designers collectively define the overall goals and high-level requirements of the system. These goals and requirements span functionality, performance, physical dimensions and production technology.

Architectural design. A basic architecture must be determined to meet the system specifications. Example decisions are

- Integration of analog and mixed-signal blocks
- Memory management – serial or parallel – and the addressing scheme
- Number and types of computational cores, such as processors and digital signal processing (DSP) units – and particular DSP algorithms
- Internal and external communication, support for standard protocols, etc.
- Usage of hard and soft intellectual-property (IP) blocks
- Pinout, packaging, and the die-package interface
- Power requirements
- Choice of process technology and layer stacks

Functional and logic design. Once the architecture is set, the functionality and connectivity of each module (such as a processor core) must be defined. During functional design, only the high-level behavior must be determined. That is, each module has a set of inputs, outputs, and timing behavior.

Logic design is performed at the *register-transfer level* (*RTL*) using a *hardware description language* (*HDL*) by means of programs that define the functional and timing behavior of a chip. Two common HDLs are *Verilog* and *VHDL*. HDL modules must be thoroughly simulated and verified.

Logic synthesis tools automate the process of converting HDL into low-level circuit elements. That is, given a Verilog or VHDL description and a technology library, a logic synthesis tool can map the described functionality to a list of signal nets, or *netlist*, and specific circuit elements such as standard cells and transistors.

Circuit design. For the bulk of digital logic on the chip, the logic synthesis tool automatically converts Boolean expressions into what is referred to as a gate-level netlist, at the granularity of standard cells or higher. However, a number of critical, low-level elements must be designed at the transistor level; this is referred to as *circuit design*. Example elements that are designed at the circuit level include static RAM blocks, I/O, analog circuits, high-speed functions (multipliers), and electrostatic discharge (ESD) protection circuits. The correctness of circuit-level design is predominantly verified by circuit simulation tools such as SPICE.

Physical design. During physical design, all design components are instantiated with their geometric representations. In other words, all macros, cells, gates, transistors, etc., with fixed shapes and sizes per fabrication layer are assigned spatial locations (placement) and have appropriate routing connections (routing) completed in metal layers. The result of physical design is a set of manufacturing specifications that must subsequently be verified.

Physical design is performed with respect to design rules that represent the physical limitations of the fabrication medium. For instance, all wires must be a prescribed minimum distance apart and have prescribed minimum width. As such, the design layout must be recreated in (*migrated* to) each new manufacturing technology.

Physical design directly impacts circuit performance, area, reliability, power, and manufacturing yield. Examples of these impacts are discussed below.

- Performance: long routes have significantly longer signal delays.
- Area: placing connected modules far apart results in larger and slower chips.
- Reliability: large number of vias can significantly reduce the reliability of the circuit.
- Power: transistors with smaller gate lengths achieve greater switching speeds at the cost of higher leakage current and manufacturing variability; larger transistors and longer wires result in greater dynamic power dissipation.
- Yield: wires routed too close together may decrease yield due to *electrical shorts* occurring during manufacturing, but spreading gates too far apart may also undermine yield due to longer wires and a higher probability of *opens*.

Due to its high complexity, physical design is split into several key steps (Fig. 1.3).

- *Partitioning* (Chap. 2) breaks up a circuit into smaller subcircuits or modules, which can each be designed or analyzed individually.
- *Floorplanning* (Chap. 3) determines the shapes and arrangement of subcircuits or modules, as well as the locations of external ports and IP or macro blocks.
- *Power and ground routing* (Chap. 3), often intrinsic to floorplanning, distributes power (*VDD*) and ground (*GND*) nets throughout the chip.
- *Placement* (Chap. 4) finds the spatial locations of all cells within each block.
- *Clock network synthesis* (Chap. 7) determines the buffering, gating (e.g., for power management) and routing of the clock signal to meet prescribed skew and delay requirements.
- *Global routing* (Chap. 5) allocates routing resources that are used for connections; example resources include routing tracks in channels and in switchboxes.
- *Detailed routing* (Chap. 6) assigns routes to specific metal layers and routing tracks within the global routing resources.
- *Timing closure* (Chap. 8) optimizes circuit performance by specialized placement and routing techniques.

After detailed routing, *electrically-accurate layout optimization* is performed at a small scale. Parasitic resistances (*R*), capacitances (*C*) and inductances (*L*) are extracted from the completed layout, and then passed to timing analysis tools to check the functional behavior of the chip. If the analyses reveal erroneous behavior or an insufficient design margin (*guardband*) against possible manufacturing and environmental variations, then incremental design optimizations are performed.

The physical design of analog circuits deviates from the above methodology, which is geared primarily toward digital circuits. For analog physical design, the geometric representation of a circuit element is created using *layout generators* or manual drawing. These generators only use circuit elements with known electrical parame-

ters, such as the resistance of a resistor, and accordingly generate the appropriate geometric representation, e.g., a resistor layout with specified length and width.

Physical verification. After physical design is completed, the layout must be fully verified to ensure correct electrical and logical functionality. Some problems found during physical verification can be tolerated if their impact on chip yield is negligible. In other cases, the layout must be changed, but these changes must be minimal and should not introduce new problems. Therefore, at this stage, layout changes are usually performed manually by experienced design engineers.

- *Design rule checking* (*DRC*) verifies that the layout meets all technology-imposed constraints. DRC also verifies layer density for *chemical-mechanical polishing* (*CMP*).
- *Layout vs. schematic* (*LVS*) checking verifies the functionality of the design. From the layout, a netlist is derived and compared with the original netlist produced from logic synthesis or circuit design.
- *Parasitic extraction* derives electrical parameters of the layout elements from their geometric representations; with the netlist, these are used to verify the electrical characteristics of the circuit.
- *Antenna rule checking* seeks to prevent *antenna effects*, which may damage transistor gates during manufacturing plasma-etch steps through accumulation of excess charge on metal wires that are not connected to PN-junction nodes.
- *Electrical rule checking* (*ERC*) verifies the correctness of power and ground connections, and that signal transition times (*slew*), capacitive loads and fan-outs are appropriately bounded.

Both analysis and synthesis techniques are integral to the design of VLSI circuits. Analysis typically entails the modeling of circuit parameters and signal transitions, and often involves the solution of various equations using established numerical methods. The choice of algorithms for these tasks is relatively straightforward, compared to the vast possibilities for syntheses and optimization. Therefore, this book focuses on optimization algorithms used in IC physical design, and does not cover computational techniques used during physical verification and signoff.

Fabrication. The final DRC-/LVS-/ERC-clean layout, usually represented in the *GDSII Stream* format, is sent for manufacturing at a dedicated silicon foundry (*fab*). The handoff of the design to the manufacturing process is called *tapeout*, even though data transmission from the design team to the silicon fab no longer relies on magnetic tape [1.6]. Generation of the data for manufacturing is sometimes referred to as *streaming out*, reflecting the use of GDSII Stream.

At the fab, the design is patterned onto different layers using photolithographic processes. Photomasks are used so that only certain patterns of silicon, specified by the layout, are exposed to a laser light source. Producing an IC requires many masks; modifying the design requires changes to some or all of the masks.

ICs are manufactured on round silicon wafers with diameters ranging from 200 mm (8 inches) to 300 mm (12 inches). The ICs must then be tested and labeled as either *functional* or *defective*, sometimes according to *bins* depending on the functional or parametric (speed, power) tests that have failed. At the end of the manufacturing process, the ICs are separated, or *diced*, by sawing the wafer into smaller pieces.

Packaging and testing. After dicing, functional chips are typically packaged. Packaging is configured early in the design process, and reflects the application along with cost and form factor requirements. Package types include *dual in-line packages* (*DIPs*), *pin grid arrays* (*PGAs*), and *ball grid arrays* (*BGAs*). After a die is positioned in the package cavity, its pins are connected to the package's pins, e.g., with *wire bonding* or solder bumps (*flip-chip*). The package is then sealed.

Manufacturing, assembly and testing can be sequenced in different ways. For example, in the increasingly important *wafer-level chip-scale packaging* (*WLCSP*) methodology, "bumping" with high-density solder bumps that facilitate delivery of power, ground and signals from the package to the die is performed before the wafer is diced. With multi-chip module based integration, chips are usually not packaged individually; rather, they are integrated as bare dice into the MCM, which is packaged separately at a later point. After packaging, the finished product may be tested to ensure that it meets design requirements such as function (input/output relations), timing or power dissipation.

1.3 VLSI Design Styles

Selecting an appropriate circuit-design style is very important because this choice affects time-to-market and design cost. VLSI design styles fall in two categories – *full-custom* and *semi-custom*. Full-custom design is primarily seen with extremely high-volume parts such as *microprocessors* or FPGAs, where the high cost of design effort is amortized over large production volumes. Semi-custom design is used more frequently because it reduces the complexity of the design process, and hence time-to-market and overall cost as well.

The following semi-custom standard design styles are the most commonly used.

- *Cell-based*: typically using standard cells and macro cells, the design has many pre-designed elements such as logic gates that are copied from libraries.
- *Array-based*: typically either gate arrays or FPGAs, the design has a portion of pre-fabricated elements connected by pre-routed wires.

Full-custom design. Among available design styles, a full-custom design style has the fewest constraints during layout generation, e.g., blocks can be placed anywhere on the chip without restriction. This approach usually results in a very compact chip

with highly optimized electrical properties. However, such effort is laborious, time-consuming, and can be error-prone due to a relative lack of automation.

Full-custom design is primarily useful for microprocessors and FPGAs, where the high cost of design effort is amortized over large production volumes, as well as for analog circuits, where extreme care must be taken to achieve matched layout and adherence to stringent electrical performance specifications.

An essential tool for full-custom design is an efficient layout editor that does more than just draw polygons (Fig. 1.4). Many improved layout editors have integrated DRC checkers that continuously verify the current layout. If all design-rule violations are fixed as they occur, the final layout will be DRC-clean by construction.

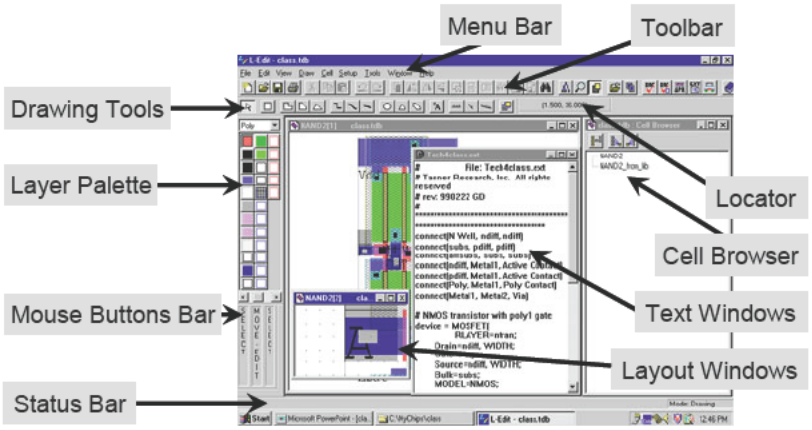
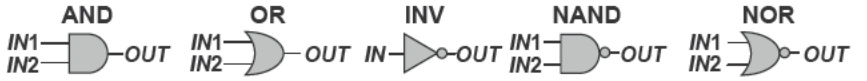


Fig. 1.4 An example of a high-functionality layout editor [From L-Edit, Tanner Research, Inc.].

Standard-cell designs. A digital standard cell is a predefined block that has fixed size and functionality. For instance, an AND cell with two inputs contains a two-input NAND gate followed by an inverter (Fig. 1.5). Standard cells are distributed in cell *libraries*, which are often provided at no cost by foundries and are pre-qualified for manufacturing.



IN1	IN2	OUT
0	0	0
1	0	0
0	1	0
1	1	1

IN1	IN2	OUT
0	0	0
1	0	1
0	1	1
1	1	1

IN	OUT
0	1
1	0

IN1	IN2	OUT
0	0	1
1	0	1
0	1	1
1	1	0

IN1	IN2	OUT
0	0	1
1	0	0
0	1	0
1	1	0

Fig. 1.5 Examples of common digital cells with their input and output behavior.

Standard cells are designed in multiples of a fixed cell height, with fixed locations of *power* (*VDD*) and *ground* (*GND*) ports. Cell widths vary depending on the transistor

network implemented. As a consequence of this restricted layout style, all cells are placeable in rows such that power and ground supply nets are distributed by (horizontal) abutment (Fig. 1.6) The cells' signal ports may be located at the "upper" and "lower" cell boundaries or distributed throughout the cell area [1.3].

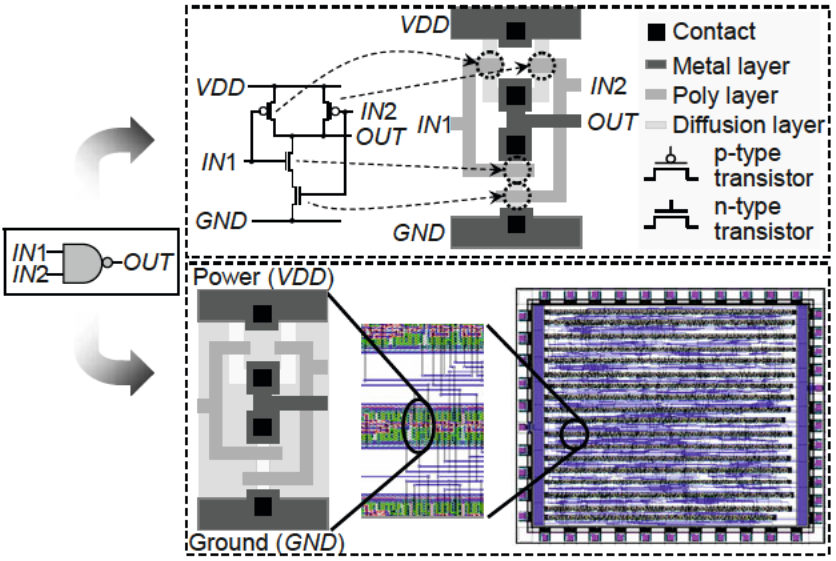


Fig. 1.6 Implementation of a NAND gate using CMOS technology (top), and as a standard cell (lower left), that can be embedded in a VLSI layout (lower right).

Since standard-cell placement has less freedom, the complexity of this design methodology is greatly reduced. This can decrease time-to-market at the cost of such metrics as power efficiency, layout density, or operating frequency, when compared to full-custom designs. Hence, standard-cell based designs, e.g., ASICs, address different market segments than full-custom designs, e.g., microprocessors, FPGAs, and memory products. A substantial initial effort is required to develop the cell library and qualify it for manufacturing.

The routing between standard-cell rows uses either *feedthrough* (empty) cells within rows or available routing tracks across rows (Fig. 1.7). When space between cell rows is available, such regions are called *channels*. These channels, along with the space above the cells, can also be used for routing. *Over-the-cell* (*OTC*) routing has become popular as a way to use multiple metal layers – up to 8-12 in modern designs – in current process technologies. This routing style is more flexible than traditional channel routing. If OTC routing is used, then adjacent standard-cell rows are typically not separated by routing channels but instead share either a power rail or a ground rail (Fig. 1.7 right). OTC routing is prevalent in industry today.

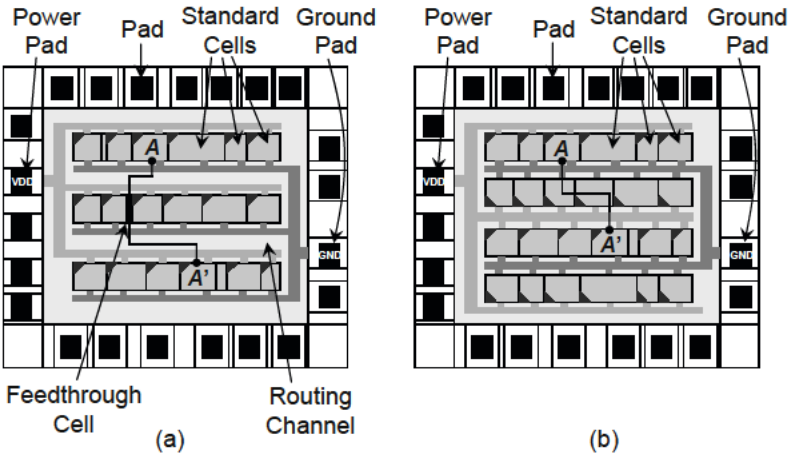


Fig. 1.7 (a) A standard-cell layout with net *A-A'* routed through a feedthrough cell and cell rows separated by channels; each row has its own power and ground rails. (b) A standard-cell layout with net *A-A'* routed using over-the-cell (OTC) routing; cell rows share power and ground rails, which requires alternating cell orientations. Designs with more than three metal layers use OTC routing.

Macro cells. Macro cells are typically larger pieces of logic that perform a reusable functionality. They can range from simple (a couple of standard cells) to highly complex (entire subcircuits reaching the scale of an embedded processor or memory block), and can vary greatly with respect to their shapes and sizes. In most cases, macro cells can be placed anywhere in the layout area with the goals of optimizing routing distance or electrical properties of the design.

Due to the increasing popularity of reusing optimized modules, macro cells, such as adders and multipliers, have become popular. In some cases, almost the entire functionality of a design can be assembled from pre-existing macros; this calls for *top-level assembly*, through which various subcircuits, e.g., analog blocks, standard-cell blocks, and "glue" logic, are combined with individual cells, e.g., buffers, to form the highest hierarchical level of a complex circuit (Fig. 1.8).

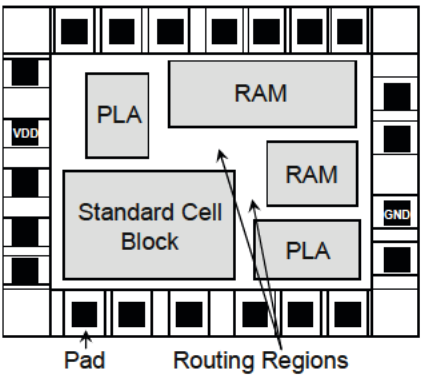


Fig. 1.8 Example layout with macro cells.

Gate arrays. Gate arrays are silicon chips with standard logic functionality, e.g., NAND and NOR, but no connections. The *interconnect* (routing) layers are added later after the chip-specific requirements are known. Since the gate arrays are not initially customized, they can be mass-produced. Then, the time-to-market of gate array-based designs is mostly constrained by the fabrication of interconnects. This makes gate array-based designs cheaper and faster to produce than standard cell-based or macro cell-based designs, particularly for low production volumes.

The layout of gate arrays is greatly restricted, so as to simplify modeling and design. Due to this limited freedom, wire-routing algorithms can be very straightforward. Only the following two tasks are needed.

- *Intracell* routing: creating a cell (logic block) by, e.g., connecting certain transistors to implement a NAND gate. Common gate connections are typically located in cell libraries.
- *Intercell* routing: connecting the logic blocks to form nets from the netlist.

During physical design of gate arrays, (1) cells are selected from what is available on the chip, and (2) since the demand for routing resources depends on the placement configuration, a bad placement may lead to failures at the routing stage. Several variants and extensions of the traditional gate-array style are now available.

Field-programmable gate arrays (FPGAs). In an FPGA, both logic elements and interconnects come prefabricated, but can be configured by users through switches (Fig. 1.9). *Logic elements* (*LEs*) are implemented by *lookup tables* (*LUTs*), each of which can represent any k-input Boolean function, e.g., $k = 4$ or $k = 5$. Interconnect is configured using *switchboxes* (*SBs*) that join wires in adjacent routing channels. Configurations of LUTs and switchboxes are read from external storage and stored locally in memory cells. The main advantage of FPGAs is their customization without the involvement of a fabrication facility. This dramatically reduces design costs, up-front investment and time-to-market. However, FPGAs typically run much slower and dissipate more power than ASICs [1.8]. Above certain production volumes, e.g., millions of chips, FPGAs become more expensive than ASICs since the non-recurring design and manufacturing costs of ASICs are amortized [1.2].

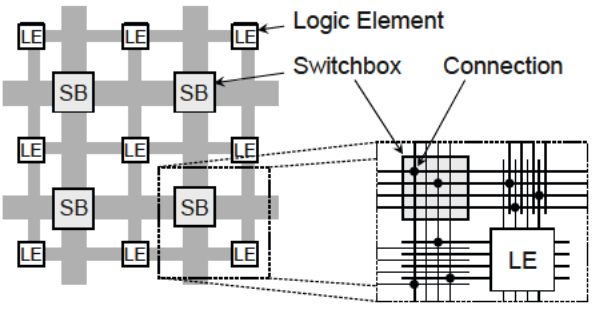


Fig. 1.9 Logic elements (LEs) are connected through switchboxes (SBs) that form a programmable routing network.

Structured-ASICs (channel-less gate arrays). A channel-less gate array s similar to an FPGA, except that the cells are usually not configurable. Unlike traditional gate arrays, sea of gate designs have many interconnect layers, removing the need for routing channels and thus improving density. The interconnects (sometimes, only via layers) are mask-programmed in the foundry, and are not field-programmable. The modern incarnation of the channel-less gate array is the *structured ASIC*.

1.4 1.4 Layout Layers and Design Rules

The gates and interconnects of an integrated circuit are formed using standard materials that are deposited and patterned on *layout layers*, with the layout patterns themselves conforming to *design rules* that ensure manufacturability, electrical performance and reliability.

Layout layers. Integrated circuits are made up of several different materials, the main ones being

- Single-crystal silicon substrate which is doped to enable construction of n- and p-channel transistors
- Silicon dioxide, which serves as an insulator
- Polycrystalline silicon or *polysilicon*, which forms transistor gates and can serve as an interconnect material
- Either aluminum or copper, which serves as metal interconnect

Silicon serves as the *diffusion layer*. The polysilicon and the aluminum and copper layers are collectively referred to as interconnect layers; the polysilicon is called *poly* and the remaining layers are called *Metal1*, *Metal2*, etc. (Fig. 1.10). *Vias* and *contacts* make connections between the different layers – vias connect metal layers and contacts connect poly and Metal1.

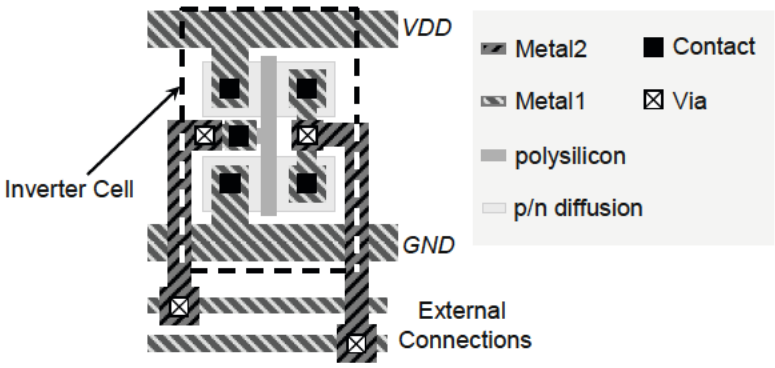


Fig. 1.10 The different layers for a simple inverter cell, showing external connections to the channel below and internal connections.

The wire resistance is usually given as *sheet resistance* in ohms per square (Ω/□). That is, for a given wire thickness, the resistance per square area remains the same – independent of the square size (a higher resistance for a longer length is compensated by the increased width of the square).[1] Hence, the resistance of any rectangular interconnect shape can be easily calculated as the number of unit-square areas multiplied by the sheet resistance of the corresponding layer.

Individual transistors are created by overlapping poly and diffusion layers. Cells, e.g., standard cells, are comprised of transistors but typically include one metal layer.

The routing between cells (Chaps. 5-7) is performed entirely within the metal layers. This is a non-trivial task – not only are poly and Metal1 mostly reserved for cells, but different layers have varying sheet resistances, which strongly affects timing characteristics. For a typical 0.35 μm CMOS process, the sheet resistance of poly is 10 Ω/□**,** that of the diffusion layer is approximately 3 Ω/□, and that of aluminum is 0.06 Ω/□. Thus, poly should be used sparingly, and most of the routing done in metal layers.

Routing through multiple metal layers requires vias. For the same 0.35 μm process, the typical resistance of a via between two metal layers is 6 Ω, while that of a contact is significantly higher – 20 Ω. As technology scales, modern copper interconnects become highly resistive due to smaller cross sections, grain effects that cause electron scattering, and the use of barrier materials to prevent reactive copper atoms from leaching into the rest of the circuit. In a typical 65 nm CMOS process, the sheet resistance of poly is 12 Ω/□, that of the diffusion layer is 17 Ω/□, and that of the copper Metal1 layer is 0.16 Ω/□. Via and contact resistances are respectively 1.5 Ω and 22 Ω in a typical 65 nm process.

Design rules. An integrated circuit is fabricated by shining laser light through *masks*, where each mask defines a certain layer pattern. For a mask to be effective, its layout pattern must meet specific technology constraints. These constraints, or *design rules*, ensure that (1) the design can be fabricated in the specified technology and (2) the design will be electrically reliable and feasible. Design rules exist both for each individual layer and for interactions across multiple layers. In particular, transistors require structural overlaps of poly and diffusion layers.

Though design rules are complex, they can be broadly grouped into three categories.

– *Size rules*, such as *minimum width*: The dimensions of any component (shape), e.g., length of a boundary edge or area of the shape, cannot be smaller than given minimum values (*a* in Fig. 1.11). These values vary across different metal layers.

[1] Since [length/width] is dimensionless, sheet resistance is measured in the same units as resistance (ohms). However, to distinguish it from resistance, it is specified in ohms per square (Ω /□).

- *Separation rules*, such as *minimum separation*: Two shapes, either on the same layer (*b* in Fig. 1.11) or on adjacent layers (*c* in Fig. 1.11), must be a minimum (rectilinear or Euclidean diagonal) distance apart (*d* in Fig. 1.11).
- *Overlap rules*, such as *minimum overlap*: Two connected shapes on adjacent layers must have a certain amount of overlap (*e* in Fig. 1.11) due to inaccuracy of mask alignment to previously-made patterns on the wafer.

To enable technology scaling, fabrication engineers use a standard unit *lambda* (λ) to represent the minimum size of any design feature [1.9]. Thus, design rules are specified in multiples of λ, which facilitates grid-based layout with base length λ. Such a framework is very convenient, in that technology scaling only affects the value of λ. However, as the size of transistors decreases, the λ metric is becoming less meaningful, as some physical and electrical properties no longer follow such ideal scaling.

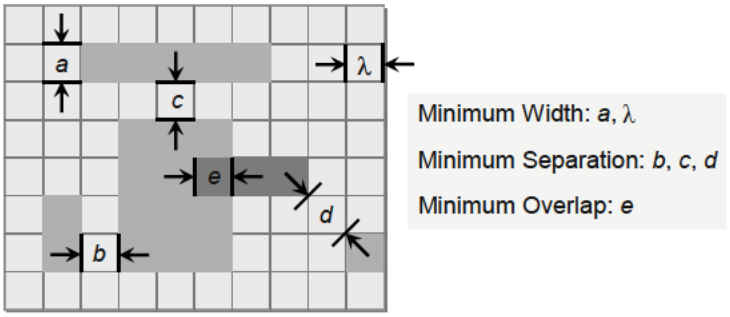


Fig. 1.11 Several classes of design rules. The grid's granularity is λ, the smallest meaningful technology-dependent unit of length.

1.5 1.5 Physical Design Optimizations

Physical design is a complex *optimization problem* with several different objectives, such as minimum chip area, minimum wirelength, and minimum of vias. Common goals of optimization are to improve circuit performance, reliability, etc. How well the optimization goals are met determines the quality of the layout.

Different optimization goals (1) may be difficult to capture within algorithms and (2) may conflict with each other. However, tradeoffs among multiple goals can often be expressed concisely by an *objective function*. For instance, wire routing can optimize

$$w_1 \cdot A + w_2 \cdot L$$

where A is the chip area, L is the total wirelength, and w_1 and w_2 are *weights* that represent the relative importance of A and L. In other words, the weights influence the impact of each objective goal on the overall cost function. In practice, $0 \leq w_1 \leq 1$, $0 \leq w_2 \leq 1$, and $w_1 + w_2 = 1$.

During layout optimization, three types of *constraints* must be met.

- *Technology constraints* enable fabrication for a specific technology node and are derived from technology restrictions. Examples include minimum layout widths and spacing values between layout shapes.
- *Electrical constraints* ensure the desired electrical behavior of the design. Examples include meeting maximum timing constraints for signal delay and staying below maximum coupling capacitances.
- *Geometry* (*design methodology*) *constraints* are introduced to reduce the overall complexity of the design process. Examples include the use of preferred wiring directions during routing, and the placement of standard cells in rows.

As technology scales further, electrical effects have become increasingly significant. Thus, many types of electrical constraints have been introduced recently to ensure correct behavior. Various constraints not required at earlier technology nodes are necessary for modern designs. Such constraints may limit *coupling capacitance* to ensure *signal integrity*, prevent *electromigration* effects in interconnects, and prevent adverse temperature-related phenomena.

A basic challenge is that new electrical effects are not easily translated into new geometric rules for layout design. For instance, is signal delay best minimized by reducing total wirelength or by reducing coupling capacitance between the routes of different nets? Such a question is further complicated by the fact that routes on other metal layers, as well as their switching activity, also affect signal delay. Although only loose geometric rules can be defined, electrical properties can be accurately extracted from layout, and physical simulation enables precise estimation of timing, noise and power. This allows designers to assess the impact of layout optimizations.

In summary, difficulties encountered when optimizing layout include the following.

- Optimization goals may conflict with each other. For example, minimizing wirelength too aggressively can result in a congested region, and increase the number of vias.
- Constraints often lead to discontinuous, qualitative effects even when objective functions remain continuous. For example, the floorplan design might permit only some of the bits of a 64-bit bus to be routed with short wires, while the remaining bits must be detoured.
- Constraints, due to scaling and increased interconnect requirements, are tightening, with new constraint types added for each new technology node.

These difficulties motivate the following rules of thumb.

- Each design style requires its own custom flow. That is, there is no universal EDA tool that supports all design styles.
- When designing a chip, imposing geometric constraints can potentially make the problem easier at the expense of layout optimization. For instance, a row-based standard-cell design is much easier to implement than a full-custom layout, but the latter could achieve significantly better electrical characteristics.
- To further reduce complexity, the design process is divided into sequential steps. For example, placement and routing are performed separately, each with specific optimization goals and constraints that are evaluated independently.
- When performing fundamental optimizations, the choice is often between (1) an abstract model of circuit performance that admits a simple computation, or (2) a realistic model that is computationally intractable. When no efficient algorithm or closed-form expression is available to obtain a globally optimal solution, the use of heuristics is a valid and effective option (Sec. 1.6).

1.6 1.6 Algorithms and Complexity

A key criterion for assessing any algorithm is its *runtime complexity*, the time required by the algorithm to complete as a function of some natural measure of the problem size. For example, in block placement, a natural measure of problem size is the number of blocks to be placed, and the time $t(n)$ needed to place n blocks can be expressed as

$$t(n) = f(n) + c$$

where $f(0) = 0$ and c is a fixed amount of "overhead" that is required independently of input size, e.g., during initialization.

While other measures of algorithm complexity such as memory ("space") are also of interest, runtime is the most important complexity metric for IC physical design algorithms. Complexity is represented in an *asymptotic* sense, with respect to the input size n, using *big-Oh notation* or $O(\ldots)$. Formally, the runtime $t(n)$ is *order* $f(n)$, written as $t(n) = O(f(n))$ when

$$\lim_{n\to\infty}\left|\frac{t(n)}{f(n)}\right| = k$$

where k is a real number. For example, if $t(n) = 7n! + n^2 + 100$, then $t(n) = O(n!)$ because $n!$ is the fastest growing term as $n \to \infty$.The real number k for $f(n) = n!$ is

$$k = \lim_{n\to\infty}\left|\frac{7n! + n^2 + 100}{n!}\right| = \lim_{n\to\infty}\left|\frac{7n!}{n!} + \frac{n^2}{n!} + \frac{100}{n!}\right| = 7 + 0 + 0 = 7$$

Placement problems and their associated computational complexities include

- Place n cells in a single row and return the wirelength: $O(n)$
- Given a single-row placement of n cells, determine whether the wirelength can be improved by swapping one pair of cells: $O(n^2)$
- Given a single-row placement of n cells, determine whether the wirelength can be improved by permuting a group of three cells at a time: $O(n^3)$
- Place n cells in a single row so as to minimize the wirelength: $O(n! \cdot n)$ with a naive algorithm

Example: Exhaustively Enumerating All Placement Possibilities
Given: n cells.
Task: find a linear (single-row) placement of n cells with minimum total wirelength by using exhaustive enumeration.

Solution:
The solution space consists of $n!$ placement options. If generating and evaluating the wirelength of each possible placement solution takes 1 microsecond (μs) and $n = 20$, the total time needed to find an optimal solution would be 77,147 years!

The first three placement tasks are considered *scalable*, since their complexities can be written as $O(n^p)$ or $O(n^p \log n)$, where p is a small integer, usually $p \in \{1,2,3\}$. Algorithms having complexities where $p > 3$ are often considered *not scalable*. Furthermore, the last problem is considerably more difficult and is impractical for even moderate values of n, despite the existence of clever algorithms. A number of important problems have best-known algorithm complexities that grow exponentially with n, e.g., $O(n!)$, $O(n^n)$, and $O(e^n)$. Many of these problems are known to be *NP-hard*,[2] and no polynomial-time algorithms are currently known that solve these problems. Thus, for such problems, no known algorithms can ensure, in a time-efficient manner, that they will return a *globally optimal* solution.

Chaps. 2-8 all deal with physical design problems that are NP-hard. For these problems, *heuristic algorithms* are used to find near-optimal solutions within practical runtime limits. In contrast to conventional algorithms, which are guaranteed to produce an optimal (valid) solution in a known amount of time, heuristics may produce inferior solutions. Algorithms that have poor worst-case complexity, but produce optimal solutions in all practical cases, are also considered heuristics. The primary goal of algorithm development for EDA is to construct heuristics that can quickly

[2] NP stands for *non-deterministic polynomial time*, and refers to the ability to validate in polynomial time any solution that was "non-deterministically guessed". NP-hard problems are at least as hard as the most difficult NP problems. For further reading, see [1.4] and [1.7].

produce near-optimal solutions on large and complex commercial designs by exploiting the typical features of such designs. Such heuristics often incorporate conventional algorithms for subtasks that can be solved optimally.

Types of algorithms. Many physical design problems, e.g., placement and routing, are NP-hard, so solving them with optimal, worst-case polynomial-time algorithms is unlikely. Many heuristics have been developed for these problems, the quality of which can be assessed based on (1) runtime and (2) solution quality, measured by *suboptimality* (difference from optimal solutions).

Heuristic algorithms can be classified as

- *Deterministic*: All decisions made by the algorithm are repeatable, i.e., not random. One example of a deterministic heuristic is Dijkstra's shortest path algorithm (Sec. 5.6.3).
- *Stochastic*: Some decisions made by the algorithm are made randomly, e.g., using a pseudo-random number generator. Thus, two independent runs of the algorithm will produce two different solutions with high probability. One example of a stochastic algorithm is simulated annealing (Sec. 3.5.3).

In terms of structure, a heuristic algorithm can be

- *Constructive*: The heuristic starts with an initial, incomplete (partial) solution and adds components until a complete solution is obtained.
- *Iterative*: The heuristic starts with a complete solution and repeatedly improves the current solution until a preset termination criterion is reached.

Physical design algorithms often employ both constructive and iterative heuristics. For instance, a constructive heuristic can be used to first generate an initial solution, which is then refined by an iterative algorithm (Fig. 1.12).

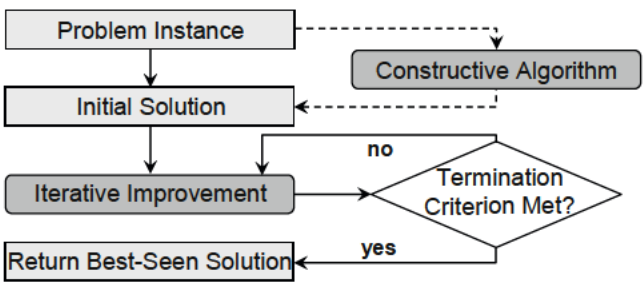


Fig. 1.12 Example of a heuristic that has both constructive and iterative stages. The constructive stage creates an initial solution that is refined by the iterative stage.

When the solution space is represented by a *graph* structure consisting of *nodes* and *edges* (Sec. 1.7), algorithms for search and optimization can be classified in another

way, as described below. The graph structure can be *explicit*, as in wire routing, or *implicit*, with edges representing small differences between possible solutions, e.g., swapping a pair of adjacent standard cells in placement.

- *Breadth-first search* (*BFS*): When searching for goal node T from starting node S_0, the algorithm checks all adjacent nodes S_1. If the goal T is not found in S_1, the algorithm searches all of S_1's adjacent nodes S_2. This process continues, resembling expansion of a "wave-front", until T is found or all nodes have been searched.
- *Depth-first search* (*DFS*): From the starting node S_0, the algorithm checks nodes in order of increasing depth, i.e., traversing as far as possible and as soon as possible. In contrast to BFS, the next-searched node S_{i+1} is a neighbor of S_i unless all neighbors of S_i have already been searched, in which case the search backtracks to the highest-index location that has an unsearched neighbor. Thus, DFS traverses as far as possible as soon as possible.
- *Best-first search*: The direction of search is based on cost criteria, not simply on adjacency. Every step taken considers a current *cost* as well as the remaining cost to the goal. The algorithm always expands or grows from the current best known node or solution. An example is Dijkstra's algorithm (Sec. 5.6.3).

Finally, some algorithms used in physical design are *greedy*. An initial solution is transformed into another solution only if the new solution is strictly better than the previous solution. Such algorithms find *locally optimal* solutions. For further reading on the theory of algorithms and complexity, see [1.4].

Solution quality. Given that most physical design algorithms are heuristic in nature, the assessment of solution quality is difficult. If the optimal solution is known, then the heuristic solution can be judged by its *suboptimality* ε with respect to the optimal solution

$$\varepsilon = \frac{\left| cost(S_H) - cost(S_{opt}) \right|}{cost(S_{opt})}$$

where $cost(S_H)$ is the cost of the heuristic solution S_H and $cost(S_{opt})$ is the cost of the optimal solution S_{opt}. This notion applies to only a tiny fraction of design problems, in that optimal solutions are known only for small (or artificially-created) instances. On the other hand, *bounds* on suboptimality can sometimes be proven for particular heuristics, and can provide useful guidance.

When finding an optimal solution is impractical, as typical for modern designs, heuristic solutions are tested across a suite of *benchmarks*. These sets of (non-trivial) problem instances represent different corner cases, as well as common cases, and are inspired by either industry or academic research. They enable assessment of a given heuristic's scalability and solution quality against previously-obtained heuristic solutions.

1.7 1.7 Graph Theory Terminology

Graphs are heavily used in physical design algorithms to describe and represent layout topologies. Thus, a basic understanding of graph theory terminology is vital to understanding how the optimization algorithms work. The following is a list of basic terms; subsequent chapters will introduce specialized terminology.

A *graph* $G(V,E)$ is made up of two sets – the set of *nodes* or *vertices* (elements), denoted as V, and the set of *edges* (relations between the elements), denoted as E (Fig. 1.13(a)). The *degree* of a node is the number of its incident edges.

A *hypergraph* consists of nodes and *hyperedges*, with each hyperedge being a subset of two or more nodes. Note that a graph is a hypergraph in which every hyperedge has cardinality two. Hyperedges are commonly used to represent multi-pin nets or multi-point connections within circuit hypergraphs (Fig. 1.13(b)).

A *multigraph* (Fig. 1.13(c)) is a graph that can have more than one edge between two given nodes. Multigraphs can be used to represent varying *net weights*; an alternative is to use an edge-weighted graph representation, which is more compact and supports non-integer weights.

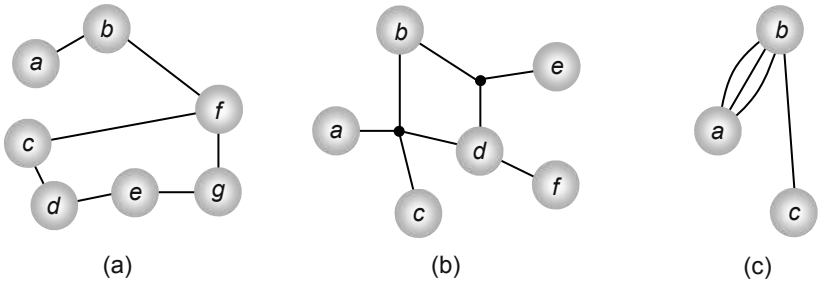


Fig. 1.13 (a) A graph with seven edges. (b) A hypergraph with three hyperedges having sizes four, three and two respectively. (c) A multigraph with four edges, where *a-b* has weight = 3.

A *path* between two nodes is an ordered sequence of edges from the start node to the end node (*a-b-f-g-e* in Fig. 1.13(a)).

A *cycle* (*loop*) is a closed path that starts and ends at the same node (*c-f-g-e-d-c* in Fig. 1.13(a)).

An *undirected graph* is a graph that represents only unordered node relations and does not have any directed edges. A *directed graph* is a graph where the direction of the edge denotes a specific ordered relation between two nodes. For example, a signal might be generated at the output pin of one gate and flow to an input pin of another gate – but not the other way around. Directed edges are drawn as arrows starting from one node and pointing to the other.

A directed graph is *cyclic* if it has at least one directed cycle (*c-f-g-d-c* in Fig. 1.14(a) or *a-b-a* in Fig. 1.14(b)). Otherwise, it is *acyclic*. Several important EDA algorithms operate on *directed acyclic graph* (*DAG*) representations of design data (Fig. 1.14(c)).

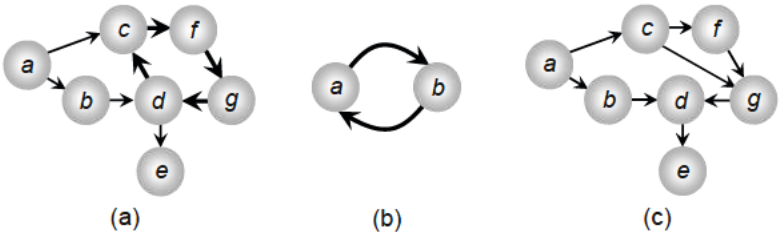


Fig. 1.14 (a) A directed graph with a cycle *c-f-g-d-c*. (b) A directed graph with a cycle *a-b-a*. (c) A directed acyclic graph (DAG).

A *complete graph* is a graph of n nodes with

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

edges, one edge between each pair of nodes, i.e., each node is connected by an edge to every other node.

A *connected graph* is a graph with at least one path between each pair of nodes.

A *tree* is a graph whose n nodes are connected by $n - 1$ edges. There are two types of trees: undirected and directed (trees that have a *root*). Both types are shown in Fig. 1.15. In an undirected tree, any node with only one incident edge is a *leaf*. In a directed tree, the root has no incoming edge, and a leaf is a node with no outgoing edge. In a directed tree, there exists a unique path from the root to any other node.

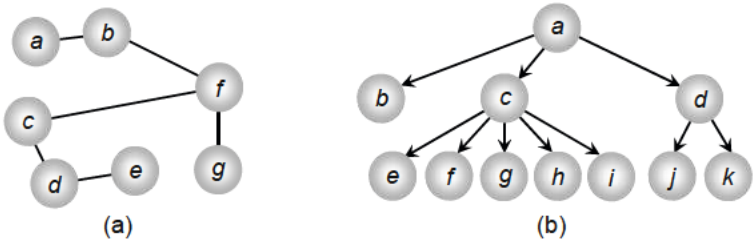


Fig. 1.15 (a) A undirected graph with leaves *a*, *e* and *g*, and maximum node degree 3. (b) A directed tree with root *a* and leaves *b* and *e-k*.

A *spanning tree* in a graph $G(V,E)$ is a connected, acyclic subgraph G' contained within G that includes (spans) every node $v \in V$.

A *minimum spanning tree* (*MST*) is a spanning tree with the smallest possible sum of edge costs (i.e., edge lengths).

A *rectilinear minimum spanning tree* (*RMST*) is an MST where all edge lengths correspond to the *Manhattan* (rectilinear) distance metric (Fig. 1.16(a)).

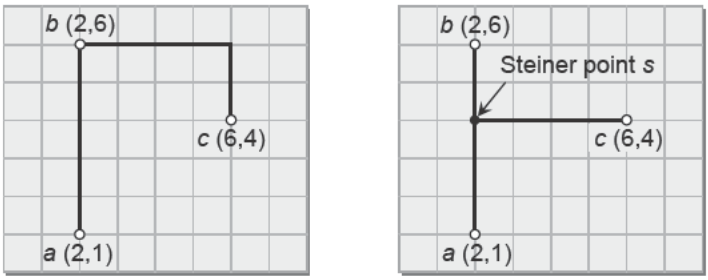


Fig. 1.16 (a) Rectilinear minimum spanning tree (RMST) connecting points *a*-*c* with tree cost = 11. (b) Rectilinear minimum Steiner tree (RSMT) connecting points *a*-*c* with tree cost = 9.

The *Steiner tree*, named after *J. Steiner* (1796-1863), is a generalization of the spanning tree. In addition to the original nodes, Steiner trees have *Steiner points* (Fig. 1.16(b)). Edges are allowed to connect to these points as well as to the original nodes. The incorporation of Steiner points can reduce the total edge cost of the tree to below that of an RMST. A *Steiner minimum tree* (*SMT*) has minimum total edge cost over all Steiner trees. If constructed in the Manhattan plane using only horizontal and vertical segments, the SMT is a *rectilinear Steiner minimum tree* (*RSMT*).

1.8 1.8 Common EDA Terminology

The following is a brief list of important and common terms used in EDA. Many of these terms will be discussed in greater detail in subsequent chapters.

Logic design is the process of mapping the HDL (typically, register-transfer level) description to circuit gates and their connections at the netlist level. The result is usually a netlist of cells or other basic circuit components and connections.

Physical design is the process of determining the geometrical arrangement of cells (or other circuit components) and their connections within the IC layout. The cells' electrical and physical properties are obtained from library files and technology information. The connection topology is obtained from the netlist. The result of physical design is a geometrically and functionally correct representation in a standard file format such as GDSII Stream.

Physical verification checks the correctness of the layout design. This includes verifying that the layout

- Complies with all technology requirements – Design Rule Checking (DRC)
- Is consistent with the original netlist – Layout vs. Schematic (LVS)
- Has no antenna effects – Antenna Rule Checking
- Complies with all electrical requirements – Electrical Rule Checking (ERC)

A *component* is a basic functional element of a circuit. Examples include transistors, resistors, and capacitors.

A *module* is a circuit partition or a grouped collection of components.

A *block* is a module with a shape, i.e., a circuit partition with fixed dimensions.

A *cell* is a logical or functional unit built from various components. In digital circuits, cells commonly refer to gates, e.g., INV, AND-OR-INVERTER (AOI), NAND, NOR. In general, the term is used to refer to either standard cells or macros.

A *standard cell* is a cell with a pre-determined functionality. Its height is a multiple of a library-specific fixed dimension. In the standard-cell methodology, the logic design is implemented with standard cells that are arranged in rows.

A *macro cell* is a cell without pre-defined dimensions. This term may also refer to a large physical layout, possibly containing millions of transistors, e.g., an SRAM or CPU core, and possibly having discrete dimensions, that can be incorporated into the IC physical design.

A *pin* is an electrical terminal used to connect a given component to its external environment. At the level of block-to-block connections (internal to the IC), I/O pins are present on lower-level metal layers such as Metal1, Metal2 and Metal3. A *pad* is an electrical terminal used to connect externally to the IC. Often, bond pads are present on topmost metal layers and interface between external connections (such as to other chips) and internal connections.

A *layer* is a manufacturing process level in which design components are patterned. During physical design, circuit components are assigned to different layers, e.g., transistors are assigned to poly and active layers, while interconnects are assigned to poly and metal layers and are routed according to the netlist.

A *contact* is a direct connection between silicon (poly or another active level) and a metal layer, typically Metal1. Contacts are often used inside cells.

A *via* is a connection between metal layers, usually to connect routing structures on different layers.

A *net* or *signal* is a set of pins or terminals that must be connected to have the same potential.

Supply nets are power (*VDD*) and ground (*GND*) nets that provide current to cells.

A *netlist* is the collection of all signal nets and the components that they connect in a design, or, a list of all the nets and connecting pins of a subsection of the design. That is, netlists can be organized as (1) *pin-oriented* – each design component has a list of associated nets (Fig. 1.17 center), or (2) *net-oriented* – each net has a list of associated design components (Fig. 1.17 right). Netlists are created during logic synthesis and are a key input to physical design.

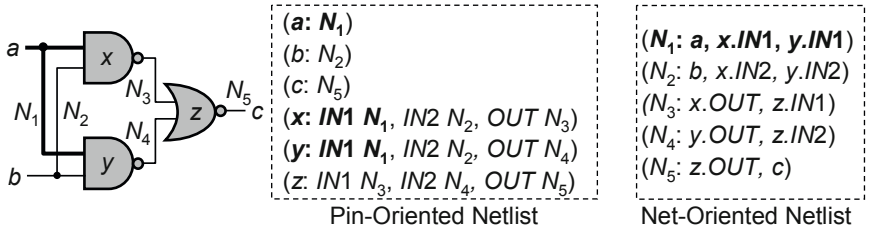


Fig. 1.17 Pin-oriented (center) and net-oriented (right) netlist examples for the sample circuit (left).

A *net weight* $w(net)$ is a numerical (typically integer) value given to a net *net* (or edge *edge*) to indicate its importance or criticality. Net weights are used primarily during placement, e.g., to minimize distance between cells that are connected by edges with high net weights, and routing, e.g., to set the priority of a net.

The *connectivity degree* or *connection cost* $c(i,j)$ between cells i and j for unweighted nets is the number of connections between i and j. With weighted nets, $c(i,j)$ is the sum of the individual connection weight between i and j.

The *connectivity* $c(i)$ of cell $cell_i$ is given by

$$c(i) = \sum_{j=1}^{|V|} c(i, j)$$

where $|V|$ is the number of cells in the netlist, and $c(i,j)$ is the connectivity degree between cells i and j. For example, cell y in Fig. 1.18 has $c(y) = 5$ if each net's weight equals 1.

A *connectivity graph* is a representation of the netlist as a graph. Cells, blocks and pads correspond to nodes, while their connections correspond to edges (Fig. 1.18). A p-pin net is represented by $\binom{p}{2}$ total connections between its nodes. Multiple edges between two nodes imply a stronger (*weighted*) connection.

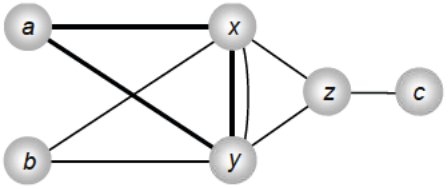


Fig. 1.18 Connectivity graph of the circuit in Fig. 1.17.

The *connectivity matrix* C is an $n \times n$ matrix that represents the circuit connectivity over n cells. Each element $C[i][j]$ represents the connection cost $c(i,j)$ between cells i and j (Fig. 1.19). Since C is symmetric, $C[i][j] = C[j][i]$ for $1 \le i, j \le m$. By definition, $C[i][i] = 0$ for $1 \le i \le m$, since a cell i does not meaningfully connect with itself.

	a	b	x	y	z	c
a	0	0	1	1	0	0
b	0	0	1	1	0	0
x	1	1	0	2	1	0
y	1	1	2	0	1	0
z	0	0	1	1	0	1
c	0	0	0	0	1	0

Fig. 1.19 The connectivity matrix of the circuit from Fig. 1.17. The entry $C[x][y] = C[y][x] = 2$ because both N_1 and N_2 contribute one unit.

The *Euclidean distance* metric between two points P_1 (x_1,y_1) and P_2 (x_2,y_2) corresponds to the length of the line segment between P_1 and P_2 (Fig. 1.20). In the coordinate plane, the Euclidean distance is

$$d_E(P_1,P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The *Manhattan distance* metric between two points P_1 (x_1,y_1) and P_2 (x_2,y_2) is the sum of the horizontal and vertical displacements between P_1 and P_2 (Fig. 1.20). In the coordinate plane, the Manhattan distance is

$$d_M(P_1,P_2) = |x_2 - x_1| + |y_2 - y_1|$$

In Fig. 1.20, the Euclidean distance between P_1 and P_2 is $d_E(P_1,P_2) = 5$, whereas the Manhattan distance between P_1 and P_2 is $d_M(P_1,P_2) = 7$.

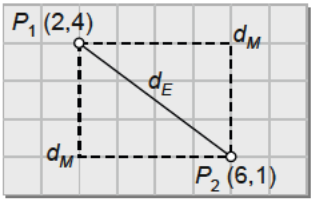


Fig. 1.20 Distance between two points P_1 and P_2 according to Euclidean (d_E) and Manhattan (d_M) distance metrics.

Chapter 1 References

[1.1] C. J. Alpert, D. P. Mehta and S. S. Sapatnekar, eds., *Handbook of Algorithms for Physical Design Automation*, CRC Press, 2009.

[1.2] D. Chen, J. Cong and P. Pan, "FPGA Design Automation: A Survey", *Foundations and Trends in EDA* 1(3) (2006), pp. 195-330.

[1.3] D. Clein, *CMOS IC Layout: Concepts, Methodologies, and Tools*, Newnes, 1999.

[1.4] T. Cormen, C. Leiserson, R. Rivest and C. Stein, *Introduction to Algorithms, 2nd Edition*, McGraw Hill, 2003.

[1.5] *International Technology Roadmap for Semiconductors*, 2009 edition, www.itrs.net/.

[1.6] H. Kaeslin, *Digital Integrated Circuit Design: From VLSI Architectures to CMOS Fabrication*, Cambridge University Press, 2008.

[1.7] B. Korte and J. Vygen, *Combinatorial Optimization: Theory and Algorithms*, Springer, 3rd edition, 2006.

[1.8] I. Kuon and J. Rose, "Measuring the Gap Between FPGAs and ASICs", *IEEE Trans. on CAD* 26(2) (2007), pp. 203-215.

[1.9] C. Mead and L. Conway, *Introduction to VLSI Systems*, Addison-Wesley, 1979.

[1.10] G. Moore, "Cramming More Components Onto Integrated Circuits", *Electronics* 38(8) (1965).

[1.11] L. Scheffer, L. Lavagno and G. Martin, eds., *EDA for IC Implementation, Circuit Design, and Process Technology*, CRC Press, 2006.

[1.12] L.-T. Wang, Y.-W. Chang and K.-T. Cheng, eds., *Electronic Design Automation: Synthesis, Verification, and Test (Systems on Silicon)*, Morgan Kaufmann, 2009.

Chapter 2

Netlist and System Partitioning

2

2

2 Netlist and System Partitioning 33
2.1 Introduction 33
2.2 Terminology 34
2.3 Optimization Goals 35
2.4 Partitioning Algorithms 36
2.4.1 Kernighan-Lin (KL) Algorithm 36
2.4.2 Extensions of the Kernighan-Lin Algorithm 41
2.4.3 Fiduccia-Mattheyses (FM) Algorithm 41
2.5 A Framework for Multilevel Partitioning 47
2.5.1 Clustering 48
2.5.2 Multilevel Partitioning 48
2.6 System Partitioning onto Multiple FPGAs 50
Chapter 2 Exercises 53
Chapter 2 References 54

2 Netlist and System Partitioning

The design complexity of modern integrated circuits has reached unprecedented scale, making full-chip layout, FPGA-based emulation and other important tasks increasingly difficult. A common strategy is to *partition* or divide the design into smaller portions, each of which can be processed with some degree of independence and parallelism. A *divide-and-conquer* strategy for chip design can be implemented by laying out each block individually and reassembling the results as *geometric partitions*. Historically, this strategy was used for manual partitioning, but became infeasible for large netlists. Instead, manual partitioning can be performed in the context of system-level modules by viewing them as single entities, in cases where hierarchical information is available. In contrast, automated *netlist partitioning* (Secs. 2.1-2.4) can handle large netlists and can redefine a physical hierarchy of an electronic system, ranging from boards to chips and from chips to blocks. Traditional netlist partitioning can be extended to *multilevel* partitioning (Sec. 2.5), which can be used to handle large-scale circuits and system partitioning on FPGAs (Sec. 2.6).

2.1 Introduction

A popular approach to decrease the design complexity of modern integrated circuits is to *partition* them into smaller *modules*. These modules can range from a small set of electrical components to fully functional integrated circuits (ICs). The partitioner divides the circuit into several subcircuits (partitions or blocks) while minimizing the number of connections between partitions, subject to design constraints such as maximum partition sizes and maximum path delay.

If each block is implemented independently, i.e., without considering other partitions, then connections between these partitions may negatively affect the overall design performance such as increased circuit delay or decreased reliability. Moreover, a large number of connections between partitions may introduce inter-block dependencies that hamper design productivity.[1] Therefore, the primary goal of partitioning is to divide the circuit such that the number of connections between subcircuits is minimized (Fig. 2.1). Each partition must also meet all design constraints. For example, the amount of logic in a partition can be limited by the size of an FPGA chip. The number of external connections of a partition may also be limited, e.g., by the number of I/O pins in the chip package.

[1] The empirical observation known as *Rent's rule* suggests a power-law relationship between the number of cells n_G and the number of external connections $n_P = t \cdot n_G{}^r$, for any subcircuit of a "well-designed" system. Here, t is the number of pins per cell and r, referred to as the *Rent's exponent* or the *Rent parameter*, is a constant < 1. In particular, Rent's rule quantifies the prevalence of short wires in ICs, which is consistent with a hierarchical organization.

A. B. Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure,*
DOI 10.1007/978-90-481-9591-6_2, © Springer Science+Business Media B.V. 2011

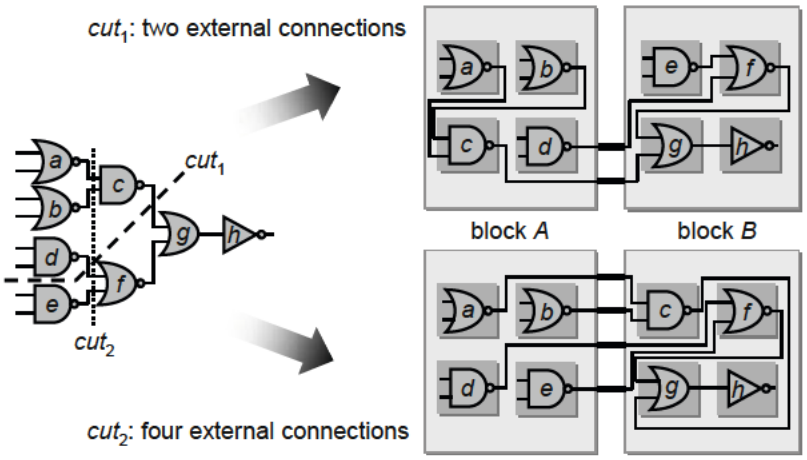


Fig. 2.1 Two different partitions induced by cuts cut_1 and cut_2, producing two and four external connections, respectively.

2.2 Terminology

The following are common terms relevant to netlist partitioning. Terminology relating to specific algorithms, such as the Kernighan-Lin algorithm (Sec. 2.4.1), will be introduced and defined in their respective sections.

A *cell* is any logical or functional unit built from components.

A *partition* or *block* is a grouped collection of components and cells.

The *k-way partitioning* problem seeks to divide a circuit into k partitions. Fig. 2.2 illustrates how the partitioning problem can be abstracted using a graph representation, where nodes represent cells, and edges represent connections between cells.

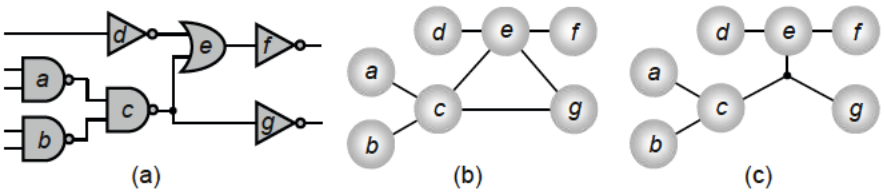


Fig. 2.2 (a) Sample circuit. (b) Possible graph representation. (c) Possible hypergraph representation.

Given a graph $G(V,E)$, for each node $v \in V$, $area(v)$ represents the area of the corresponding cell or module. For each edge $e \in E$, $w(e)$ represents the priority or weight, e.g., timing criticality, of the corresponding edge.

Though this chapter discusses the partitioning problem and partitioning algorithms within the graph context, logic circuits are more accurately represented using *hypergraphs*, where each *hyperedge*[2] connects two or more cells. Many graph-based algorithms can be directly extended to hypergraphs.

The set of all partitions $|Part|$ is *disjoint* if each node $v \in V$ is assigned to *exactly one* of the partitions.

An edge between two nodes i and j is *cut* if i and j belong to different partitions A and B, i.e., $i \in A, j \in B$, and $(i,j) \in E$ (Fig. 2.3).

A *cut set* Ψ is the collection of all cut edges.

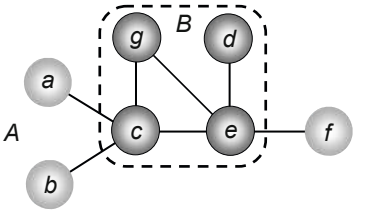


Fig. 2.3 A 2-way partitioning of the circuit in Fig. 2.2. A contains nodes a, b and f. B contains nodes c, d, e and g. Edges (a,c), (b,c) and (e,f) are cut. Edges (c,e), (c,g), (d,e) and (e,g) are not cut.

2.3 Optimization Goals

The most common partitioning objective is to minimize the number or total weight of cut edges while balancing the sizes of the partitions. If Ψ denotes the set of cut edges, the minimization objective is

$$\sum_{e \in \Psi} w(e)$$

Often, partition area is limited due to packing considerations and other boundary conditions implied by system hierarchy, chip size, or floorplan restrictions. For any subset of nodes $V' \subseteq V$, let $area(V')$ be the total area of all cells represented by the nodes of V'. *Bounded-size partitioning* enforces an upper bound UB on the total area of each partition V'. That is, $area(V_i) \leq UB_i$, where $V_i \subseteq V$, $i = 1, \ldots, k$, and k is the number of partitions. Often, a circuit must be divided evenly, with

$$area(V_i) = \sum_{v \in V_i} area(v) \leq \frac{1}{k} \sum_{v \in V} area(v) = \frac{1}{k} area(V)$$

[2] For convenience, hyperedges may be referred to as edges. However, graph edges are formally defined as node pairs.

For the special case where all nodes have unit area, the *balance criterion* is

$$|V_i| \leq \left\lceil \frac{|V|}{k} \right\rceil$$

2.4 2.4 Partitioning Algorithms

Circuit partitioning, like many other combinatorial optimization problems discussed in this book, is *NP-hard*. That is, as the problem size grows linearly, the effort needed to find an optimal solution grows faster than any polynomial function. To date, there is no known polynomial-time, globally optimal algorithm for balance-constrained partitioning (Sec. 1.6). However, several efficient heuristics were developed in the 1970s and 1980s. These algorithms find high-quality circuit partitioning solutions and in practice are implemented to run in low-order polynomial time – the Kernighan-Lin (KL) algorithm (Sec. 2.4.1), its extensions (Sec. 2.4.2) and the Fiduccia-Mattheyses (FM) algorithm (Sec. 2.4.3). Additionally, optimization by *simulated annealing* can be used to solve particularly difficult partitioning formulations. In general, stochastic hill-climbing algorithms require more than polynomial time to produce high-quality solutions, but can be accelerated by sacrificing solution quality. In practice, simulated annealing is rarely competitive.

► 2.4.1 Kernighan-Lin (KL) Algorithm

The Kernighan-Lin (KL) algorithm performs partitioning through iterative-improvement steps. It was proposed by *B. W. Kernighan* and *S. Lin* in 1970 [2.6] for *bipartitioning* ($k = 2$) graphs, where every node has unit weight. This algorithm has been extended to support k-way partitioning ($k > 2$) as well as cells with arbitrary areas (Sec. 2.4.2).

Introduction. The KL algorithm operates on a graph representation of the circuit, where nodes (edges) represent cells (connections between cells). Formally, let a graph $G(V,E)$ have $|V| = 2n$ nodes, where each node $v \in V$ has the same weight, and each edge $e \in E$ has a non-negative edge weight. The KL algorithm partitions V into two disjoint subsets A and B with minimum cut cost and $|A| = |B| = n$.

The KL algorithm is based on exchanging (swapping) pairs of nodes, each node from a different partition. The two nodes that generate the highest reduction in cut size are swapped. To prevent immediate move reversal (*undo*) and subsequent infinite loops, the KL algorithm *fixes* nodes after swapping them. Fixed nodes cannot be swapped until they are released, i.e., become *free*.

Execution of the KL algorithm proceeds in *passes*. Typically, the first pass or iteration begins with an arbitrary initial partition. In a given pass, after all nodes become fixed, the algorithm determines the prefix of the sequence of swaps within this pass that produces the largest *gain*, i.e., reduction of cut cost. All nodes included in this sequence are moved accordingly. The pass finishes by releasing all fixed nodes, so that all nodes are once again free. In each subsequent pass, the algorithm starts with the two partitions from the previous pass. All possible pair swaps are then re-evaluated. If no improvement is found during a given pass, the algorithm terminates.

Terminology. The following terms are specifically relevant to the KL algorithm.

The *cut size* or *cut cost* of a graph with either unweighted or uniform-weight edges is the number of edges that have nodes in more than one partition. With weighted edges, the cut cost is the sum of the weights of all cut edges.

The cost $D(v)$ of moving a node $v \in V$ in a graph from partition A to B is

$$D(v) = |E_B(v)| - |E_A(v)|$$

where $E_B(v)$ is the set of v's incident edges that are cut by the cut line, and $E_A(v)$ is the set of v's incident edges that are not cut by the cut line. High costs ($D > 0$) indicate that the node should move, while low costs ($D < 0$) indicate that the node should stay within the same partition.

The *gain* $\Delta g(a,b)$ of swapping a pair of nodes a and b is the improvement in overall cut cost that would result from the node swap. A positive gain ($\Delta g > 0$) means that the cut cost is decreased, while a negative gain ($\Delta g < 0$) means that the cut cost is increased. The gain of swapping two nodes a and b is

$$\Delta g(a,b) = D(a) + D(b) - 2c(a,b)$$

where $D(a)$ and $D(b)$ are the respective costs of nodes a and b, and $c(a,b)$ is the connection weight between a and b. If an edge exists between a and b, then $c(a,b)$ = the edge weight between a and b. Otherwise, $c(a,b) = 0$.

Notice that simply adding $D(a)$ and $D(b)$ when calculating Δg assumes that an edge is cut (uncut) *before* the swap and will be uncut (cut) *after* the swap. However, this does not apply if the nodes are connected by an edge e, as it will be cut both before and after the swap. Therefore, the term $2c(a,b)$ corrects for this overestimation of gain from the swap.

The *maximum positive gain* G_m corresponds to the best prefix of m swaps within the swap sequence of a given pass. These m swaps lead to the partition with the minimum cut cost encountered during the pass. G_m is computed as the sum of Δg values over the first m swaps of the pass, with m chosen such that G_m is maximized.

$$G_m = \sum_{i=1}^{m} \Delta g_i$$

Within a pass of the KL algorithm, the moves are only used to find the move sequence <1 … m> and G_m. The moves are then applied after these have been found.

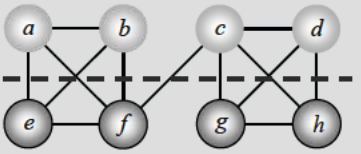
Algorithm. The Kernighan-Lin algorithm progresses as follows.

```
Kernighan-Lin Algorithm
Input: graph G(V,E) with |V| = 2n
Output: partitioned graph G(V,E)
1.    (A,B) = INITIAL_PARTITION(G)          // arbitrary initial partition
2.    Gm = ∞
3.    while (Gm > 0)
4.      i = 1
5.      order = Ø
6.      foreach (node v ∈ V)
7.        status[v] = FREE                  // set every node as free
8.        D[v] = COST(v)                    // compute D(v) for each node
9.      while (!IS_FIXED(V))                // while all cells are not fixed, select free
10.       (Δgi,(ai,bi)) = MAX_GAIN(A,B)     //   node ai from A and free node bi from
                                            //   B so that Δgi = D(ai) + D(bi) – 2c(ai,bi)
                                            //   is maximized
11.       ADD(order,(Δgi,(ai,bi)))          // keep track of swapped cells
12.       TRY_SWAP(ai,bi,A,B)               // move ai to B, and move bi to A
13.       status[ai] = FIXED                // mark ai as fixed
14.       status[bi] = FIXED                // mark bi as fixed
15.       foreach (free node vf connected to ai and bi)
16.         D[vf] = COST(vf)                // compute and update D(vf)
17.       i = i + 1
18.     (Gm,m) = BEST_MOVES(order)          // swap sequence 1 … m that
                                            //   maximizes Gm
19.     if (Gm > 0)
20.       CONFIRM_MOVES(order,m)            // execute move sequence
```

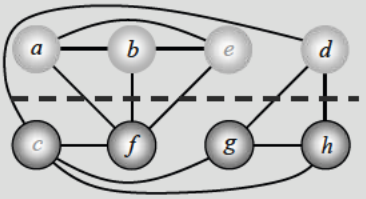
First, partition the input graph G into two arbitrary partitions A and B (line 1), and set the maximum positive gain value G_m to ∞ (line 2). During each pass (line 3), for each node $v \in V$, compute the cost $D(v)$ and set v to free (lines 4-6). Then, while all nodes are not fixed (line 9), select, swap, and fix two free nodes a_i and b_i from A and B, respectively, such that Δg_i is maximized (lines 10-14). Update $D(v)$ for all free nodes that are adjacent to a_i and b_i (lines 15-16).

After all nodes have been fixed, find the move sequence <1 … m> that maximizes G_m (line 18). If G_m is positive, execute the move sequence (lines 19-20), and perform another pass (lines 3-20). Otherwise, terminate the algorithm.

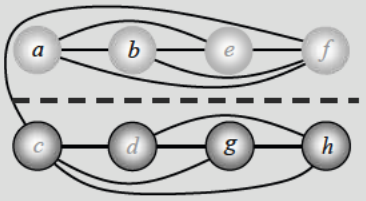
Example: KL Algorithm
Given: initial partition of nodes *a*-*h* (right).
Task: perform the first pass of the KL algorithm.



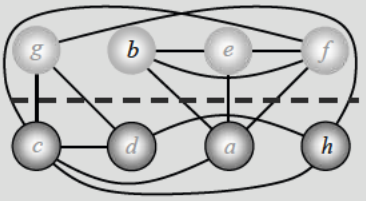
Solution:
Initial cut cost = 9.
Compute $D(v)$ costs for all free nodes *a*-*h*.
$D(a) = 1,\ D(b) = 1,\ D(c) = 2,\ D(d) = 1,$
$D(e) = 1,\ D(f) = 2,\ D(g) = 1,\ D(h) = 1$
$\Delta g_1 = D(c) + D(e) - 2c(c,e) = 2 + 1 - 0 = 3$
Swap and fix nodes *c* and *e*.



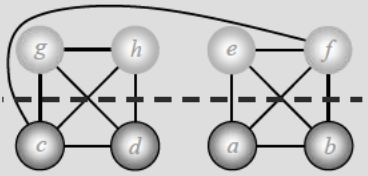
Update $D(v)$ costs for all free nodes connected to newly swapped nodes *c* and *e*: *a*, *b*, *d*, *f*, *g* and *h*.
$D(a) = -1,\ D(b) = -1,\ D(d) = 3,$
$D(f) = 2,\ \ D(g) = -1,\ D(h) = -1$
$\Delta g_2 = D(d) + D(f) - 2c(d,f) = 3 + 2 - 0 = 5$
Swap and fix nodes *d* and *f*.



Update $D(v)$ costs for all free nodes connected to newly swapped nodes *d* and *f*: *a*, *b*, *g* and *h*.
$D(a) = -3,\ D(b) = -3,\ D(g) = -3,\ D(h) = -3$
$\Delta g_3 = D(a) + D(g) - 2c(a,g) = -3 + -3 - 0 = -6$
Swap and fix nodes *a* and *g*.



Update $D(v)$ costs for all free nodes connected to newly swapped nodes *a* and *g*: *b* and *h*.
$D(b) = -1,\ D(h) = -1$
$\Delta g_4 = D(b) + D(h) - 2c(b,h) = -1 + -1 - 0 = -2$
Swap and fix nodes *b* and *h*.



Compute maximum positive gain G_m.

$G_1 = \Delta g_1 = 3$
$G_2 = \Delta g_1 + \Delta g_2 = 3 + 5 = 8$
$G_3 = \Delta g_1 + \Delta g_2 + \Delta g_3 = 3 + 5 + -6 = 2$
$G_4 = \Delta g_1 + \Delta g_2 + \Delta g_3 + \Delta g_4 = 3 + 5 + -6 + -2 = 0$

$G_m = 8$ with $m = 2$. Since $G_m > 0$, the first $m = 2$ swaps are executed: (c,e) and (d,f). Additional passes are performed until $G_m \leq 0$.

The KL algorithm does not always find an optimal solution, but performs reasonably well in practice on graphs with up to several hundred nodes. In general, the number of passes grows with the size of the graph, but in practice improvement often ceases after four passes.

Notice that Δg cannot always be positive: after all nodes have been swapped between two partitions, the cut cost will be exactly the same as the initial cut cost, so some best-gain values during the pass can be negative. However, since other moves (gains) might compensate for this, the entire pass should be completed, computing all moves until all cells are fixed.

The runtime of the KL algorithm is dominated by two factors – gain updates and pair selection. The KL algorithm selects n pairs of nodes to swap, where n is the number of nodes in each partition. For each node v, the required time to update the gains and compare is on the order of $O(n)$. That is, after swapping a_i and b_i in move i, at most $(2n - 2i)$ gains of free nodes must be updated. Therefore, the time spent updating gains over the n moves in a pass is at most

$$\sum_{i=1}^{n} 2n - 2i = O(n^2)$$

During pair comparison in a given move i, there are as many as $(n - i + 1)^2 = O(n^2)$ pairs to choose from. The time to perform n pair comparisons is bounded by

$$\sum_{i=1}^{n} (n - i + 1)^2 = O(n^3)$$

Therefore, the KL algorithm requires a total of $O(n^2) + O(n^3) = O(n^3)$ time.

An optimized KL implementation has $O(n^2 \log n)$ runtime complexity. To speed up pair comparison, node pairs can be sorted ahead of time. Since the goal is to maximize $\Delta g(a,b) = D(a) + D(b) - 2c(a,b)$, the gains of each node move can be sorted in descending order. That is, for each node $a \in A$, order the gains $D(a)$ such that

$$D(a_1) \geq D(a_2) \geq \ldots \geq D(a_{n\ i+1})$$

Similarly, for each node $b \in B$, order the gains $D(b)$ such that

$$D(b_1) \geq D(b_2) \geq \ldots \geq D(b_{n\ i+1})$$

Then, evaluate pairwise gains, starting with the first elements from both lists. A clever order of evaluation – exploiting advanced data structures and bounded node degrees [2.3] – allows the pair evaluation process to stop once a pair of gains $D(a_j)$ and $D(b_k)$ is found with $D(a_j) + D(b_k)$ is less than the best previously-found gain (no better pair-swap can exist). In practice, the best pair-swap at the kth move can be found in $O(n - k)$ time after sorting the free node gains in $O((n - k) \log (n - k))$ time [2.2]. The time required to perform pair comparison is thus reduced from $O(n^2)$ time to $O(n \log n)$ time.

► 2.4.2 Extensions of the Kernighan-Lin Algorithm

To accommodate *unequal partition sizes* $|A| \neq |B|$, arbitrarily split the nodes among the two partitions A and B, where one partition contains $\min(|A|,|B|)$ nodes and the other $\max(|A|,|B|)$ nodes. Apply the KL algorithm with the restriction that only $\min(|A|,|B|)$ node pairs can be swapped.

To accommodate *unequal cell sizes* or *unequal node weights*, assign a *unit area* that denotes the smallest cell area, i.e., the greatest common divisor of all cell areas. All unequal node sizes are then cast as integer multiples of the unit area. Each node portion (all parts of an original node that was split up) is connected to each of its counterparts by *infinite-weight*, i.e., high-priority edges. Apply the KL algorithm.

To perform *k-way partitioning*, arbitrarily assign all $k \cdot n$ nodes to partitions such that each partition has n nodes. Apply the KL 2-way partitioning algorithm to all possible pairs of subsets (1 and 2, 2 and 3, etc.) until none of the consecutive KL applications obtains any improvement on the cut size.

► 2.4.3 Fiduccia-Mattheyses (FM) Algorithm

Given a graph $G(V,E)$ with nodes and *weighted* edges, the goal of (bi)partitioning is to assign all nodes to disjoint partitions, so as to minimize the total cost (weight) of all cut nets while satisfying partition size constraints. The Fiduccia-Mattheyses (FM) algorithm is a partitioning heuristic, published in 1982 by *C. M. Fiduccia* and *R. M. Mattheyses* [2.4], offers substantial improvements over the KL algorithm.

- *Single* cells are moved independently instead of swapping pairs of cells. Thus, this algorithm is more naturally applicable to partitions of unequal size or the presence of initially fixed cells.
- Cut costs are extended to include *hypergraphs* (Sec. 1.8). Thus, all nets with two or more pins can be considered. While the KL algorithm aims to minimize cut costs based on *edges*, the FM algorithm minimizes cut costs based on *nets*.
- The area of each individual cell is taken into account.
- The selection of cells to move is much faster. The FM algorithm has runtime complexity of $O(|Pins|)$ per pass, where $|Pins|$ is the total number of pins, defined as the sum of all edge degrees $|e|$ over all edges $e \in E$.

Introduction. The FM algorithm is typically applied to large circuit netlists. For this section, all nodes and subgraphs are referred to as *cells* and *blocks*, respectively.

The FM move selection process is similar to that of the KL algorithm, with the underlying objective being to minimize cut cost. However, the FM algorithm computes the gain of each individual cell move, rather than that of each pair-swap. Like the KL algorithm, the FM algorithm selects the best prefix of moves from within a *pass*.

During an FM pass, once a cell is moved, it becomes fixed and cannot be moved for the remainder of the pass. The cells that are moved during the FM algorithm are denoted by the sequence $<c_1 \ldots c_m>$, whereas the KL algorithm swaps the first m pairs.

Terminology. The following definitions are relevant to the FM algorithm.

A net is *cut* if its cells occupy more than one partition. Otherwise, the net is *uncut*.

The *cut set* of a partition *part* is the set of all nets that are marked as *cut* within *part*.

The *gain* $\Delta g(c)$ for cell c is the change in the cut set size if c moves. The higher the gain $\Delta g(c)$, the higher is the priority to move the cell c to the other partition. Formally, the cell gain is defined as

$$\Delta g(c) = FS(c) - TE(c)$$

where $FS(c)$ is the number of nets connected to c but not connected to any other cells within c's partition, i.e., *cut nets that connect only to c*, and $TE(c)$ is the number of *uncut* nets connected to c. Informally, $FS(c)$ is like a *moving* force – the higher the $FS(c)$ value, the stronger the pull to move c to the other partition. $TE(c)$ is like a *retention force* – the higher the $TE(c)$ value, the stronger the desire to remain in the current partition.

The *maximum positive gain* G_m of a pass is the cumulative cell gain of m moves that produce a minimum cut cost. G_m is determined by the maximum sum of cell gains Δg over a prefix of m moves in a pass

$$G_m = \sum_{i=1}^{m} \Delta g_i$$

As in the KL algorithm, all moves in a pass are used to determine G_m and the move sequence $<c_1 \ldots c_m>$. Only at the end of the pass, i.e., after determining G_m and the corresponding m moves, are the cell positions updated (moved).

The *ratio factor* is the relative balance between the two partitions with respect to cell area. This ratio factor is used to prevent all cells from clustering into one partition. The ratio factor r is defined as

$$r = \frac{area(A)}{area(A) + area(B)}$$

where $area(A)$ and $area(B)$ are the total respective areas of partitions A and B, and

$$area(A) + area(B) = area(V)$$

where $area(V)$ is the total area of all cells $c \in V$, and is defined as

$$area(V) = \sum_{c \in V} area(c)$$

The *balance criterion* enforces the ratio factor. To ensure feasibility, the maximum cell area $area_{max}(V)$ must be taken into account. A partitioning of V into two partitions A and B is said to be *balanced* if

$$r \cdot area(V) - area_{max}(V) \leq area(A) \leq r \cdot area(V) + area_{max}(V)$$

A *base cell* is a cell c that has maximum cell gain $\Delta g(c)$ among all free cells, and whose move does not violate the balance criterion.

The *pin distribution* of a net *net* is given as a pair $(A(net),B(net))$, where $A(net)$ is the number of pins in partition A and $B(net)$ is the number of pins in partition B.

A net *net* is *critical* if it contains a cell c whose move changes the cut state of *net*. A critical net is either contained completely within a partition, or has exactly one of its cells in one partition and all of its other cells in the other partition. If *net* is critical, then either $A(net) = 0$, $A(net) = 1$, $B(net) = 0$, or $B(net) = 1$ must hold (Fig. 2.4).

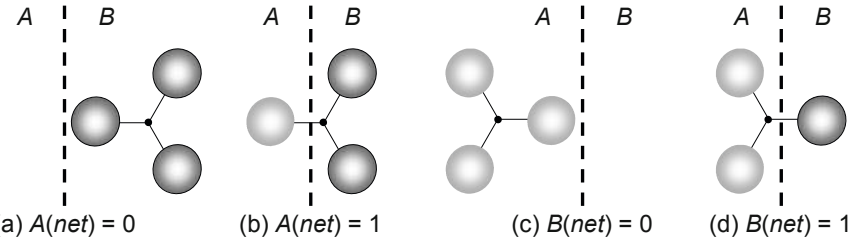


Fig. 2.4 Cases when a net *net* is critical. (a) $A(net) = 0$. (b) $A(net) = 1$. (c) $B(net) = 0$. (d) $B(net) = 1$.

Critical nets simplify the calculation of cell gains. Only cells belonging to critical nets need to be considered in the gain computation, as it is only for such nets that the movement of *a single cell* can change the cut state. *B. Krishnamurthy* [2.8] generalized the concept of criticality and improved the FM algorithm such that it comprehends how many cell moves nets are away from being critical. This results in a *gain vector* for each cell instead of a single *gain value* – the ith element of the gain vector for a free cell c_f records how many nets will become i cell moves away from being uncut if c_f moves.

The *from-block F* and *to-block T* define the direction in which a cell moves. That is, a cell moves from F to T.

Algorithm. The Fiduccia-Mattheyses algorithm progresses as follows.

```
Fiduccia-Mattheyses Algorithm
Input: graph G(V,E), ratio factor r
Output: partitioned graph G(V,E)
1.    (lb,ub) = BALANCE_CRITERION(G,r)   // compute balance criterion
2.    (A,B) = PARTITION(G)               // initial partition
3.    Gm = ∞
4.    while (Gm > 0)
5.      i = 1
6.      order = Ø
7.      foreach (cell c ∈ V)             // for each cell, compute the
8.        Δg[i][c] = FS(c) – TE(c)       //   gain for current iteration,
9.        status[c] = FREE               //   and set each cell as free
10.     while (!IS_FIXED(V))             // while there are free cells, find
11.       cell = MAX_GAIN(Δg[i],lb,ub)   //   the cell with maximum gain
12.       ADD(order,(cell,Δg[i]))        // keep track of cells moved
13.       critical_nets = CRITICAL_NETS(cell) // critical nets connected to cell
14.       if (cell ∈ A)                  // if cell belongs to partition A,
15.         TRY_MOVE(cell,A,B)           //   move cell from A to B
16.       else                           // otherwise, if cell belongs to B,
17.         TRY_MOVE(cell,B,A)           //   move cell from B to A
18.       status[cell] = FIXED           // mark cell as fixed
19.       foreach (net net ∈ critical_nets) // update gains for critical cells
20.         foreach (cell c ∈ net, c ≠ cell)
21.           if (status[c] == FREE)
22.             UPDATE_GAIN(Δg[i][c])
23.       i = i + 1
24.     (Gm, m ) = BEST_MOVES(order)     // move sequence c1 … cm that
                                         //   maximizes Gm
25.     if (Gm > 0)
26.       CONFIRM_MOVES(order,m )        // execute move sequence
```

First, compute the balance criterion (line 1) and partition the graph (line 2). During each pass (line 4), determine the gain for each free cells c based on the number of incident cut nets $FS(c)$ and incident uncut nets $TE(c)$ (lines 7-8). After all the cell gains have been determined, select the base cell maximizes gain subject to satisfying the balance criterion (line 11). If multiple cells meet these two criteria then break ties, e.g., to keep partition sizes as close as possible to the middle of the ranges given by the balance criterion.

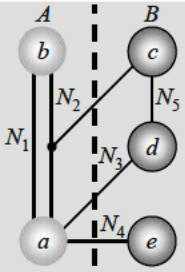
After the base cell has been selected, move it from its current partition to the other partition, and mark it as fixed (line 14-18). Of the remaining free cells, only those connected to the base cell by critical nets need to have their gains updated (lines 19-22). The gain of an unmoved cell can only change if it is connected to a moved cell (a base cell). Therefore, gain values change only when a net goes from occupying two partitions to one (positive gain) or vice-versa (negative gain). After updating the necessary cell gains, select the next base cell that has maximum gain value and

satisfies the balance criterion. Continue this process until there are no remaining free cells (lines 10-23). Once all cells have been fixed, find the best prefix of the move sequence $<c_1 \ldots c_m>$ (line 24) to achieve maximum positive gain

$$G_m = \sum_{i\ 1}^{m} \Delta g_i$$

As long as $G_m > 0$, execute the move sequence, free all cells, and begin a new pass. Otherwise, terminate the algorithm (lines 25-26).

Example: FM Algorithm



Given: (1) weighted cells *a*-*e*, (2) ratio factor $r = 0.375$, (3) nets N_1-N_5, and (4) initial partition (right).

$area(a) = 2$ $area(b) = 4$ $area(c) = 1$ $area(d) = 4$ $area(e) = 5$

$N_1 = (a,b)$ $N_2 = (a,b,c)$ $N_3 = (a,d)$ $N_4 = (a,e)$ $N_5 = (c,d)$

Task: perform the first pass of the FM algorithm.

Solution:

Compute the balance criterion.

$r \cdot area(V) - area_{max} \le area(A) \le r \cdot area(V) + area_{max}(V)$

$r \cdot area(V) - area_{max}(V) = 0.375 \cdot 16 - 5 = 1, r \cdot area(V) + area_{max}(V) = 0.375 \cdot 16 + 5 = 11.$

Size range of A: $1 \le area(A) \le 11$.

Compute the gains of each cell *a*-*e*.

Nets N_3 and N_4 are cut: $FS(a) = 2$. Net N_1 is connected to *a* but is not cut: $TE(a) = 1$.

$\Delta g_1(a) = 2 - 1 = 1$. The cut size will be reduced if *a* moves from *A* to *B*.

Similarly, the gains of the remaining cells are computed.

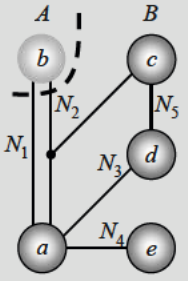
b:	$FS(b) = 0$	$TE(b) = 1$	$\Delta g_1(b) = -1$
c:	$FS(c) = 1$	$TE(c) = 1$	$\Delta g_1(c) = 0$
d:	$FS(d) = 1$	$TE(d) = 1$	$\Delta g_1(d) = 0$
e:	$FS(e) = 1$	$TE(e) = 0$	$\Delta g_1(e) = 1$

Select the base cell. Possible base cells are *a* and *e*.

Balance criterion after moving *a*: $area(A) = area(b) = 4$.

Balance criterion after moving *e*: $area(A) = area(a) + area(b) + area(e) = 11$.

Both moves respect the balance criterion, but *a* is selected, moved, and fixed as a result of the tie-breaking criterion described above.



Update gains for all free cells that are connected by critical nets to *a*. To determine whether a given net is critical, the number of cells associated with that net in each partition is counted before and after the move.

For a given net *net*, let the number of cells in the from-block and to-block *before* the move be $F(net)$ and $T(net)$, respectively. Let the number of cells in the from-block and to-block *after* the move be $F'(net)$ and $T'(net)$, respectively. If any of these values is 0 or 1, then *net* is critical. For nets N_1, N_2, N_3, N_4, $T(N_1) = 0$ and $T(N_2) = T(N_3) = T(N_4) = 1$. Therefore, cells *b*, *c*, *d* and *e* need updating. The gain values do not have to be computed explicitly but can be derived from $T(net)$.

If $T(net) = 0$, all gain values of free cells connected to *net* increase by 1. Since $T(N_1) = 0$, cell *b* has updated gain value of $\Delta g_1(b) = \Delta g_1(b) + 1$. That is, net N_1 (connected to cell *b*) has increased the cut set of the partition. The increase in $\Delta g(b)$ reflects the motivation to move cell *b*. Since net N_1 is now cut, moving cell *b* is justified.

If $T(net) = 1$, all gain values of free cells connected to *net* decrease by 1. Since $T(N_2) = T(N_3) = T(N_4) = 1$, cells *c*, *d* and *e* have updated gain values of $\Delta g_1(c,d,e) = \Delta g_1(c,d,e) - 1$. That is, nets N_2, N_3 and N_4 (connected to cells *c*, *d* and *e*, respectively) have decreased the cut set of the partition. This reduction in $\Delta g_1(c,d,e)$ reflects the motivation to not move cells *c*, *d* and *e*. Similarly, when $F'(net) = 0$, all cell gains connected to *net* are reduced by 1, and when $F'(net) = 1$, all cell gains connected to *n* are increased by 1.

The updated Δg values are

b:	$FS(b) = 2$	$TE(b) = 0$	$\Delta g_1(b) = 2$
c:	$FS(c) = 0$	$TE(c) = 1$	$\Delta g_1(c) = -1$
d:	$FS(d) = 0$	$TE(d) = 2$	$\Delta g_1(d) = -2$
e:	$FS(e) = 0$	$TE(e) = 1$	$\Delta g_1(e) = -1$

Iteration $i = 1$
Partitions: $A_1 = \{b\}$, $B_1 = \{a,c,d,e\}$, with fixed cells $\{a\}$.

Iteration $i = 2$
Cell *b* has maximum gain $\Delta g_2 = 2$, $area(A) = 0$, balance criterion is violated.
Cell *c* has next maximum gain $\Delta g_2 = -1$, $area(A) = 5$, balance criterion is met.
Cell *e* has next maximum gain $\Delta g_2 = -1$, $area(A) = 9$, balance criterion is met.
Move cell *c*, updated partitions: $A_2 = \{b,c\}$, $B_2 = \{a,d,e\}$, with fixed cells $\{a,c\}$.

Iteration $i = 3$
Gain values: $\Delta g_3(b) = 1$, $\Delta g_3(d) = 0$, $\Delta g_3(e) = -1$.
Cell *b* has maximum gain $\Delta g_3 = 1$, $area(A) = 1$, balance criterion is met.
Move cell *b*, updated partitions: $A_3 = \{c\}$, $B_3 = \{a,b,d,e\}$, with fixed cells $\{a,b,c\}$.

Iteration $i = 4$
Gain values: $\Delta g_4(d) = 0$, $\Delta g_4(e) = -1$.
Cell *d* has maximum gain $\Delta g_4 = 0$, $area(A) = 5$, balance criterion is met.
Move cell *d*, updated partitions: $A_4 = \{c,d\}$, $B_4 = \{a,b,e\}$, with fixed cells $\{a,b,c,d\}$.

Iteration $i = 5$
Gain values: $\Delta g_5(e) = -1$.
Cell e has maximum gain $\Delta g_5 = -1$, $area(A) = 10$, balance criterion is met.
Move cell e, updated partitions: $A_5 = \{c,d,e\}$, $B_5 = \{a,b\}$, all cells fixed.

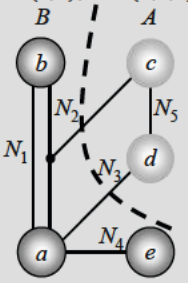
Find best move sequence $<c_1 \ldots c_m>$

$$G_1 = \Delta g_1 = 1$$
$$G_2 = \Delta g_1 + \Delta g_2 = 0$$
$$G_3 = \Delta g_1 + \Delta g_2 + \Delta g_3 = 1$$
$$G_4 = \Delta g_1 + \Delta g_2 + \Delta g_3 + \Delta g_4 = 1$$
$$G_5 = \Delta g_1 + \Delta g_2 + \Delta g_3 + \Delta g_4 + \Delta g_5 = 0$$

Maximum positive cumulative gain $G_m = \sum_{i=1}^{m} \Delta g_i = 1$ found in iterations 1, 3 and 4.

The move prefix $m = 4$ is selected due to the better balance ratio ($area(A) = 5$); the four cells a, b, c and d are then moved.

Result of Pass 1: Current partitions: $A = \{c,d\}$, $B = \{a,b,e\}$, cut cost reduced from 3 to 2.



Pass 2 is left as a further exercise (Exercise 3). In this pass, the cut cost is reduced from 2 to 1. In Pass 3, no further improvement can be made. Thus, the partition found in Pass 2 is the final solution returned by the FM algorithm with a cut cost of 1.

2.5 A Framework for Multilevel Partitioning 2.5

Among the partitioning techniques discussed so far, the Fiduccia-Mattheyses heuristic offers the best tradeoff between solution quality and runtime. In particular, it is much faster than other techniques and, in practice, finds better partitions given the same amount of time. Unfortunately, if the partitioned hypergraph includes more than several hundred nodes, the FM algorithm may terminate with a high net cut or make a large number of passes, each producing minimal improvement.

To improve the scalability of netlist partitioning, the FM algorithm is typically embedded into a *multilevel* framework that consists of several distinct steps. First, during the *coarsening* phase, the original "flat" netlist is hierarchically clustered. Sec-

ond, FM is applied to the clustered netlist. Third, the netlist is partially unclustered during the *uncoarsening* phase. Fourth, during the *refinement* phase, FM is applied incrementally to the partially unclustered netlist. The third and fourth steps are continued until the netlist is fully unclustered. In other words, FM is applied to the partially unclustered netlist and the solution is unclustered further – with this process repeating until the solution is completely flat.

For circuits with hundreds of thousands of gates, the multilevel framework dramatically improves runtime because many of the FM calls operate on smaller netlists, and each incremental FM call has a relatively high-quality initial solution. Furthermore, solution quality is improved, as applying FM to clustered netlists allows the algorithm to reassign entire clusters to different partitions where appropriate.

► 2.5.1 Clustering

To construct a coarsened netlist, groups of tightly-connected nodes can be *clustered*, absorbing connections between these nodes (Fig. 2.5). The remaining connections between clusters retain the overall structure of the original netlist. In specific applications, the size of each cluster is often limited so as to prevent *degenerate* clustering, where a single large cluster dominates other clusters.

When merging nodes, a cluster is assigned the sum of the weights of its constituent nodes. As closed-form objective functions for clustering are difficult to formulate, clustering is performed by application-specific algorithms. Additionally, clustering must be performed quickly to ensure the scalability of multilevel partitioning.

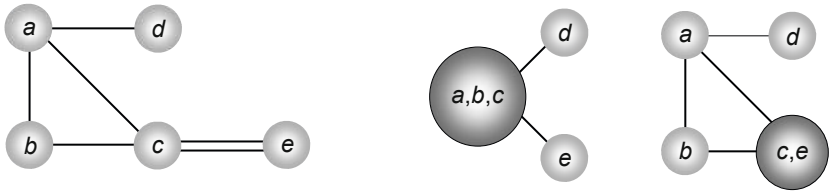


Fig. 2.5 An initial graph (left), and possible clusterings of the graph (right).

► 2.5.2 Multilevel Partitioning

Multilevel partitioning techniques begin with a *coarsening* phase in which the input graph G is clustered into a smaller graph G', which, in turn, can also be clustered into another graph G'', and so on. Let l be the number of levels, i.e., times, that G goes through the coarsening stage. Each node at level l represents a cluster of nodes at level $l + 1$. For large-scale applications, the *clustering ratio*, i.e., the average number of nodes per cluster, is often 1.3 (*Hypergraph Partitioning and Clustering*, Chap. 61 in [2.5]). For a graph with $|V|$ nodes, the number of levels can be estimated as

$$\lceil \log(|V| / v_0) \rceil$$

where v_0 is the number of nodes in the most-clustered graph (level 0), and the base of the logarithm is the clustering ratio. Clustering with this ratio is repeated until the graph is small enough to be processed efficiently by the FM partitioning algorithm. In practice, v_0 is typically 75-200, an instance size for which the FM algorithm is nearly optimal. For netlists with more than 200 nodes, multilevel partitioning techniques improve both runtime and solution quality. Academic multilevel partitioners include *hMetis* [2.6], available as a binary, and *MLPart* [2.1], available as open-source software.

The most-clustered netlist is partitioned using the FM algorithm. Then, its cluster-to-partition assignment is projected onto the larger, partially-unclustered netlist one level below. This is done by assigning every subcluster to the partition to which its parent cluster was assigned.

Using that partition as a starting configuration, subclusters in the partially-unclustered graph can be moved by the FM algorithm from one partition to another to improve cost while satisfying balance criteria (*refinement*). The process continues until the bottom-level netlist is refined (Fig. 2.6).

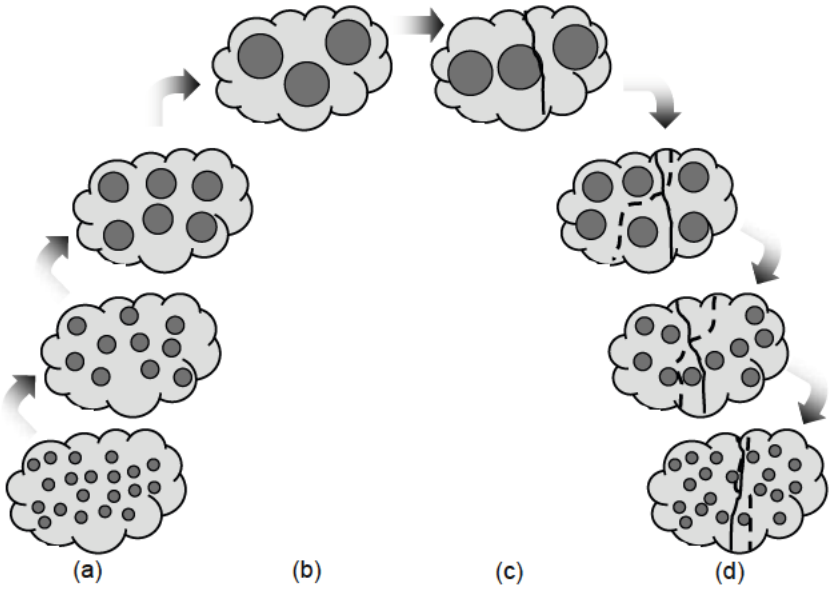


Fig. 2.6 An illustration of multilevel partitioning. (a) The original graph is first coarsened through several levels. (b) The graph after coarsening. (c) After coarsening, a heuristic partition is found of the most-coarsened graph. (d) That partition is then projected onto the next-coarsest graph (dotted line) and then refined (solid line). Projection and refinement continue until a partitioning solution for the original graph is found.

2.6 System Partitioning onto Multiple FPGAs

System implementation using field-programmable gate arrays (FPGAs), such as those manufactured by *Xilinx* or *Altera*, is an increasingly important application of partitioning. There are two main reasons for this trend. First, FPGA-based system prototyping allows products to meet shorter time-to-market windows, since embedded software development and design debugging can proceed concurrently with hardware design, rather than having to wait until the packaged dies arrive from the foundry. Second, with increased non-recurring engineering costs (mask sets and probe cards) in advanced technology nodes, products with lower production volumes become economically feasible only when implemented using field-programmable devices. However, field-programmability (e.g., using SRAM-based lookup tables to implement reconfigurable logic and interconnect) comes at the cost of density, speed and power. Hence, even if a system easily fits onto a single ASIC, its prototype may require multiple FPGA devices.

Functionally, FPGA-based systems may be viewed as logic (implemented using reprogrammable FPGAs) and interconnects (implemented using field-programmable interconnect chips, or FPICs). Many system components, including embedded processor cores, embedded memories, and standard interfaces, are available as configurable IPs on modern FPGA devices. Moreover, FPICs themselves can be implemented using FPGAs. An example FPGA-based system topology is illustrated in Fig. 2.7(a), where the FPGA and FPIC devices are connected using a *Clos* network topology, which allows any two devices to communicate directly (or a small number of hops). Fig. 2.7(b) demonstrates how a typical system architecture of logic and memory can be mapped onto multiple FPGA devices.

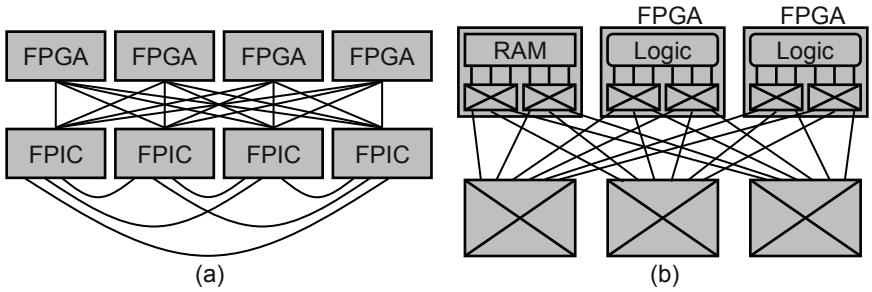


Fig. 2.7 (a) Reconfigurable system with multiple FPGA and FPIC devices. (b) Mapping of a typical system architecture onto multiple FPGAs.

Key challenges for multi-way system partitioning onto FPGAs include (1) low utilization of FPGA gate capacity because of hard I/O pin limits, (2) low clock speeds due to long interconnect delays between multiple FPGAs, and (3) long runtimes for the system partitioning process itself. This section discusses the associated algorithmic challenges in physical design that are unique to system implementation with multiple FPGAs.

Variant multi-way partitioning formulations. Multi-way partitioning for system prototyping seeks to minimize the number of FPGA devices needed while taking into account both area constraints, i.e., the partitions must each fit into individual FPGAs, and I/O constraints, i.e., each FPGA has a fixed number of pins. In contrast to the single-chip context, a small change in balance or cut size can make a feasible solution infeasible. Thus, a challenge for partitioning algorithms is to achieve high utilization of the FPGA devices while meeting all I/O constraints.

Once the number of FPGA devices has been determined, the secondary optimization objective is to minimize the amount of communication between the connected devices. Adopting general techniques for minimizing the net cut to FPGA-based architectures can significantly improve the overall speed of the system. However, the traditional net cut objective does not distinguish whether the gates of a five-pin net are split across two, three, four of five FPGA devices. However, splitting a net across k FPGA devices consumes k I/O pins. Hence, k should be minimized first.

Variant placement formulations. The reprogrammable nature of FPGAs allows systems to be implemented as true reconfigurable computing machines, where device configuration bits are updated to match the implemented logic to the required computation. This induces an extra "dimension" to the problems of logic partitioning and placement – the solution must explicitly evolve through time, i.e., through the course of the computation.

System implementation degrees of freedom. More performance optimizations are available, and needed, at the system level than during place-and-route. System prototyping may need to explore netlist transformations such as *cloning* (Sec. 8.5.3) and *retiming* (Sec. 8.6) in order to minimize cut size (I/O usage) or system cycle time. Such transformations are needed as inter-device delays can be relatively large and because devices are often I/O-limited. *L.-T. Liu et al.* [2.9] proposed a partitioning algorithm that permits logic replication to minimize both cut size and clock cycle of sequential circuits. Given a netlist $G = (V,E)$, their approach chooses two modules as seeds s and t and constructs a "replication graph" that is twice the size of the original circuit. This graph has the special property that a type of directed minimum cut yields the *replication cut*, i.e., a decomposition of V into S, T and R, where $s \in S$, $t \in T$ and $R = V - S - T$ is the replicated logic, that is optimal. A directed version of the Fiduccia-Mattheyses algorithm (Sec. 2.4.3) is used to find a heuristic directed minimum cut in the replication graph.

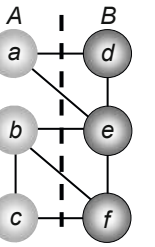
"Flow-based" multi-way partitioning method. To decompose a system into multiple devices, *C.-W. Yeh et al.* [2.10] proposed a "flow-based" algorithm inspired by the relationship between *multi-commodity flow* [2.2] and the traditional problem of min-cut partitioning. The algorithm constructs a flow network wherein each signal net initially corresponds to an edge with unit flow cost. To visualize this, one can imagine a network of roads, where each road corresponds to a signal net in the netlist, and where driving along each individual road requires a unit toll. Two random modules in the network are chosen, and the shortest (lowest-cost) path between them

is computed. A constant $\gamma < 1$ is added to the flow for each net in the shortest path, and the cost for every net in the path is incremented. Adjusting the cost penalizes paths through congested areas and forces alternative shortest paths. This random shortest path computation is repeated until every path between the chosen pair of modules passes through at least one "saturated" net (in the analogy, this would be a "congested road"). The set of saturated nets induces a multi-way partitioning in which two modules belong to the same cluster if and only if there is a path of unsaturated nets between them. A second phase of the algorithm makes the multi-way partitioning more balanced. Since this approach has efficient runtime and is easily parallelizable, it is well-suited for large-scale multi-way system partitioning.

Commercial tools for partitioning large systems onto FPGAs typically base their algorithms on the multilevel extensions of the FM algorithm (Sec. 2.4.3). While it is not always possible to modify such algorithms to track relevant partitioning objectives directly, these algorithms often produce reasonable initial partitions when guided by the net cut objective and its variants.

Chapter 2 Exercises

Exercise 1: KL Algorithm

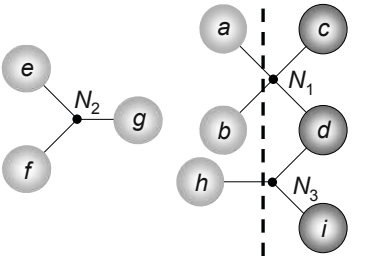


The graph to the right (nodes *a*-*f*) can be optimally partitioned using the Kernighan-Lin algorithm. Perform the first pass of the algorithm. The dotted line represents the initial partitioning. Assume all nodes have the same weight and all edges have the same priority.

Note: Clearly describe each step of the algorithm. Also, show the resulting partitioning (after one pass) in graphical form.

Exercise 2: Critical Nets and Gain During the FM Algorithm

(a) For cells *a*-*i*, determine the critical nets connected to these cells and which critical nets remain after partitioning. For the first iteration of the FM algorithm, determine which cells would need to have their gains updated due to a move. Hint: It may be helpful to prepare a table with one row per move that records (1) the cell moved, (2) critical nets before the move, (3) critical nets after the move, and (4) which cells require a gain update.



(b) Determine $\Delta g(c)$ for each cell $c \in V$.

Exercise 3: FM Algorithm

Perform Pass 2 of the FM algorithm example given in Sec. 2.4.3. Clearly describe each step. Show the result of each iteration in both numerical and graphical form.

Exercise 4: System and Netlist Partitioning

Explain key differences between partitioning formulations used for FPGA-based system emulation and traditional min-cut partitioning.

Exercise 5: Multilevel FM Partitioning

List and explain the advantages that a multilevel framework offers compared to the FM algorithm alone.

Exercise 6: Clustering

Consider a partitioned netlist. Clustering algorithms covered in this chapter do not take a given partitioning into account. Explain how these algorithms can be modified such that each new cluster is consistent with one of the initial partitions.

Chapter 2 References

[2.1] A. E. Caldwell, A. B. Kahng and I. L. Markov, "Design and Implementation of Move-Based Heuristics for VLSI Hypergraph Partitioning", *J. Experimental Algorithmics* 5 (2000), pp. 1-21.

[2.2] T. Cormen, C. Leiserson, R. Rivest and C. Stein, *Introduction to Algorithms, 2nd Edition*, McGraw Hill, 2003.

[2.3] S. Dutt, "New Faster Kernighan-Lin-Type Graph-Partitioning Algorithms", *Proc. Intl. Conf. on CAD*, 1993, pp. 370-377.

[2.4] C. M. Fiduccia and R. M. Mattheyses, "A Linear-Time Heuristic for Improving Network Partitions", *Proc. Design Autom. Conf.*, 1982, pp. 175-181.

[2.5] T. F. Gonzalez, ed., *Handbook of Approximation Algorithms and Metaheuristics*, CRC Press, 2007.

[2.6] G. Karypis, R. Aggarwal, V. Kumar and S. Shekhar, "Multilevel Hypergraph Partitioning: Application in VLSI Domain", *Proc. Design Autom. Conf.*, 1997, pp. 526-529.

[2.7] B. W. Kernighan and S. Lin, "An Efficient Heuristic Procedure for Partitioning Graphs", *Bell Sys. Tech. J.* 49(2) (1970), pp. 291-307.

[2.8] B. Krishnamurthy, "An Improved Min-Cut Algorithm for Partitioning VLSI Networks", *IEEE Trans. on Computers* 33(5) (1984), pp. 438-446.

[2.9] L.-T. Liu, M.-T. Kuo, C.-K. Cheng and T. C. Hu, "A Replication Cut for Two-Way Partitioning", *IEEE Trans. on CAD* 14(5) (1995), pp. 623-630.

[2.10] C.-W. Yeh, C.-K. Cheng and T.-T. Y. Lin, "A General Purpose, Multiple-Way Partitioning Algorithm", *IEEE Trans. on CAD* 13(12) (1994), pp. 1480-1488.

Chapter 3

Chip Planning

3

3

3 Chip Planning 57
3.1 Introduction to Floorplanning 58
3.2 Optimization Goals in Floorplanning 59
3.3 Terminology 61
3.4 Floorplan Representations 63
3.4.1 Floorplan to a Constraint-Graph Pair 63
3.4.2 Floorplan to a Sequence Pair 64
3.4.3 Sequence Pair to a Floorplan 65
3.5 Floorplanning Algorithms 68
3.5.1 Floorplan Sizing 69
3.5.2 Cluster Growth 73
3.5.3 Simulated Annealing 77
3.5.4 Integrated Floorplanning Algorithms 81
3.6 Pin Assignment 82
3.7 Power and Ground Routing 86
3.7.1 Design of a Power-Ground Distribution Network 87
3.7.2 Planar Routing 87
3.7.3 Mesh Routing 89
Chapter 3 Exercises 91
Chapter 3 References 92

3 Chip Planning

Chip planning deals with large modules such as caches, embedded memories, and intellectual property (IP) cores that have known areas, fixed or changeable shapes, and possibly fixed locations. When modules are not clearly specified, chip planning relies on *netlist partitioning* (Chap. 2) to identify such modules in large designs. Assigning shapes and locations to circuit modules during chip planning produces *blocks*, and enables early estimates of interconnect length, circuit delay and chip performance. Such early analysis can identify modules that need improvement. Chip planning consists of three major stages (1) *floorplanning*, (2) *pin assignment*, and (3) *power planning*.

Recall from Chap. 2 that a gate-level or RTL netlist can be automatically partitioned into *modules*. Alternatively, such modules can be extracted from a hierarchical design representation. Large chip modules are laid out as *blocks* or rectangular shapes (Fig. 3.1). *Floorplanning* determines the locations and dimensions of these shapes, based on the areas and aspect ratios of the modules so as to optimize chip size, reduce interconnect and improve timing. *Pin assignment* connects outgoing signal nets to block pins. *I/O placement* finds the locations for the chip's input and output pads, often at the periphery of the chip. This step is (ideally) performed before floorplanning, but locations can be updated during and after floorplanning. *Power planning* builds the *power supply network*, i.e., *power* and *ground* nets, so as to ensure that each block is provided with appropriate supply voltage. The results of partitioning and chip planning greatly affect subsequent design steps.

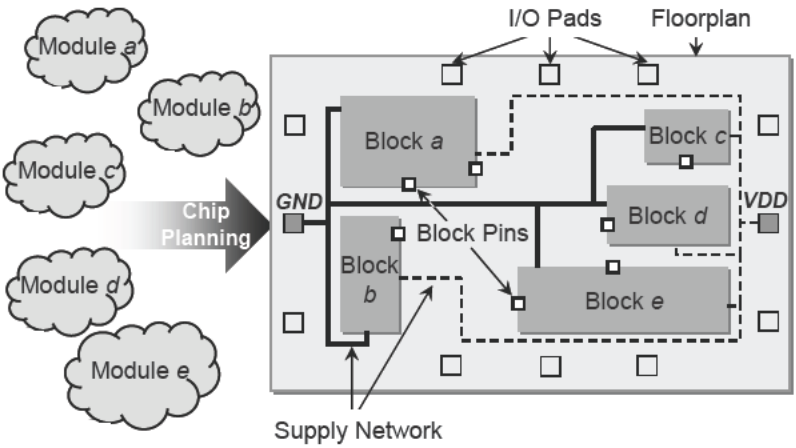


Fig. 3.1 Chip planning and relevant terminology. A module is a cluster of logic with a known area. Once it has been assigned shape or dimensions, it becomes a block. Connections between blocks are implemented through internal pins but are not shown in the figure.

A. B. Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure,*
DOI 10.1007/978-90-481-9591-6_3, © Springer Science+Business Media B.V. 2011

3.1

3.1 Introduction to Floorplanning

Before the floorplanning stage, the design is split into individual circuit modules. A module becomes a *rectangular block* after it is assigned dimensions or a shape.[1] These blocks can be either *hard* or *soft*. The dimensions and areas of hard blocks are fixed. For a soft block, the area is fixed but the aspect ratio can be changed, either continuously or in discrete steps. The entire arrangement of blocks, including their positions, is called a *floorplan*. In large designs, individual modules may also be floorplanned in a recursive top-down fashion, but it is common to focus on one hierarchical level of floorplanning at a time. In this case, the floorplan of the highest level is called the *top-level floorplan*.

The floorplanning stage ensures that (1) every chip module is assigned a shape and a location, so as to facilitate gate placement, and (2) every pin that has an external connection is assigned a location, so that internal and external nets can be routed.

The floorplanning stage determines the external characteristics – fixed dimensions and external pin locations – of each module. These characteristics are necessary for the subsequent placement (Chap. 4) and routing (Chaps. 5-7) steps, which determine the internal characteristics of the blocks. Floorplan optimization involves multiple degrees of freedom; while it includes some aspects of placement (finding locations) and connection routing (pin assignment), module shape optimization is unique to floorplanning. Floorplanning with *hard blocks* is particularly relevant when reusing pre-existing blocks, including *intellectual property* (*IP*). Mathematically, this problem can be viewed as a constrained case of floorplanning with *soft* parameters, but in practice, it may require specialized computational techniques.

A floorplanning instance commonly includes the following parameters – (1) the area of each module, (2) all potential aspect ratios of each module, and (3) the netlist of all (external) connections incident to the module.

Example: Floorplan Area Minimization

Given: three modules *a*-*c* with the following potential widths and heights.

a: $w_a = 1, h_a = 4$ or $w_a = 4, h_a = 1$ or $w_a = 2, h_a = 2$

b: $w_b = 1, h_b = 2$ or $w_b = 2, h_b = 1$

c: $w_c = 1, h_c = 3$ or $w_c = 3, h_c = 1$

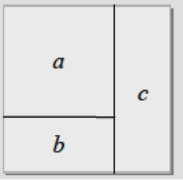
Task: find a floorplan with minimum total area enclosed by its global bounding box (definition in Sec. 3.2).

Solution:

a: $w_a = 2, h_a = 2$ *b*: $w_b = 2, h_b = 1$ *c*: $w_c = 1, h_c = 3$

This floorplan has a global bounding box with minimum possible area (9 square units).

A possible arrangement of the blocks (right).



[1] Blocks can also be rectilinear.

3.2 Optimization Goals in Floorplanning

Floorplan design optimizes both the *locations* and the *aspect ratios* of the individual blocks, using simple objective functions to capture practically desirable floorplan attributes. This section introduces several objective functions for floorplanning. Goals for pin assignment are discussed in Sec. 3.6, and goals for power planning are described in Sec. 3.7.

Area and shape of the global bounding box. The *global bounding box* of a floorplan is the minimum axis-aligned (isothetic) rectangle that contains all floorplan blocks. The area of the global bounding box represents the area of the top-level floorplan (the full design) and directly impacts circuit performance, yield, and manufacturing cost. Minimizing the area of the global bounding box involves finding (x,y) locations, as well as shapes, of the individual modules such that they pack densely together.

Beyond area minimization, another optimization objective is to keep the aspect ratio of the global bounding box as close as possible to a given target value. For instance, due to manufacturing and package size considerations, a square chip (aspect ratio ≈ 1) may be preferable to a non-square chip. To this end, the shape flexibility of the individual modules can be exploited. Area and aspect ratio of the global bounding box are interrelated, and these two objectives are often considered together.

Total wirelength. Long connections between floorplan blocks may increase signal propagation delays in the design. Therefore, layout of high-performance circuits seeks to shorten such interconnects. Switching the logic value carried by a particular net requires energy dissipation that grows with wire capacitance. Therefore, power minimization may also seek to shorten all routes. A third context for wirelength minimization involves routability and manufacturing cost. When the total length of all connections is too high or when the connections are overly dense in a particular region, there may not be enough routing resources to complete all connections. Although circuit blocks may be spread further apart to add new routing tracks, this increases chip size and manufacturing cost, and may further increase net length.

To simplify computation of the total wirelength of the floorplan, one option is to connect all nets to the centers of the blocks. Although this technique does not yield a precise wirelength estimate, it is relatively accurate for medium-sized and small blocks, and enables rapid interconnect evaluation [3.17]. Two common approaches to model connectivity within a floorplan are to use (1) a connection matrix C (Sec. 1.8) representing the union of all nets, along with pairwise distances between blocks, or (2) a minimum spanning tree for each net (Sec. 5.6). Using the first model, the total connection length $L(F)$ of the floorplan F is estimated as

$$L(F) = \sum_{i,j \in F} C[i][j] \cdot d_M(i,j)$$

where element $C[i][j]$ of C is the degree of connectivity between blocks i and j, and $d_M(i,j)$ is the Manhattan distance between the center points of i and j (Sec. 1.8).

Using the second model, the total connection length $L(F)$ is estimated as

$$L(F) = \sum_{net \in F} L_{MST}(net)$$

where $L_{MST}(net)$ is the minimal spanning tree cost of net *net*.

In practice, more sophisticated wirelength objectives are often used. The center-pin location assumption may be improved by using actual pin locations [3.17]. The Manhattan distance wiring cost approximation may be improved by using pin-to-pin shortest paths in a graph representation of available routing resources. This can reflect not only distance, but routing congestion, signal delay, obstacles, and routing channels as well. With these refinements, wiring estimation in a floorplan relies on the construction of heuristic Steiner minimum trees in a weighted graph (Chap. 5).

Combination of area and total wirelength. To reduce both the total area $area(F)$ and the total wirelength $L(F)$ of floorplan F, it is common to minimize

$$\alpha \cdot area(F) + (1 - \alpha) \cdot L(F)$$

where the parameter $0 \leq \alpha \leq 1$ gives the relative importance between $area(F)$ and $L(F)$. Other terms, such as the aspect ratio of the floorplan, can be added to this objective function [3.3]. In practice, the area of the global bounding box may be a constraint rather than an optimization objective. This is appropriate when the package size and its cavity dimensions are fixed, or when the global bounding box is part of a higher-level system organization across multiple chips. In this case, wirelength and other objectives are optimized subject to the constraint that the floorplan fits inside a prescribed global bounding box (the *fixed-outline floorplanning problem*).

Signal delays. Until the 1990s, transistors that made up logic gates were the greatest contributor to chip delay. Since then, due to different delay scaling rates, interconnect delays have gradually become more important, and increasingly determine the chip's achievable clock frequency. Delays of long wires are particularly sensitive to the locations and shapes of floorplan blocks. A desirable quality of a floorplan is short wires connecting its blocks, such that all timing requirements are met. Often, critical paths and nets are given priority during floorplanning so that they span short distances.

Floorplan optimization techniques have been developed that use *static timing analysis* (Sec. 8.2.1) to identify the interconnects that lie on *critical paths*. If timing is violated, i.e., path delays exceed given constraints, the floorplan is modified to shorten critical interconnects and meet timing constraints [3.7].

3.3 Terminology

3.3

A *rectangular dissection* is a division of the chip area into a set of *blocks* or non-overlapping rectangles.

A *slicing floorplan* is a rectangular dissection obtained by repeatedly dividing each rectangle, starting with the entire chip area, into two smaller rectangles using a horizontal or vertical cut line.

A *slicing tree* or *slicing floorplan tree* is a binary tree with k leaves and $k-1$ internal nodes, where each leaf represents a block and each internal node represents a horizontal or vertical cut line (Fig. 3.2). This book uses a standard notation, denoting horizontal and vertical cuts by H and V, respectively. A key characteristic of the slicing tree is that each internal node has exactly two children. Therefore, every slicing floorplan can be represented by at least one slicing tree.

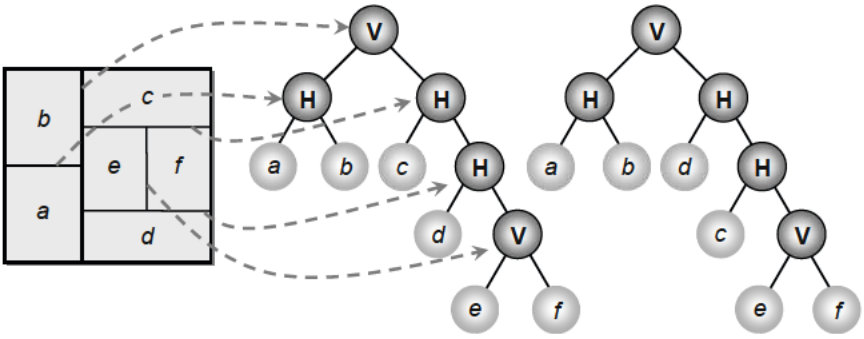


Fig. 3.2 A slicing floorplan of blocks *a-f* and two possible corresponding slicing trees.

A *non-slicing floorplan* is a floorplan that *cannot* be formed by a sequence of only vertical or horizontal cuts in a parent block. The smallest example of a non-slicing floorplan without wasted space is the *wheel* (Fig. 3.3).

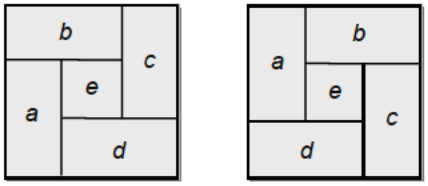


Fig. 3.3 Two different minimal non-slicing floorplans, also known as *wheels*.

A *floorplan tree* is a tree that represents a hierarchical floorplan (Fig. 3.4). Each leaf node represents a block while each internal node represents either a *horizontal cut* (H), a *vertical cut* (V), or a *wheel* (W). The *order* of the floorplan tree is the number of its internal (non-leaf) nodes.

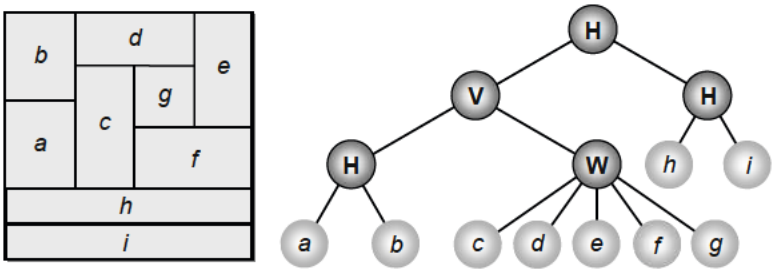


Fig. 3.4 A hierarchical floorplan (left) and its corresponding floorplan tree of order five (right). The internal node *W* represents a wheel with blocks *c*-*g*.

A *constraint-graph pair* is a floorplan representation that consists of two directed graphs – *vertical constraint graph* and *horizontal constraint graph* – which capture the relations between block positions. A constraint graph consists of edges connecting $n + 2$ weighted nodes – one source node s, one sink node t, and n block nodes $v_1, v_2, \ldots, v_n$ representing n blocks $m_1, m_2, \ldots, m_n$. The weight $w(v_i)$ of a block node v_i, $1 \leq i \leq n$, represents the size (relevant dimension) of the corresponding block; the weights of the source node $w(s)$ and sink node $w(t)$ are zero. Fig. 3.5 illustrates a sample floorplan and its corresponding constraint graphs. The process of deriving a constraint-graph pair from a floorplan is discussed in Sec. 3.4.1.

In a *vertical constraint graph* (*VCG*), node weights represent the heights of the corresponding blocks. Two nodes v_i and v_j, with corresponding blocks m_i and m_j, are connected with a directed edge from v_i to v_j if m_i is below m_j.

In a *horizontal constraint graph* (*HCG*), node weights represent the widths of the corresponding blocks. Two nodes v_i and v_j, with corresponding blocks m_i and m_j, are connected with a directed edge from v_i to v_j if m_i is to the left of m_j.

The longest path in the VCG corresponds to the minimum vertical extent required to pack the blocks (floorplan height). Similarly, the longest path in the HCG corresponds to the minimum horizontal extent required (floorplan width). In Fig. 3.5, a longest path is shown in both the HCG and VCG.

A *sequence pair* is an ordered pair (S_+, S_-) of block permutations. Together, the two permutations represent geometric relations between every pair of blocks a and b. Specifically, if a appears before b in both S_+ and S_-, then a is to the left of b. Otherwise, if a appears before b in S_+ but not in S_-, then a is above b.

S_+: <…a…b…> S_-: <…a…b…> if block a is *left of* block b

S_+: <…a…b…> S_-: <…b…a…> if block a is *above* block b

Fig. 3.5 illustrates a sample floorplan and its corresponding sequence pair. The process to derive a sequence pair from a floorplan is discussed in Sec. 3.4.2.

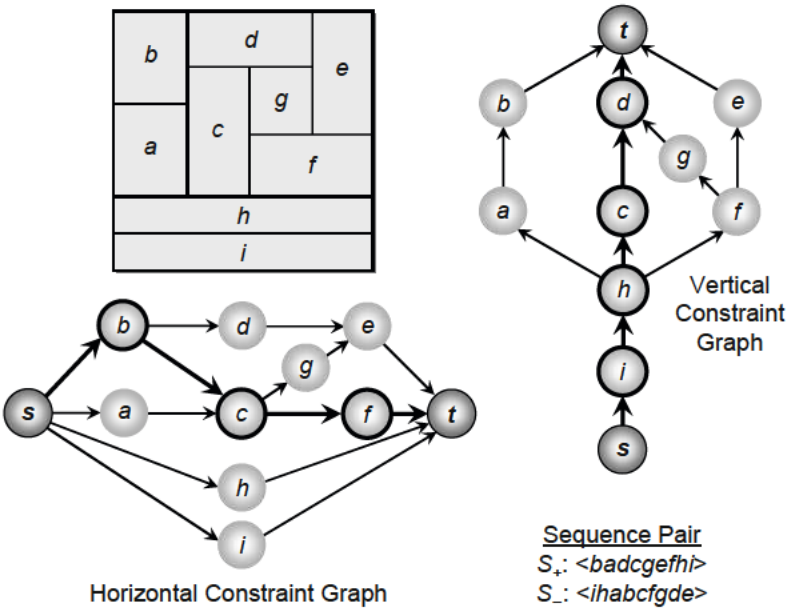


Fig. 3.5 A floorplan of blocks *a*-*i* (top left), its vertical constraint graph (top right), its horizontal constraint graph (bottom left), and its sequence pair (bottom right).

3.4 Floorplan Representations

3.4

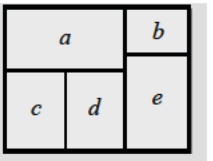
This section discusses how to convert (1) a floorplan into a constraint-graph pair, (2) a floorplan into a sequence pair, and (3) a sequence pair into a floorplan. Note that modern sequence pair algorithms do not compute constraint graphs explicitly. However, the constraint graphs offer a useful, intermediate representation between floorplans and sequence pairs for a better conceptual understanding.

► 3.4.1 Floorplan to a Constraint-Graph Pair

A floorplan can be converted to a constraint-graph pair in three steps. First, for each constraint graph, create a block node v_i for each of the n blocks m_i, $1 \le i \le n$, a source node s and a sink node t. Second, for the vertical (horizontal) constraint graph, add a directed edge (v_i,v_j) if m_i is *below* (*left of*) m_j. Third, for each constraint graph, remove all edges that cannot be derived from other edges by transitivity.

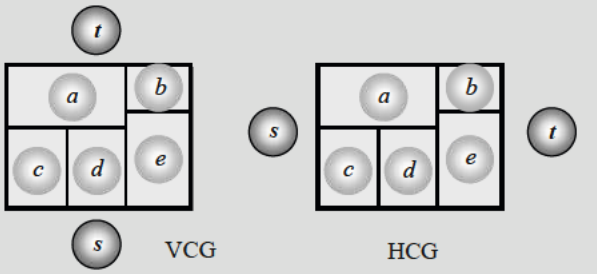
Example: Floorplan to a Constraint-Graph Pair
Given: a floorplan with blocks *a*-*e* (right).

Task: generate the corresponding horizontal constraint graph (HCG) and vertical constraint graph (VCG).



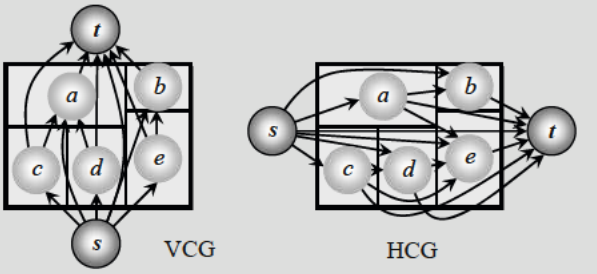
Solution:

VCG and HCG: Create nodes *a-e* for blocks *a-e*. Create the source node *s* and sink node *t*.

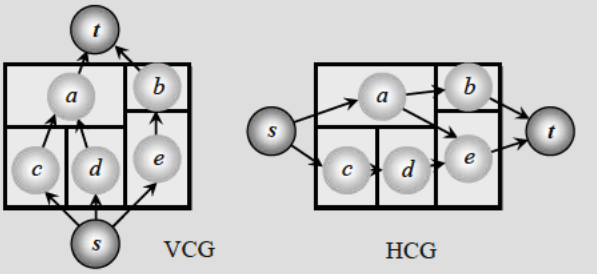


VCG: Add a directed edge (v_i,v_j) if m_i is below m_j.

HCG: Add a directed edge (v_i,v_j) if m_i is left of m_j.



VCG and HCG: Remove all transitive (redundant) edges.



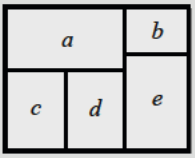
► 3.4.2 Floorplan to a Sequence Pair

A sequence pair (S_+,S_-) encodes the same relations as the constraint-graph pair. Given two blocks a and b, in the sequence pair, if a comes before b in S_+, i.e., <…a…b…>, and a comes *before* b in S_-, i.e., <…a…b…>, then a is to the *left* of b. If a comes before b in S_+, i.e., <…a…b…>, and a comes *after* b in S_-, i.e., <…b…a…>, then a is *above* b. Conceptually, the constraint graphs are generated first, and then the block ordering rules are applied. However, the constraint graphs do not need to be created explicitly. Every pair of *non-overlapping* blocks is ordered either horizontally or vertically, based on the block locations. These ordering relations are encoded as constraints. When either constraint can be chosen, ties are broken, e.g., to always establish a horizontal constraint. Consider blocks a and b with locations (x_a,y_a) and (x_b,y_b) and dimensions (w_a,h_a) and (w_b,h_b), respectively.

$$\text{if } ((x_a + w_a \le x_b) \text{ and not } (y_a + h_a \le y_b \text{ or } y_b + h_b \le y_a)), \text{ then } a \text{ is left of } b$$

$$\text{if } ((y_b + h_b \le y_a) \text{ and not } (x_a + w_a \le x_b \text{ or } x_b + w_b \le x_a)), \text{ then } a \text{ is above } b$$

Example: A Floorplan to a Sequence Pair
Given: a floorplan with blocks *a-e* (right, same as in Sec. 3.4.1).
Task: generate the corresponding sequence pair.



Solution:
Generate the VCG and HCG (example in Sec. 3.4.1). Evaluate each graph independently.

VCG: Blocks *c* and *d* are below block *a*.
Sequence pair so far: S_+: <…*a*…*c*…*d*…> S_-: <…*c*…*d*…*a*...>
VCG: Block *e* is below block *b*.
Sequence pair so far: S_+: <*acdbe*> S_-: <*cdaeb*>

HCG: Block *a* is left of blocks *b* and *e*.
Sequence pair so far: S_+: <*acdbe*> S_-: <*cdaeb*>
HCG: Block *c* is left of block *d*.
Sequence pair so far: S_+: <*acdbe*> S_-: <*cdaeb*>
HCG: Block *d* is left of block *e*.
Sequence pair: S_+: <*acdbe*> S_-: <*cdaeb*>

► 3.4.3 Sequence Pair to a Floorplan

Given a sequence pair, the following additional information is needed to generate a floorplan: (1) the origin of the floorplan, (2) the width and height of each block, and (3) the packing direction of the floorplan. The origin denotes the starting location at which the first block should be placed. The block dimensions are needed to calculate the relative displacements between blocks. The packing direction facilitates area minimization. Given different floorplan packing strategies, e.g., packing as left and downward as possible, or as right and upward as possible, different floorplans can be generated. For this section, assume that the packing direction is left and down.

Evaluating a sequence pair to generate a floorplan is closely related to finding the *weighted longest common subsequence* (*LCS*) of the two sequences [3.13]. That is, finding the *x*-coordinates of each block is equivalent to computing $LCS(S_+,S_-)$, and finding the *y*-coordinates of each block is equivalent to computing $LCS(S_+^R,S_-)$, where S_+^R is the reverse of S_+.

Sequence Pair Evaluation Algorithm
Input: sequence pair <S_+,S_->, widths (heights) of *n* blocks *widths*[*n*] (*heights*[*n*])
Output: *x*- (*y*-) coordinates *x_coords* (*y_coords*), dimensions of floorplan *W* × *H*

```
1.  for (i = 1 to n)
2.      weights[i] = widths[i]                  // weights vector as block widths
3.  (x_coords,W) = LCS(S+,S-,weights)           // x-coordinates, total width W
4.  for (i = 1 to n)
5.      weights[i] = heights[i]                 // weights vector as block heights
6.      S+R[i] = S+[n + 1 – i]                  // reverses S+
7.  (y_coords,H) = LCS(S+R,S-,weights)          // y-coordinates, total height H
```

In the sequence pair evaluation algorithm, lines 1-2 initialize *weights* as the block widths for n blocks. Line 3 computes the x-coordinates of each block of the floorplan by finding the weighted longest common subsequence (*LCS* algorithm) of S_+ and S_-, given *weights*. Lines 4-5 re-initialize *weights* as the block heights. Line 6 reverses S_+ to obtain S_+^R. Line 7 computes the y-coordinates of each block of the floorplan by finding the weighted LCS of S_+^R and S_-, given *weights*.

Longest Common Subsequence (LCS) Algorithm
Input: sequences S_1 and S_2, weights of n elements *weights*[n]
Output: positions of each block *positions*, total span L

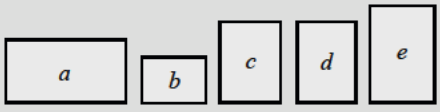
```
1.  for (i = 1 to n)
2.     block_order[S2[i]] = i                    // index in S2 of each block
3.     lengths[i] = 0                            // initialize total span of all blocks
4.  for (i = 1 to n)
5.     block = S1[i]                             // current block
6.     index = block_order[block]                // index in S2 of current block
7.     positions[block] = lengths[index]         // compute position of block
8.     t_span = positions[block] + weights[block]  // find span of current block
9.     for (j = index to n)
10.       if (t_span > lengths[j])
11.          lengths[j] = t_span                 // current block span > previous
12.       else break
13. L = lengths[n]                               // total span
```

The procedure *LCS*(S_1,S_2,*weights*) computes the weighted LCS of two sequences S_1 and S_2 with weights *weights*. The vector *block_order* records the index in S_2 of each block (lines 1-2). Line 3 initializes *lengths*, the vector that stores the (maximum) span (representing the height or width) of each block, to 0. The variable *block* (line 4) corresponds to the current block in S_1 (line 5); *index* is for the index in S_2 of this current block (line 6). The position of *block* is then set to the first position not occupied by other blocks (line 7). That is, all blocks to the left of *block* are packed into the interval up to *lengths*[*block*], and *block* is positioned immediately afterward. Lines 9-12 update *lengths* to reflect the new total span of the floorplan – e.g., the last element *lengths*[n] stores the total span after the locations of all n blocks have been determined (line 13). Lines 10-11 update *lengths*[j] with (position of *block* + weight of *block*) if this exceeds the current span.

To find the x-coordinates of the floorplan, the LCS algorithm is called with $S_1 = S_+$, $S_2 = S_-$, and *weights* = *widths*. To find the y-coordinates, the LCS algorithm is called with $S_1 = S_+^R$, $S_2 = S_-$, and *weights* = *heights*.

Example: Sequence Pair to a Floorplan
Given: (1) sequence pair S_+: <*acdbe*> S_-: <*cdaeb*>, (2) packing direction: left and down, (3) floorplan origin (0,0), and (4) blocks *a*-*e* with their dimensions.

A: $(w_a,h_a) = (8,4)$ B: $(w_b,h_b) = (4,3)$
C: $(w_c,h_c) = (4,5)$ D: $(w_d,h_d) = (4,5)$
E: $(w_e,h_e) = (4,6)$



Task: generate the corresponding floorplan.

Solution:
widths[*a b c d e*] = [8 4 4 4 4] *heights*[*a b c d e*] = [4 3 5 5 6]

Find *x*-coordinates.
$S_1 = S_+ =$ <*acdbe*>, $S_2 = S_- =$ <*cdaeb*>
weights[*a b c d e*] = *widths*[*a b c d e*] = [8 4 4 4 4]
block_order[*a b c d e*] = [3 5 1 2 4]
lengths = [0 0 0 0 0]

Iteration *i* = 1: *block* = *a*
index = *block_order*[*a*] = 3
positions[*a*] = *lengths*[*index*] = *lengths*[3] = 0
t_span = *positions*[*a*] + *weights*[*a*] = 0 + 8 = 8
Update *lengths* vector from *index* = 3 to *n* = 5: *lengths* = [0 0 **8 8 8**]

Iteration *i* = 2: *block* = *c*
index = *block_order*[*c*] = 1
positions[*c*] = *lengths*[*index*] = *lengths*[1] = 0
t_span = *positions*[*c*] + *weights*[*c*] = 0 + 4 = 4
Update *lengths* vector from *index* = 1 to *n* = 5: *lengths* = [**4 4 8 8 8**]

Iteration *i* = 3: *block* = *d*
index = *block_order*[*d*] = 2
positions[*d*] = *lengths*[*index*] = *lengths*[2] = 4
t_span = *positions*[*d*] + *weights*[*d*] = 4 + 4 = 8
Update *lengths* vector from *index* = 2 to *n* = 5: *lengths* = [4 **8 8 8 8**]

Iteration *i* = 4: *block* = *b*
index = *block_order*[*b*] = 5
positions[*b*] = *lengths*[*index*] = *lengths*[5] = 8
t_span = *positions*[*b*] + *weights*[*b*] = 8 + 4 = 12
Update *lengths* vector from *index* = 5 to *n* = 5: *lengths* = [4 8 8 8 **12**]

Iteration *i* = 5: *block* = *e*
index = *block_order*[*e*] = 4
positions[*e*] = *lengths*[*index*] = *lengths*[4] = 8
t_span = *positions*[*e*] + *weights*[*e*] = 8 + 4 = 12
Update *lengths* vector from *index* = 4 to *n* = 5: *lengths* = [4 8 8 **12 12**]

x-coordinates: *positions*[*a b c d e*] = [0 8 0 4 8], width of floorplan *W* = *lengths*[5] = 12.

Find *y*-coordinates.
$S_1 = S_+^R =$ <*ebdca*>, $S_2 = S_- =$ <*cdaeb*>
weights[*a b c d e*] = *heights*[*a b c d e*] = [4 3 5 5 6]
block_order[*a b c d e*] = [3 5 1 2 4]
lengths = [0 0 0 0 0]

Iteration $i = 1$: $block = e$
$index = block_order[e] = 4$
$positions[e] = lengths[index] = lengths[4] = 0$
$t_span = positions[e] + weights[e] = 0 + 6 = 6$
Update *lengths* vector from $index = 4$ to $n = 5$: $lengths = [0\ 0\ 0\ \mathbf{6}\ \mathbf{6}]$

Iteration $i = 2$: $block = b$
$index = block_order[b] = 5$
$positions[b] = lengths[index] = lengths[5] = 6$
$t_span = positions[b] + weights[b] = 6 + 3 = 9$
Update *lengths* vector from $index = 5$ to $n = 5$: $lengths = [0\ 0\ 0\ 6\ \mathbf{9}]$

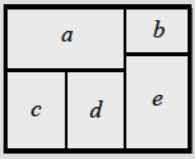
Iteration $i = 3$: $block = d$
$index = block_order[d] = 2$
$positions[d] = lengths[index] = lengths[2] = 0$
$t_span = positions[d] + weights[d] = 0 + 5 = 5$
Update *lengths* vector from $index = 2$ to $n = 5$: $lengths = [0\ \mathbf{5}\ \mathbf{5}\ \mathbf{6}\ \mathbf{9}]$

Iteration $i = 4$: $block = c$
$index = block_order[c] = 1$
$positions[c] = lengths[index] = lengths[1] = 0$
$t_span = positions[c] + weights[c] = 0 + 5 = 5$
Update *lengths* vector from $index = 1$ to $n = 5$: $lengths = [\mathbf{5}\ \mathbf{5}\ \mathbf{5}\ \mathbf{6}\ \mathbf{9}]$

Iteration $i = 5$: $block = a$
$index = block_order[a] = 3$
$positions[a] = lengths[index] = lengths[3] = 5$
$t_span = positions[a] + weights[a] = 5 + 4 = 9$
Update *lengths* vector from $index = 3$ to $n = 5$: $lengths = [5\ 5\ \mathbf{9}\ \mathbf{9}\ \mathbf{9}]$

y-coordinates: $positions[a\ b\ c\ d\ e] = [5\ 6\ 0\ 0\ 0]$, height of floorplan $H = lengths[5] = 9$.

Floorplan size: $W = 12 \times H = 9$
Coordinates of blocks *A*-*E*:
a (0,5) *b* (8,6) *c* (0,0) *d* (4, 0) *e* (8,0)



3.5 3.5 Floorplanning Algorithms

This section presents several algorithms used in floorplan optimization. Given a set of blocks, *floorplan sizing* determines the minimum area of the floorplan as well as the associated orientations and dimensions of each individual block. Techniques such as *cluster growth* and *simulated annealing* comprehend the netlist of interconnects between blocks and seek to (1) minimize the total length of interconnect, subject to an upper bound on the floorplan area, or (2) simultaneously optimize wirelength and area.

► 3.5.1 Floorplan Sizing

Floorplan sizing finds the dimensions of the minimum-area floorplan and corresponding dimensions of the individual blocks. The earliest algorithms, due to *Otten* [3.8] and *Stockmeyer* [3.11], make use of flexible dimensions for the individual blocks to find minimal top-level floorplan areas and shapes. A minimal top-level floorplan is selected, and the individual block shapes are chosen accordingly. Since the algorithm uses the shapes of both the individual blocks and the top-level floorplan, *shape functions* and *corner points* (limits) play a major role in determining an optimal floorplan.

Shape functions (shape curves) and **corner points.** Consider a block *block* with area *area*(*block*). By definition, if a block has width w_{block} and height h_{block}, then

$$w_{block} \cdot h_{block} \geq area(block)$$

This relation can be rewritten in terms of width as a *shape function* (Fig. 3.6(a)):

$$h_{block}(w) = \frac{area(block)}{w}$$

That is, the shape function $h_{block}(w)$ states that for a given block width w, any block height $h \geq h_{block}(w)$ is legal. Shape functions can also include lower bounds on the block's width $LB(w_{block})$ and height $LB(h_{block})$. With such lower bounds, some (h,w) pairs are excluded (Fig. 3.6(b)).

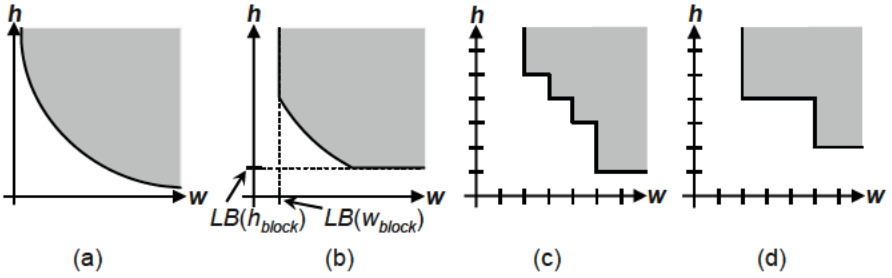


Fig. 3.6 Shape functions of blocks. The gray regions contains all (h,w) pairs that form a valid block [3.3]. (a) Shape function with no restrictions. (b) Shape function with minimum width $LB(w_{block})$ and height $LB(h_{block})$ restrictions for a block *block*. (c) Shape function with discrete (h,w) values. (d) Shape function of a possible hard library block, where its orientation can be more restricted.

Due to technology-dependent *design rules*, (h,w) pairs can be restricted to discrete values (Fig. 3.6(c)). Certain block libraries can impose even stronger restrictions. For instance, both the range of block dimensions, as well as the orientation of the block can be limited (Fig. 3.6(d)). Typically, blocks are also allowed to be reflected about the x- or y-axis.

The discrete dimensions of the block, sometimes called *outer* nodes (with respect to the feasible region of (*h*,*w*) pairs), can be thought of as non-dominated *corner points* that limit the shape function (Fig. 3.7). One academic floorplanner, *DeFer*, extends this calculus of shape functions to simultaneously optimize the shapes and locations of blocks in slicing floorplans [3.16].

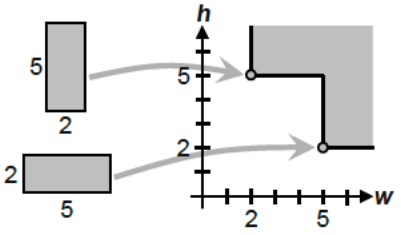


Fig. 3.7 A 2 × 5 rotatable library block that has corner points $w = 2$ and $w = 5$ in its shape function.

Minimum-area floorplan. This algorithm finds the minimum floorplan area for a given slicing floorplan in polynomial time. For non-slicing floorplans, the problem is *NP-hard* [3.3].

Floorplan sizing consists of three major steps.

1. Construct the shape functions of all individual blocks.
2. Determine the shape function of the top-level floorplan from the shape functions of the individual blocks using a bottom-up strategy. That is, start with the lowest-level blocks and perform horizontal and vertical composition until the optimal size and shape of the top-level floorplan are determined.
3. From the corner point that corresponds to the minimum top-level floorplan area, trace in a top-down fashion back to each block's shape function to find that block's dimensions and location.

Step 1: Construct the shape functions of the blocks
Since the top-level floorplan's shape function depends upon the shape functions of individual blocks, the shape functions of all blocks must be identified first (Fig. 3.8).

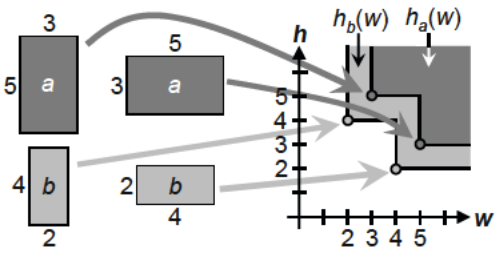


Fig. 3.8 Shape functions of two library blocks *a* (5 × 3) and *b* (4 × 2). The shape functions $h_a(w)$ and $h_b(w)$ show the feasible height-width combinations of the blocks.

Step 2: Determine the shape function of the top-level floorplan
The shape function of the top-level floorplan is derived from the individual blocks' shape functions. Two different kinds of *composition* – *vertical* and *horizontal* – can yield different results.

In Fig. 3.9, block a is *vertically* aligned with block b. Let the shape function of block a be $h_a(w)$, and let the shape function of block b be $h_b(w)$. Then, the shape function of the top-level floorplan F is

$$h_F(w) = h_a(w) + h_b(w)$$

The height of F is determined by h_F, adding $h_a(w)$ and $h_b(w)$ for every corner point. The width of F is found as $w_F = \max(w_a, w_b)$.

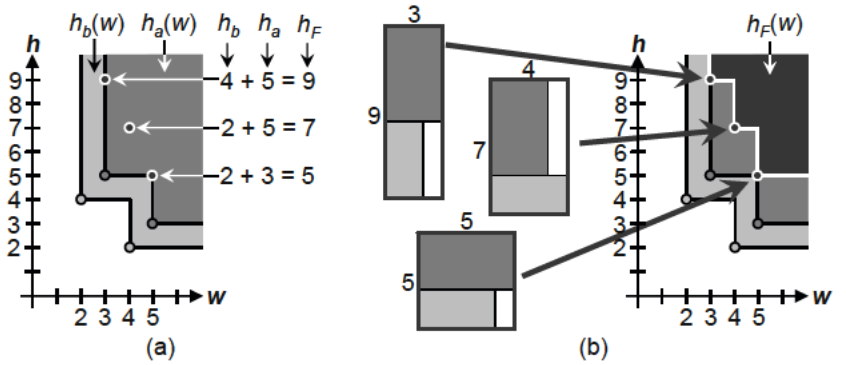


Fig. 3.9 Vertical composition of two library blocks a (5 × 3) and b (4 × 2) superimposed for vertical composition. (a) To find the smallest bounding box for the top-level floorplan $h_F(w)$ from the functions $h_a(w)$ and $h_b(w)$, the block heights of the respective corner points are added. In this example, $w = 3$ and $w = 5$ (from $h_a(w)$) are combined with $w = 4$ (from $h_b(w)$). Note that $w = 2$ (from $h_b(w)$) is ignored, since the width of the top-level floorplan cannot be smaller than 3, which is the width of block a. (b) The potential floorplans of F are the corner points of $h_F(w)$.

In Fig. 3.10, block a is *horizontally* aligned with block b. The width of F is determined by $w_F(h)$, adding $w_a(h)$ and $w_b(h)$ for every corner point. The height of F is $h_F = \max(h_a, h_b)$.

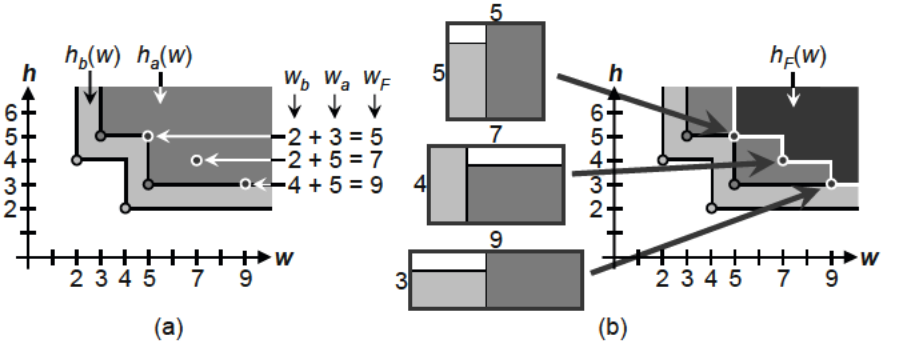


Fig. 3.10 Shape functions of two library blocks a (5 × 3) and b (4 × 2) superimposed for horizontal composition. (a) Finding $h_F(w)$. (b) The potential floorplans of F are the corner points of $h_F(w)$.

Step 3: Find the floorplan and individual blocks' dimensions and locations
Once the shape function of the top-level floorplan has been determined, the minimum floorplan area is computed. All minimum-area floorplans are always on corner points of the shape function (Fig. 3.11). After finding the corner point for the minimum area, each individual block's dimensions and relative location are found by backtracing from the floorplan's shape function to the block's shape function.

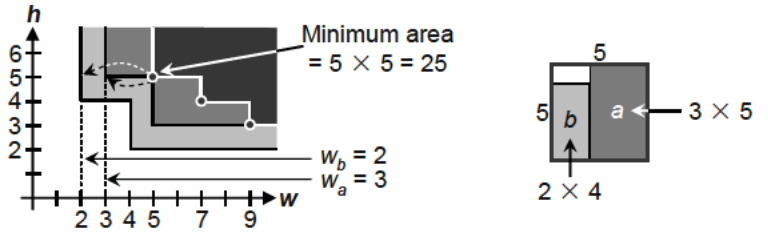
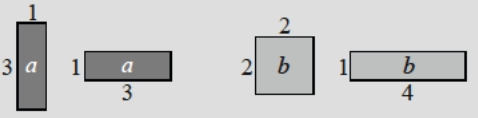


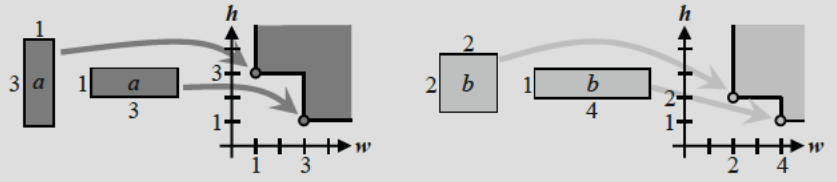
Fig. 3.11 Finding the shapes of the individual blocks from the same top-level floorplan example (Fig. 3.9 and Fig. 3.10) with library blocks a (5 × 3) and b (2 × 4). The minimum-area floorplan F has $w_F = 5$ and $h_F = 5$ (left). The dimensions and locations of the individual blocks a and b (right) are derived by tracing back to the blocks' respective shape functions.

Example: Floorplan Sizing
Given: two blocks a and b.
a: $w_a = 1$, $h_a = 3$ or $w_a = 3$, $h_a = 1$
b: $w_b = 2$, $h_b = 2$ or $w_b = 4$, $h_b = 1$

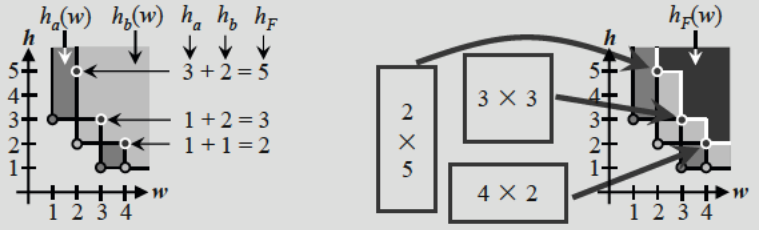


Task: find the minimum-area floorplan F using both horizontal and vertical composition, and its corresponding slicing tree.

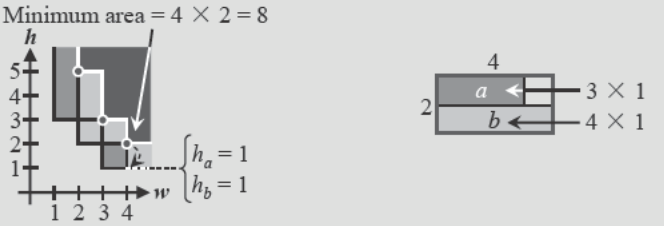
Solution:
Construct the shape functions $h_a(w)$ and $h_b(w)$ of blocks a (left) and b (right).



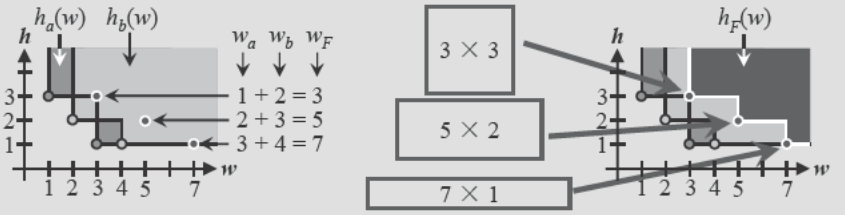
Vertical composition: determine the shape function $h_F(w)$ of F and the minimum-area corner point. The minimum area of F is 8, with dimensions $w_F = 4$ and $h_F = 2$.



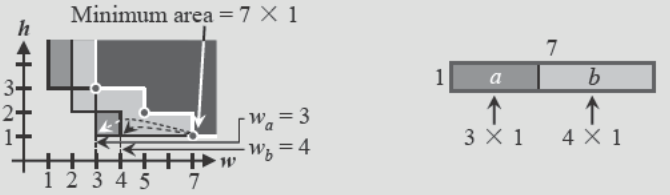
Find the dimensions and locations of blocks a and b.



Horizontal composition: determine the shape function $h_F(w)$ of F and the minimum-area corner point. The minimum area of F is 7, with dimensions $w_F = 7$ and $h_F = 1$.



Find the dimensions and locations of blocks a and b.



Slicing tree of the floorplan with minimum area = min(8,7):



► 3.5.2 Cluster Growth

In a cluster growth-based method, the floorplan is constructed by iteratively adding blocks until all blocks have been assigned (Fig. 3.12). An initial block is chosen and placed in the lower-left (or any other) corner. Subsequent blocks are added, one at a time, and merged either horizontally, vertically, or diagonally with the cluster. The location and orientation of the next block depends on the current shape of the cluster; it will be placed to best accommodate the objective function of the floorplan. In contrast to the floorplan-sizing algorithm, only the different orientations of the individual blocks are taken into account. Methods that directly and simultaneously optimize both the shapes of each block and the floorplan are not currently known. The order of the blocks is typically chosen by a linear-ordering algorithm.

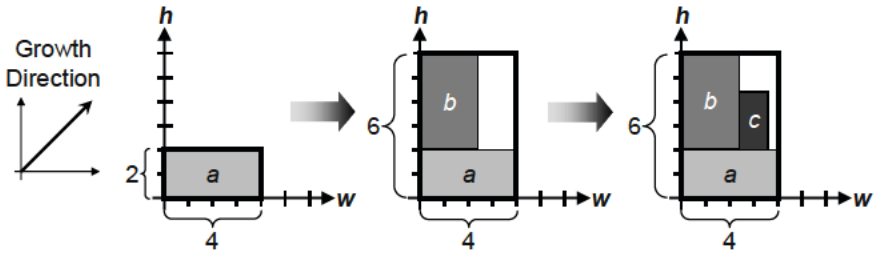


Fig. 3.12 Floorplan construction with a cluster-growth algorithm based on minimum floorplan area. Block *a* is placed first. Blocks *b* and *c* are placed so that the increase to the floorplan's dimensions is minimum.

Linear ordering. Linear-ordering algorithms are often invoked to produce initial placement solutions for iterative-improvement placement algorithms. The objective of linear ordering is to arrange given blocks in a single row so as to minimum the total wirelength of connections between blocks.

Kang [3.4] classified incident nets of a given block into the following three categories with respect to a partially constructed, left-to-right ordering (Fig. 3.13).

- *Terminating nets* have no other incident blocks that are unplaced.
- *New nets* have no pins on any block from the partially-constructed ordering.
- *Continuing nets* have at least one pin on a block from the partially-constructed ordering and at least one pin on an unordered block.

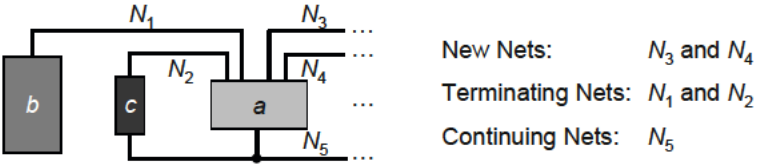


Fig. 3.13 Classification of nets based on the partially-constructed linear ordering.

The ordering of a particular block m directly depends on the types of nets attached to m. Specifically, the block that completes the greatest number of "unfinished" nets should be placed first. In other words, the block with the highest difference between the number of terminating and new nets is chosen as the next block in the sequence.

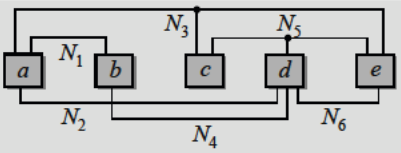
The linear-ordering algorithm starts by choosing an initial block. This can be done either arbitrarily or based on the number of connections to the other blocks (line 1). During every iteration, the gain *gain* for each block $m \in M$ is calculated (lines 5-8), where *gain* is the difference between the number of terminating nets and new nets of m. The blocks with the maximum gain are selected (line 9). In case there are multiple blocks that have the maximum gain value, the blocks with the most terminating nets are selected (lines 10-11). If there are still multiple blocks, the

blocks with the most continuing nets are selected (lines 12-13). If there are still multiple blocks, the blocks with the lowest connectivity are selected (lines 14-15). If there are still multiple blocks, a block is selected arbitrarily (lines 16-17). The selected block is then added to *order* and removed from *M* (lines 18-19).

Linear Ordering Algorithm [3.4]
Input: set of all blocks *M*
Output: ordering of blocks *order*

```
1.   seed = starting block
2.   order = seed
3.   REMOVE(M,seed)                          // remove seed from M
4.   while (|M| > 0)                         // while M is not empty
5.     foreach (block m ∈ M)
6.       term_nets[m] = number of terminating nets incident to m
7.       new_nets[m] = number of new nets incident to m
8.       gain[m] = term_nets[m] – new_nets[m]
9.     M' = block(s) that have maximum gain[m]
10.    if (|M'| > 1)                         // multiple blocks
11.      M' = block(s) with the most terminating nets
12.    if (|M'| > 1)                         // multiple blocks
13.    M' = block(s) with the most continuing nets
14.    if (|M'| > 1)                         // multiple blocks
15.      M' = block(s) with the fewest connected nets
16.    if (|M'| > 1)                         // multiple blocks
17.      M' = arbitrary block in M
18.    ADD(order,M')                         // add M' to order
19.    REMOVE(M,M')                          // remove M' from M
```

Example: Linear Ordering Algorithm
Given: (1) netlist with five blocks *a*-*e*,
(2) starting block *a*, and (3) six nets N_1-N_6.
$N_1 = (a,b)$ $N_2 = (a,d)$ $N_3 = (a,c,e)$
$N_4 = (b,d)$ $N_5 = (c,d,e)$ $N_6 = (d,e)$



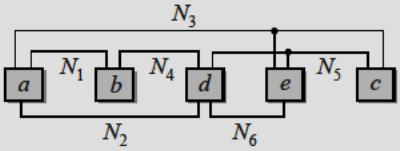
Task: find a linear ordering using the linear ordering algorithm.

Solution:

Iteration #	Block	New Nets	Terminating Nets	*gain*	Continuing Nets
0	***a***	N_1,N_2,N_3	--	-3	--
1	***b***	N_4	N_1	0	--
	c	N_5	--	-1	N_3
	d	N_4,N_5,N_6	N_2	-2	--
	e	N_5,N_6	--	-2	N_3
2	***c***	N_5	--	-1	N_3
	d	N_5,N_6	N_2,N_4	0	--
	e	N_5,N_6	--	-2	N_3
3	***c***	--	--	0	N_3,N_5
	e	--	N_6	1	N_3,N_5
4	***c***	--	N_3,N_5	2	--

For each iteration, **bold font** denotes the block with maximum gain.
Iteration 0: set block *a* as the first block in the ordering.
Iteration 1: block *b* has maximum gain. Set as the second block in the ordering.
Iteration 2: block *d* has maximum gain. Set as the third block in the ordering.
Iteration 3: block *e* has maximum gain. Set as the fourth block in the ordering.
Iteration 4: set *c* as the fifth (last) block in the ordering.

The linear ordering that heuristically minimizes total net cost is <*a b d e c*>.



Cluster growth. In the cluster-growth algorithm, the blocks are ordered using the linear ordering algorithm (line 2). For each block *curr_block* (line 4), the algorithm finds a location such that the floorplan grows evenly – toward the upper and right sides – while satisfying criteria such as the shape constraint of the top-level floorplan (line 5). Other typical criteria include total wirelength or the amount of *deadspace* within the floorplan. This algorithm is similar to the *Tetris* algorithm used for cell legalization (Sec. 4.4).

Cluster-Growth Algorithm
Input: set of all blocks *M*, cost function *C*
Output: optimized floorplan *F* based on *C*

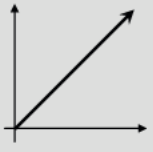
```
1.  F = Ø
2.  order = LINEAR_ORDERING(M)              // generate linear ordering
3.  for (i = 1 to |order|)
4.     curr_block = order[i]
5.     ADD_TO_FLOORPLAN(F,curr_block,C)     // find location and orientation
                                            // of curr_block that causes
                                            // smallest increase based on
                                            // C while obeying constraints
```

The example below illustrates the cluster-growth algorithm. It uses the same linear ordering found in the previous example. The objective is to find the smallest floorplan, i.e., to minimize the area of the top-level floorplan. Though it produces mediocre solutions, the cluster-growth algorithm is fast and easy to implement; therefore, it is often used to find initial floorplan solutions for iterative algorithms such as *simulated annealing*.

Example: Floorplan Construction by Cluster Growth
Given: (1) blocks *a*-*e* and (2) linear ordering <*a b d e c*>.

a: $w_a = 2, h_a = 3$ or $w_a = 3, h_a = 2$
b: $w_b = 2, h_b = 1$ or $w_b = 1, h_b = 2$
c: $w_c = 2, h_c = 4$ or $w_c = 4, h_c = 2$
d: $w_d = 3, h_d = 3$
e: $w_e = 6, h_e = 1$ or $w_e = 1, h_e = 6$

Growth Direction:

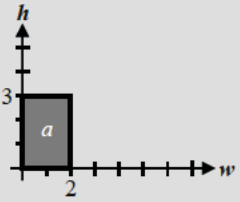


Task: find a floorplan with minimum global bounding box area.

Solution: (multiple solutions possible; one is illustrated below)

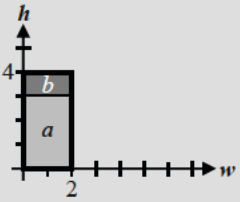
Block a: place in lower-left corner with $w_a = 2$ and $h_a = 3$.

Current bounding box area = $2 \times 3 = 6$.



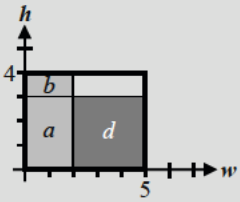
Block b: place above block a with $w_b = 2$ and $h_b = 1$.

Current bounding box area = $2 \times 4 = 8$.



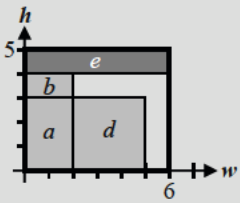
Block d: place to the right of block a with $w_d = 3$ and $h_d = 3$.

Current bounding box area = $5 \times 4 = 20$.



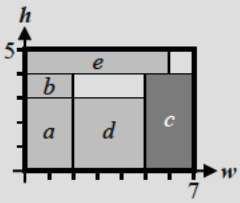
Block e: place above block b with $w_e = 6$ and $h_e = 1$.

Current bounding box area = $6 \times 5 = 30$.



Block c: place to the right of block d with $w_c = 2$ and $h_c = 4$.

Current bounding box area = $7 \times 5 = 35$.



► 3.5.3 Simulated Annealing

Simulated annealing (*SA*) algorithms are iterative in nature – they begin with an initial (arbitrary) solution and seek to incrementally improve the objective function. During each iteration, a local *neighborhood* of the current solution is considered. A new candidate solution is formed by a small perturbation to the current solution within its neighborhood.

Unlike *greedy algorithms*, SA algorithms do not always reject candidate solutions with higher cost. That is, in a greedy algorithm, if the new solution is better, e.g., has

lower cost (assuming a minimization objective), than the current solution, it is *accepted* and replaces the current solution. If no better solution exists in the local neighborhood of the current solution, the algorithm has reached a *local minimum*. The key drawback of greedy algorithms is only beneficial changes to the current solution are accepted.

Fig. 3.14 illustrates how the greedy approach breaks down. Starting from initial solution *I*, the algorithm goes *downhill* and eventually reaches solution state *L*. However, this is only a *local minimum*, not the *global minimum G*. A greedy iterative improvement algorithm, unless given a fortuitous initial state, will be unable to reach *G*. On the other hand, iterative approaches that accept inferior (non-improving) solutions can *hill-climb* away from *L* and potentially reach *G*. The simulated annealing algorithm is one of the most successful optimization strategies to integrate hill-climbing with iterative improvement.

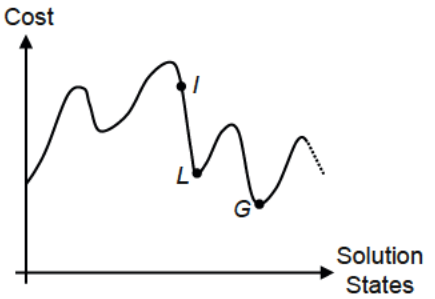


Fig. 3.14 For a minimization problem, an initial solution *I*, along with local optimum (*L*) and global optimum (*G*) solutions. In the neighborhood structure shown, the cost function has several local minima, as is the case with most (intractable) layout problems.

Principle of simulated annealing. In materials science, *annealing* refers to the controlled cooling of high-temperature materials to modify their properties. The goal of annealing is to alter the atomic structure of the material and reach a *minimum-energy configuration*. For instance, the atoms of a high-temperature metal are in high-energy, disordered states (chaos), while the atoms of a low-temperature metal are in low-energy, ordered states (crystalline structures). However, the same metal may experience atomic configurations that are brittle or mechanically hard, depending on the size of the individual crystals. When the high-temperature metal is cooled, it goes from a highly randomized state to a more structured state. The rate at which the metal is cooled will drastically affect the final structure. Moreover, the way in which the atoms settle is *probabilistic* in nature. In practice, a slower cooling process implies a greater chance that the atoms will settle as a perfect lattice, forming a minimum-energy configuration.

The cooling process occurs in steps. At each step of the *cooling schedule*, the temperature is held constant for a specified amount of time. This allows the atoms to gradually cool down and stabilize, i.e., reach a *thermodynamic equilibrium*, at each given temperature. Though the atoms have the ability to move across large distances and create new higher-energy states, the probability of such drastic changes in this configuration decreases with temperature.

As the cooling process continues, the atoms eventually will settle in a local, and possibly global, minimum-energy configuration. Both the rate and step size of the temperature decrease will affect how the atoms will settle. If the rate is sufficiently slow and the increment is sufficiently small, the atoms will settle, with high probability, at a global minimum. On the other hand, if cooling is too fast or the increment is too large, then the atoms are less likely to attain the global minimum-energy configuration, and instead will settle in a local minimum instead.

Annealing-based optimization. The principle of annealing can be applied to solve combinatorial optimization problems. In the context of minimization, finding the lowest-cost solution in an optimization problem is analogous to finding a minimum-energy state of a material. Thus, *simulated annealing* algorithms take a "chaotic" (higher-cost) solution and emulate physical annealing to produce a "structured" (lower-cost) solution.

The simulated annealing algorithm generates an initial solution and evaluates its cost. At each step, the algorithm generates a new solution by performing a *random walk* in the solution by applying a small perturbation (change in structure). This new solution is then *accepted* or *rejected* based on a *temperature* parameter T. When T is high (low), the algorithm has a higher (lower) chance of accepting a solution with higher cost. Analogous to physical annealing, the algorithm slowly decreases T, which correspondingly decreases the probability of accepting an inferior, higher-cost solution. One method for probabilistically accepting moves is based on the *Boltzmann acceptance criterion*, where the new solution is acceptance if

$$e^{\frac{cost(curr_sol) \;\; cost(next_sol)}{T}} > r$$

Here, *curr_sol* is the current solution, *next_sol* is the new solution after a perturbation, T is the current temperature, and r is a random number between [0,1) based on a uniform distribution. For a minimization problem, the final solution will be in a *valley*; for a maximization problem, it will be at a *peak*.

The rate of temperature decrease is extremely important – it (1) must enable sufficient high-temperature exploration of the solution space at the beginning, while (2) allowing enough time at low temperatures to have sufficient probability of settling at a near-optimal solution. Just as slow cooling of high-temperature metal has a high probability of finding a globally optimal, energy-minimal crystal lattice, a simulated annealing algorithm with a sufficiently slow cooling schedule has high probability of finding a high-quality solution for a given optimization problem [3.5].

The simulated annealing algorithm is *stochastic* by nature – two runs usually yield two different results. The difference in quality stems from probabilistic decisions such as generation of new, perturbed solutions (e.g., by a cell swap), and the acceptance or rejection of moves.

Algorithm. The algorithm begins with an initial (arbitrary) solution *curr_sol* (lines 3-4), and generates a new solution by perturbing *curr_sol* (lines 8-9). The resulting new cost *trial_cost* is computed (line 10), and compared with the current cost *curr_cost* (line 11). If the new cost is better, i.e., $\Delta cost < 0$, the change of solution is accepted (lines 12-14). Otherwise, the change may still be probabilistically accepted. A random number $0 \le r < 1$ is generated (line 16), and if r is smaller than $e^{-\Delta cost/T}$, the change is accepted. Otherwise, the change is rejected (lines 17-19).

To perform maximization, reverse the order of operands for the subtraction in line 11. In the context of floorplanning, the function TRY_MOVE (line 9) can be replaced with operations such as moving a single block or swapping two blocks.

Simulated Annealing Algorithm
Input: initial solution *init_sol*
Output: optimized new solution *curr_sol*

```
1.   T = T0                                   // initialization
2.   i = 0
3.   curr_sol = init_sol
4.   curr_cost = COST(curr_sol)
5.   while (T > Tmin)
6.     while (stopping criterion is not met)
7.       i = i + 1
8.       (ai,bi) = SELECT_PAIR(curr_sol)     // select two objects to perturb
9.       trial_sol = TRY_MOVE(ai,bi)         // try small local change
10.      trial_cost = COST(trial_sol)
11.      Δcost = trial_cost – curr_cost
12.      if (Δcost < 0)                      // if there is improvement,
13.        curr_cost = trial_cost            //   update the cost and
14.        curr_sol = MOVE(ai,bi)            //   execute the move
15.      else
16.        r = RANDOM(0,1)                   // random number [0,1)
17.        if (r < e^(-Δcost/T))             // if it meets threshold,
18.          curr_cost = trial_cost          //   update the cost and
19.          curr_sol = MOVE(ai,bi)          //   execute the move
20.    T = α · T                             // 0 < α < 1, T reduction
```

The probability of the exchange depends on both cost and temperature, and a larger (worse) cost difference implies a smaller chance that the change is accepted. At high temperatures, i.e., $T \to \infty$, $-\Delta cost / T \approx 0$ and the probability of accepting a change approaches $e^0 = 1$. Again, this leads to frequent acceptance of inferior solutions, which helps the algorithm escape "low-quality" regions of the solution space. On the other hand, at low temperatures, i.e., $T \to 0$, $-\Delta cost / T \approx \infty$ and the probability of accepting a change approaches $e^{\infty} = 1/e^{\infty} = 0$. The initial temperature (T_0) and the degradation rate (α) are usually set empirically. In general, better results are produced with a high initial temperature and a slow rate of cooling, at the cost of increased runtime.

Simulated annealing-based floorplanning. The first simulated annealing algorithm for floorplanning was proposed in 1984 by *R. Otten* and *L. van Ginneken* [3.9]. Since then, simulated annealing has become one of the most common iterative methods used in floorplanning.

In the *direct approach*, SA is applied directly to the physical layout, using the actual coordinates, sizes, and shapes of the blocks. However, finding a fully legal solution – a floorplan with no block overlaps – is difficult. Thus, intermediate solutions are allowed to have overlapping blocks, and a penalty function is incorporated to encourage legal solutions. The final produced solution, though, must be completely *legal* (see [3.10] for further reading).

In the *indirect approach*, simulated annealing is applied to an abstraction of the physical layout. Abstract representations capture the floorplan using trees or constraint graphs. A final mapping is also required to generate the floorplan from the abstract representation. One advantage of this process over the direct approach is that all intermediate solutions are overlap-free.

For further reading on simulated annealing-based floorplanning, see [3.1], [3.3], [3.14] and [3.15].

► 3.5.4 Integrated Floorplanning Algorithms

Analytic techniques map the floorplanning problem to a set of equations where the variables represent block locations. These equations describe boundary conditions, attempt to prevent block overlap, and capture other relations between blocks. In addition, an objective function quantifies the important parameters of the floorplan.

One well-known analytic method is *mixed integer-linear programming* (*MILP*), where the location variables are integers. This technique does not allow for overlaps and seeks globally optimal solutions. However, it is limited due to its computational complexity. For a problem size of 100 blocks, the integer program can have over 10,000 variables and over 20,000 equations. Thus, MILP is usable only for small (10 or fewer blocks) instances.

A faster alternative that offers some compromises is to use a linear programming (LP) relaxation. Compared to MILP, the LP formulation does not limit the locations to be integers. However, LP can be used for larger problem instances.

For further discussion of floorplanning with analytic methods, see [3.1]. A technique for *floorplan repair* (legalization) is described in [3.7].

3.6 Pin Assignment

Given the large geometric sizes of blocks during floorplanning, the terminal locations of nets connecting these blocks are very important. *I/O pins* (net terminals) and their locations are usually on the periphery of a block to reduce interconnect length. However, the best locations depend on the relative placement of the blocks.

Problem formulation. *During pin assignment, all nets (signals) are assigned to unique pin locations such that the overall design performance is optimized. Common optimization goals include maximizing routability and minimizing electrical parasitics both inside and outside of the block.*

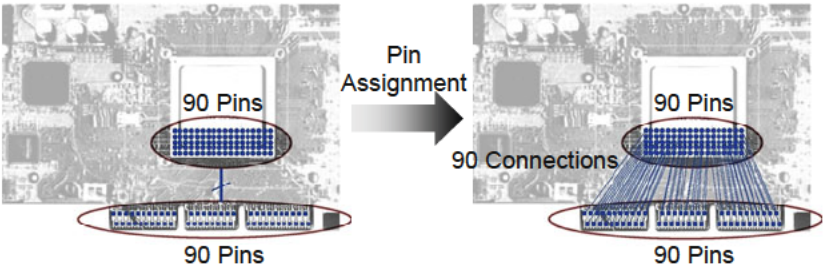


Fig. 3.15 The pin assignment process. Here, each of the 90 I/O pins from the chip is assigned a specific I/O pin on a printed circuit board. Each pin pair is then connected by a route.

The goal of *external pin assignment* is to connect each incoming or outgoing signal to a unique I/O pin. Once the necessary nets have each been assigned a unique pin, they must be connected such that wirelength and electrical parasitics, e.g., coupling or reduced signal integrity, are minimized. For instance, Fig. 3.15 shows 90 pins on the microprocessor chip, each of which must be connected to an I/O pad at the next hierarchy level. After pin assignment, each pin from the chip has a connection to a unique pin on the external device, connected with short routes.

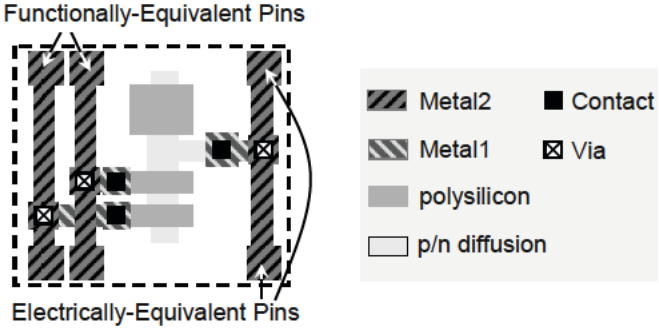


Fig. 3.16 Functionally-equivalent input pins and electrically-equivalent output pins for a simplified example of an nMOS NAND gate.

Alternatively, pin assignment can be used to connect cell pins that are *functionally* or *electrically equivalent*, such as during standard-cell placement (Chap. 4). Two pins are functionally equivalent if swapping them does not affect the design's logic and electrically equivalent (equipotential) if they are connected (Fig. 3.16).

The main objective of *internal pin assignment* for cells is to reduce congestion and interconnect length between the cells (Fig. 3.17). Pin assignment techniques discussed below apply to both chip planning and placement (Chap. 4).

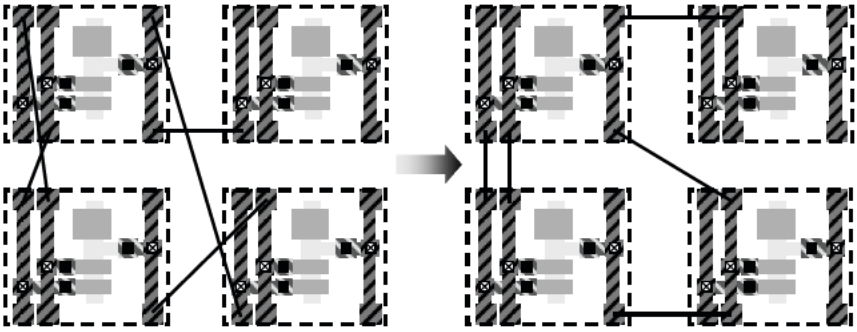
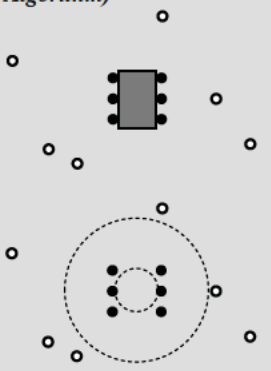


Fig. 3.17 Pin assignment using the example given in Fig. 3.16. The assignment aims to minimize the total connection length by exploiting functionally and electrically equivalent pins.

Pin assignment using concentric circles. The objective of this algorithm is to establish connections between a block and all its related pins in other blocks such that net crossings are minimized. The following simple algorithm [3.6], introduced in 1972, assumes that all *outer pins* (pins outside of the current block) have fixed locations. All *inner pins* (pins in the current block) will be assigned locations based on the locations of the electrically-equivalent outer pins. The algorithm uses two concentric circles – an *inner circle* for the pins of the block under consideration, and an *outer circle* for the pins in the other blocks. The goal is to assign legal pin locations to both circles such that there is no net overlap.

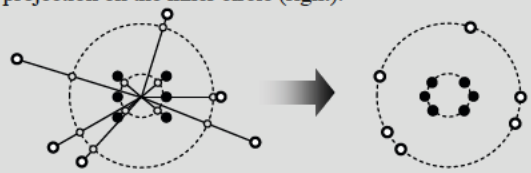
Example: Pin Assignment Using Concentric Circles (Including Algorithm)

Given: (1) set of pins on the block (black) and (2) set of pins on the chip (white) to which the block pins must connect.
Task: perform pin assignment using concentric circles such that each pin on the block is connected to exactly one pin on the chip, and vice-versa (i.e., a 1-to-1 mapping).



Solution:
Determine the circles. The two circles are drawn such that (1) all pins that belong to the block (black) are outside of the inner circle but within the outer circle and (2) all external pins (white) are outside of the outer circle.

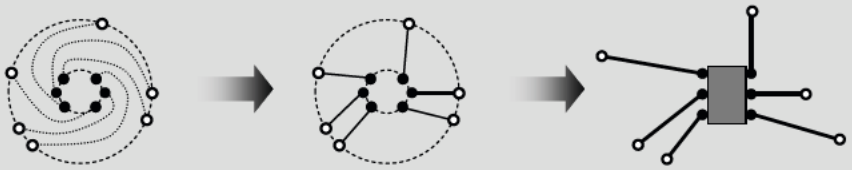
Determine the points. For each point, draw a line from that point to the center of the circles (left). Move each outer (white) point to its projection on the outer circle, and move each inner (black) point to its projection on the inner circle (right).



Determine initial mapping. The initial mapping is an assignment from each outer pin to a corresponding inner pin. Choose a starting point and assign it arbitrarily (left). Then, assign the remaining points in clockwise or counter-clockwise direction (right).



Optimize the mapping. Repeat the mapping process for other outer-inner point-pair combinations. That is, for the same starting point on the outer circle, assign a different point on the inner circle and map the remaining points. Do this until all point-pair combinations have been considered. The best mapping is the one with the shortest Euclidean distance. For the problem instance, a possible mapping is shown on the left, the best mapping is shown in the center, and the final pin assignment is shown on the right.



For each remaining block, go to the first step.

Topological pin assignment. In 1984, *H. N. Brady* improved the concentric-circle pin assignment algorithm by taking into account external block positions and multi-pin nets (connected to more than two pins) [3.2]. Specifically, this enabled pin assignment when external pins are behind other blocks or obstacles.

Fig. 3.18(a) shows an example assigning pins from the main component m to an external block b. A *midpoint line* $l_{m\sim b}$ is drawn from the center of m through the midpoint of b. In Fig. 3.18(b), the pins of b are "unwrapped" and expanded as a line l' at the *dividing point* d – the farther point on $l_{m\sim b}$ that intersections b. The pins are then projected onto the outer circle (from the original concentric- circle algorithm).

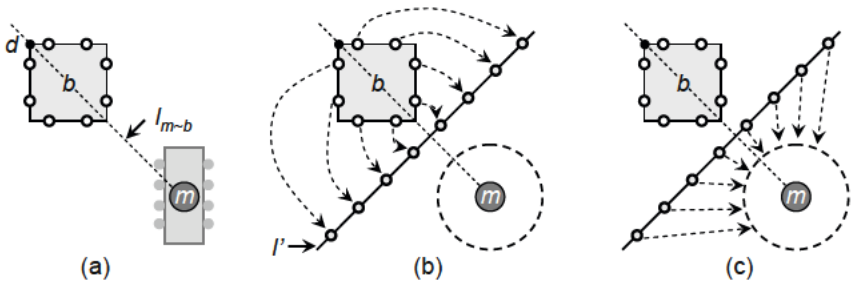


Fig. 3.18 Pin assignment from the main component m to an external block b. (a) The main component m is collapsed as a single point. A midpoint line $l_{m\sim b}$ is drawn from m through the center of b. A dividing point d is formed at the farther point where $l_{m\sim b}$ intersects b. (b) From d, project b's pins onto l'. (c) The pins are then projected onto the outer circle of m [3.2].

This improvement also allows several blocks to be considered simultaneously. Let the set of all external blocks be denoted as B, and let the main component be denoted as m. A midpoint line $l_{m\sim b}$ is drawn from m through the midpoint of every external block $b \in B$.

A dividing point on block b is formed at the *farther* point where $l_{m\sim b}$ intersects b, and at the *closer* point where each midpoint line $l_{m\sim b'}$ intersect b, where $b' \in B$ is located between m and b. Based on these dividing points, the pins are separated and extended accordingly (Fig. 3.19).

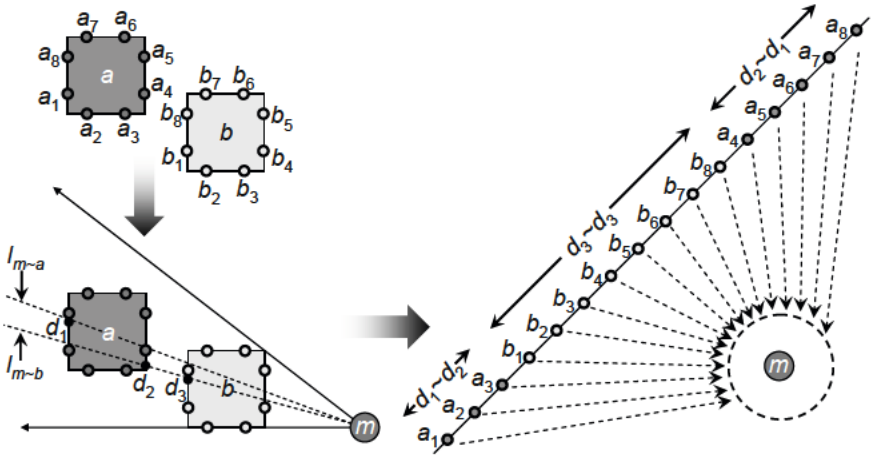


Fig. 3.19 Pin assignment on blocks a and b [3.2]. On block a, consider the midpoint lines $l_{m\sim a}$ and $l_{m\sim b}$. The dividing point d_1 is formed because it is the farther point on $l_{m\sim a}$; the dividing point d_2 is formed because it is the closer point on $l_{m\sim b}$. On block b, consider only $l_{m\sim b}$. The dividing point d_3 is formed because it is the farther point on $l_{m\sim b}$. Using d_1-d_3, the pins are "unwrapped" accordingly when projected onto m's outer circle.

3.7 Power and Ground Routing

On-chip supply voltages scale more slowly than chip frequencies and transistor counts. Therefore, currents supplied to the chip steadily increase with each technology generation. Improved packaging and cooling technologies, together with market demands for functionality, lead to ever-greater power budgets and denser power grids. Today, up to 20-40% of all metal resources on the chip are used to supply *power* (*VDD*) and *ground* (*GND*) nets. Since floorplanning precedes place-and-route, i.e., block- and chip-level implementation, power-ground planning has become an essential part of the modern chip planning process.

Chip planning determines not only the layout of the power-ground distribution network, but also the placement of *supply I/O pads* (with wire-bond packaging) or *bumps* (with flip-chip packaging). The pads or bumps are preferentially located in or near high-activity regions of the chip to minimize the $V = IR$ voltage drop.[2] In general, the power planning process is highly iterative, including (1) early simulation of major power dissipation components, (2) early quantification of chip power, (3) analyses of total chip power and maximum power density, (4) analyses of total chip power fluctuations, (5) analyses of inherent and added fluctuations due to clock gating, and (6) early power distribution analysis: average, maximum and multi-cycle fluctuations.

To construct an appropriate supply network, many aspects of the design and the process technology must be considered. For example, to estimate chip power, the designer must plan for (1) use of low-V_{th} devices and dynamic circuits that consume more power, (2) use of clock gating for low power, and (3) quantity and placement of added decoupling capacitors that mitigate switching noise.

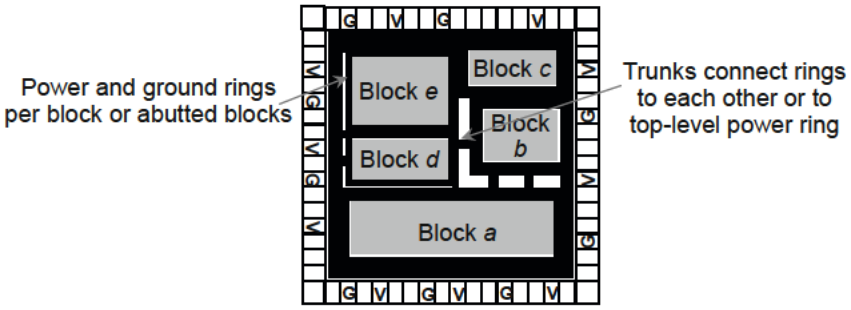


Fig. 3.20 Custom style of power-ground distribution for a chip floorplan.

[2] A supply I/O pad can deliver tens of milliamperes of current, while a supply bump can deliver hundreds of milliamperes, i.e., an order of magnitude more current. As an example of how much die resource goes to power-ground distribution – the Intel Pentium 4 microprocessor chip uses 423 bumps, of which 223 are for delivery of *VDD* and *GND*.

This section discusses the physical design of *power-ground distribution networks*. Fig. 3.20 illustrates conceptually how a floorplan in a *custom* design approach might associate supply rings with each block, for later connection to a chip-level power distribution plan such as those discussed below.

► 3.7.1 Design of a Power-Ground Distribution Network

The *supply nets*, *VDD* and *GND*, connect each cell in the design to a power source. As each cell must have both *VDD* and *GND* connections, the supply nets (1) are large, (2) span across the entire chip, and (3) are routed first before any signal routing. *Core* supply nets are distinguished *I/O* supply nets, which are typically at a higher voltage. In many applications, one core power net and one core ground net are sufficient. Some ICs, such as mixed-signal or low-power (supply-gated or multiple voltage level) designs, can have multiple power and ground nets.

Routing of supply nets is different from routing of signals. Power and ground nets should have dedicated metal layers to avoid consuming signal routing resources. In addition, supply nets prefer thick metal layers – typically, the top two layers in the back-end-of-line process – due to their low resistance. When the power-ground network traverses multiple layers, there must be sufficient *vias* to carry current while avoiding *electromigration* and other *reliability* issues.

Since supply nets have high current loads, they are often much wider than standard signal routes. The widths of the individual wire segments may be tailored to accommodate their respective estimated branch currents. For logic gates to have correct timing performance, the net segment width must be chosen to keep the voltage drop, $V = IR$, within a specified tolerance, e.g., 5% of *VDD*. Wider segments have lower resistance, and hence lower voltage drop.[3]

There are two approaches to the physical design of power-ground distribution – the *planar* approach, which is used primarily in analog or custom blocks, and the *mesh* approach, which is predominant in digital ICs.

► 3.7.2 Planar Routing

Power supply nets can be laid out using *planar routing* when (1) only two supply nets are present in the design, and (2) a cell needs a connection to both supply nets. Planar routing separates the two supply regions by a *Hamiltonian path* that connects all the cells, such that each supply net can be attached either to the *left* or *right* of every cell. The Hamiltonian path allows both supply nets to be routed across the layout – one to the left and one to the right of the path– with no *conflicts* (Fig. 3.21).

[3] Some design manuals will refer to an *IR* drop limit of 10% of *VDD*. This means that the supply can *drop* (*droop*) by 5% of *VDD* and the ground can *bounce* by 5% as well, resulting in a worst-case of 10% supply reduction.

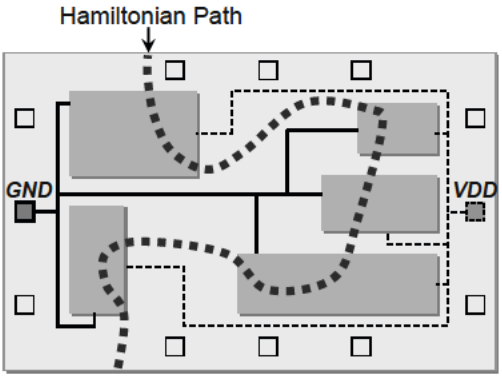


Fig. 3.21 Given a Hamiltonian path that connects all cells, each supply net has its own "uninterrupted" path to each cell, and hence both supply nets can be routed on one layer.

Routing the power and ground nets in this planar fashion can be accomplished with the following three steps.

Step 1: Planarize the topology of the nets
As both power and ground nets must be routed on one layer, the design should be split using the Hamiltonian path. In Fig. 3.22(a), the two nets start from the left and right sides. Both nets grow in a tree-like fashion, without any conflict (overlap), and separated by the Hamiltonian path. The exact routes will depend on the pin locations. Finally, cells are connected wherever a pin is encountered.

Step 2: Layer assignment
Net segments are assigned to appropriate routing layers based on routability, the resistance and capacitance properties of each available layer, and design rule information.

Step 3: Determining the widths of the net segments
The width of each segment (branch) depends on the maximum current flow. That is, a segment's width is determined from the sum of the currents from all the cells to which it connects in accordance with *Kirchhoff*'s *Current Law* (*KCL*). When dealing with large currents, designers often extend the "width" of the "planar" route in the vertical dimension with superposed segments on multiple layers that are stapled together with vias. In addition, the width determination is typically an iterative process, since currents depend on timing and noise, which depend on voltage drops, which, in turn, depend on currents. In other words, there is a cyclic dependency within the power, timing and noise analyses. This loop is typically addressed through multiple iterations and by experienced designers. After executing the above three steps, the segments of the power-ground route form obstacles during general signal routing (Fig. 3.22(b)).

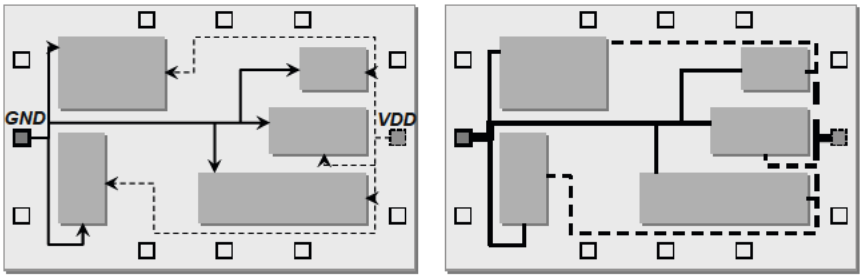


Fig. 3.22 (a) Generating the topology of the two supply nets. (b) Adjusting the widths of the individual segments in (a) with respect to their maximum current loads.

▶ 3.7.3 Mesh Routing

Power-ground routing in modern digital ICs typically has a *mesh* topology that is created through the following five steps.

Step 1: creating a ring
Typically, a ring is constructed to surround the entire core area of the chip, and possibly individual blocks. The purpose of the ring is to connect the supply I/O cells and, possibly, electrostatic discharge protection structures with the global power mesh of the chip or block. For low resistance, these connections and the ring itself are on many layers. For example, a ring might use metal layers Metal2- Metal8 (every layer except Metal1).

Step 2: connecting I/O pads to the ring
The top left of Fig. 3.23 shows connectors from the I/O pads to the ring. Each I/O pad will have a number of "fingers" emanating on each of several metal layers. These should be maximally connected to the power ring in order to minimize resistance and maximize the ability to carry current to the core.

Step 3: creating a mesh
A power mesh consists of a set of stripes at defined pitches on two or more layers (Fig. 3.23). The width and pitch of stripes are determined from estimated power consumption as well as layout design rules. The stripes are laid out in pairs, alternating as *VDD-GND*, *VDD-GND*, and so on. The power mesh uses the uppermost and thickest layers, and is sparser on any lower layers to avoid signal routing congestion. Stripes on adjacent layers are typically connected with as many vias as possible, again to minimize resistance.

Step 4: creating Metal1 rails
The Metal1 layer is where the power-ground distribution network meets the logic gates of the design. The width (current supply capability) and pitch of the Metal1

rails are typically determined by the standard-cell library. Standard-cell rows are laid out "back-to-back" so that each supply net is shared by two adjacent cell rows.

Step 5: connecting the Metal1 rails to the mesh

Finally, the Metal1 rails are connected to the mesh with stacked vias. A key consideration is the proper size of (number of vias in) the via stack. For example, the most resistive part of the power distribution should be the Metal1 segments between via stacks, rather than the stack itself. In addition, the via stack is optimized to maintain routability of the design. For instance, a 1 × 4 array of vias may be preferable to a 2 × 2 array, depending on the direction in which routing is congested.

Figure 3.23 illustrates the mesh approach to power-ground distribution. In the figure, layers Metal8 through Metal4 are used for the mesh. In practice, many chips will use fewer layers (e.g., Metal8 and Metal7 only) due to routing resource constraints.

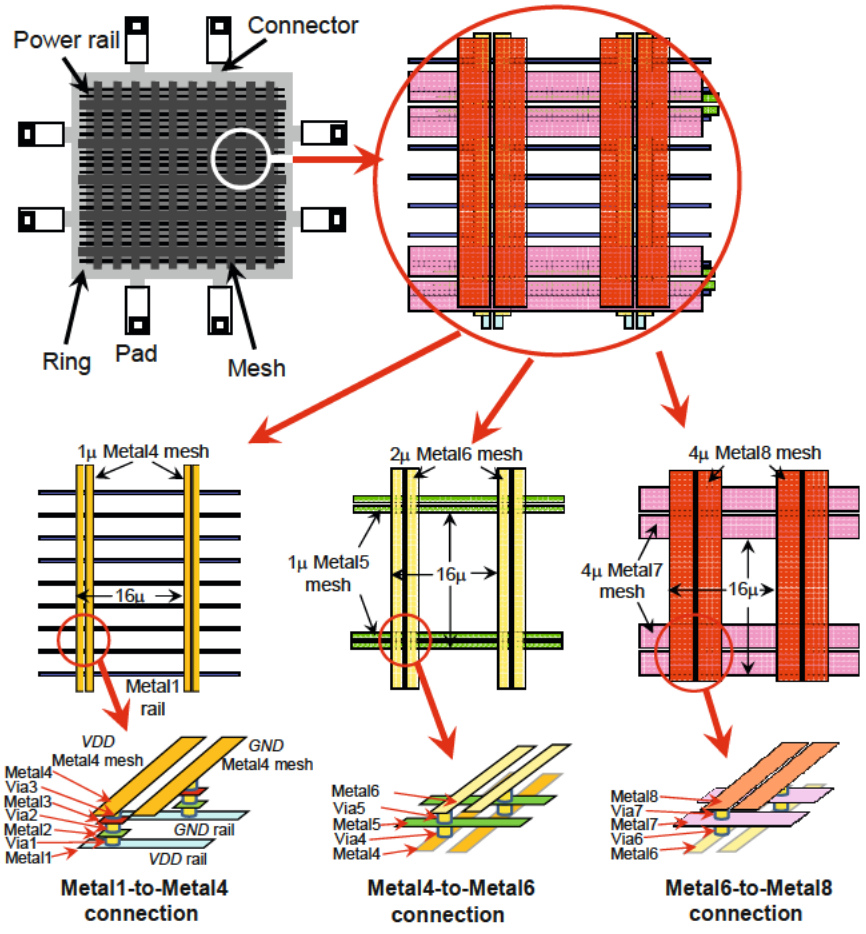
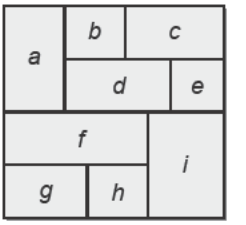


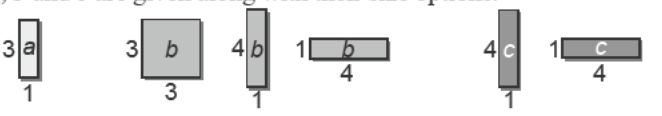
Fig. 3.23 Construction of a mesh power-ground distribution network.

Chapter 3 Exercises

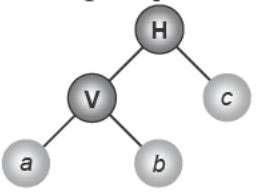
Exercise 1: Slicing Trees and Constraint Graphs
For the given floorplan (right), generate its slicing tree, vertical constraint graph and horizontal constraint graph.



Exercise 2: Floorplan-Sizing Algorithm
Three blocks *a*, *b* and *c* are given along with their size options.

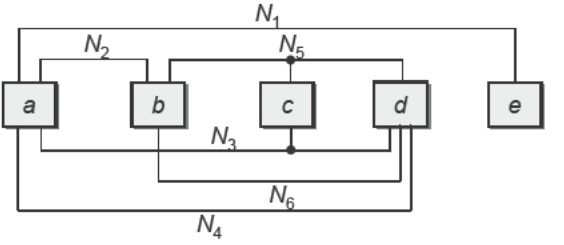


(a) Determine the shape functions for each block *a*, *b*, *c*.
(b) Find the minimum area of the top-level floorplan using the given tree structure and determine the shape function of the top-level floorplan. In the shape function, find the corner point that yields the minimal area. Finally, determine the dimensions of each block and draw the resulting floorplan.



Exercise 3: Linear-Ordering Algorithm
For the netlist with five blocks *a*-*e* and six nets N_1-N_6, determine the linear ordering that minimizes total wirelength. Let the starting block be block *a*. Place it in the first (leftmost) position. Draw the resulting placement.

$N_1 = (a,e)$ $N_2 = (a,b)$
$N_3 = (a,c,d)$ $N_4 = (a,d)$
$N_5 = (b,c,d)$ $N_6 = (b,d)$



Exercise 4: Non-Slicing Floorplans
Recall that the smallest non-slicing floorplans with *no* wasted space exhibit the structure of a clockwise or counter-clockwise *wheel* with five blocks. Draw a non-slicing floorplan with only four blocks *a*-*d*.

Chapter 3 References

[3.1] C. J. Alpert, D. P. Mehta and S. S. Sapatnekar, eds., *Handbook of Algorithms for Physical Design Automation*, CRC Press, 2009.

[3.2] H. N. Brady, "An Approach to Topological Pin Assignment", *IEEE Trans. on CAD* 3(3) (1984), pp. 250-255.

[3.3] T.-C. Chen and Y.-W. Chang, "Modern Floorplanning Based on B*-Tree and Fast Simulated Annealing", *IEEE Trans. on CAD* 25(4) (2006), pp. 637-650.

[3.4] S. Kang, "Linear Ordering and Application to Placement", *Proc. Design Autom. Conf.*, 1983, pp. 457-464.

[3.5] S. Kirkpatrick, C. D. Gelatt and M. P. Vecchi, "Optimization by Simulated Annealing", *Science* 220(4598) (1983), pp. 671-680.

[3.6] N. L. Koren, "Pin Assignment in Automated Printed Circuit Board Design", *Proc. Design Autom. Workshop*, 1972, pp. 72-79.

[3.7] M. D. Moffitt, J. A. Roy, I. L. Markov and M. E. Pollack, "Constraint-Driven Floorplan Repair", *ACM Trans. on Design Autom. of Electronic Sys.* 13(4) (2008), pp. 1-13.

[3.8] R. H. J. M. Otten, "Efficient Floorplan Optimization", *Proc. Intl. Conf. on Computer Design*, 1983, pp. 499-502.

[3.9] R. H. J. M. Otten and L. P. P. P. van Ginneken, "Floorplan Design Using Simulated Annealing", *Proc. Intl. Conf. on CAD*, 1984, pp. 96-98.

[3.10] C. Sechen, "Chip Planning, Placement and Global Routing of Macro/Custom Cell Integrated Circuits Using Simulated Annealing", *Proc. Design Autom. Conf.*, 1988, pp. 73-80.

[3.11] L. Stockmeyer, "Optimal Orientation of Cells in Slicing Floorplan Designs", *Information and Control* 57 (1983), pp. 91-101.

[3.12] S. Sutanthavibul, E. Shragowitz and J. B. Rosen, "An Analytical Approach to Floorplan Design and Optimization", *IEEE Trans. on CAD* 10 (1991), pp. 761-769.

[3.13] X. Tang, R. Tian and D. F. Wong, "Fast Evaluation of Sequence Pair in Block Placement by Longest Common Subsequence Computation", *Proc. Design, Autom. and Test in Europe*, 2000, pp. 106-111.

[3.14] D. F. Wong and C. L. Liu, "A New Algorithm for Floorplan Design", *Proc. Design Autom. Conf.*, 1986, pp. 101-107.

[3.15] J. Xiong, Y.-C. Wong, E. Sarto and L. He, "Constraint Driven I/O Planning and Placement for Chip-Package Co-Design", *Proc. Asia and South Pacific Design Autom. Conf.*, 2006, pp. 207-212.

[3.16] J. Z. Yan and C. Chu, "DeFer: Deferred Decision Making Enabled Fixed-Outline Floorplanner", *Proc. Design Autom. Conf.*, 2008, pp. 161-166.

[3.17] T. Yan and H. Murata, "Fast Wire Length Estimation by Net Bundling for Block Placement", *Proc. Intl. Conf. on CAD*, 2006, pp. 172-178.

Chapter 4

Global and Detailed Placement

4

4

4 Global and Detailed Placement 95
4.1 Introduction 95
4.2 Optimization Objectives 96
4.3 Global Placement 103
4.3.1 Min-Cut Placement 104
4.3.2 Analytic Placement 110
4.3.3 Simulated Annealing 117
4.3.4 Modern Placement Algorithms 120
4.4 Legalization and Detailed Placement 122
Chapter 4 Exercises 125
Chapter 4 References 126

4 Global and Detailed Placement

After partitioning the circuit into smaller modules and floorplanning the layout to determine block outlines and pin locations, *placement* seeks to determine the locations of standard cells or logic elements within each block while addressing optimization objectives, e.g., minimizing the total length of connections between elements. Specifically, *global placement* (Sec. 4.3) assigns general locations to movable objects, while *detailed placement* (Sec. 4.4) refines object locations to legal cell sites and enforces nonoverlapping constraints. The detailed locations enable more accurate estimates of circuit delay for the purpose of timing optimization.

4.1 Introduction

The objective of placement is to determine the locations and orientations of all circuit elements within a (planar) layout, given solution constraints (e.g., no overlapping cells) and optimization goals (e.g., minimizing total wirelength).

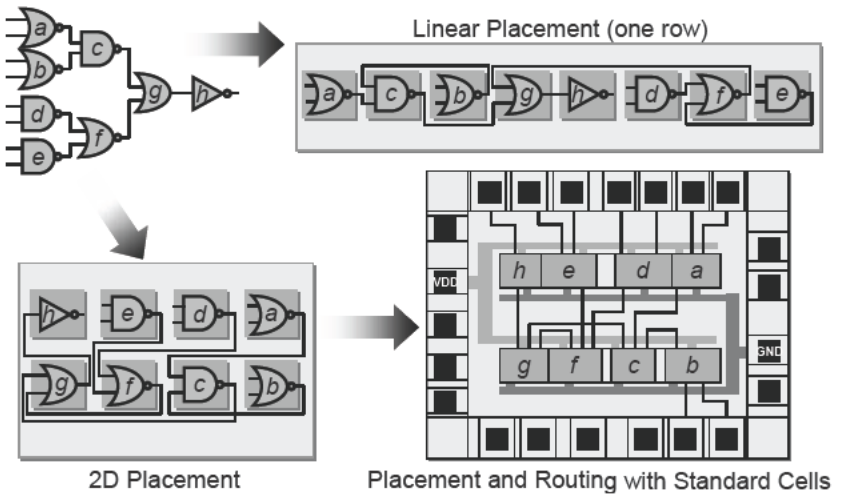


Fig. 4.1 A simple circuit (top left) along with example linear placement (top right), 2D placement (bottom left), and placement and routing with standard cells (bottom right).

Circuit elements (e.g., gates, standard cells and macro blocks) have rectangular shapes and are represented by nodes, while nets are represented by edges (Fig. 4.1). Some circuit elements may have fixed locations while others are movable (placeable). The placement of movable objects will determine the achievable quality of the subsequent routing stages. However, detailed routing information, such as

A. B. Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure,*
DOI 10.1007/978-90-481-9591-6_4, © Springer Science+Business Media B.V. 2011

track assignment, is not available at the placement stage, and hence the placer estimates the eventual *Manhattan* routing, where only horizontal and vertical wires are allowed (Sec. 4.2). In Fig. 4.1, assuming unit distance between horizontally- or vertically-adjacent placement sites, both the linear and 2D placements have 10 units of total wirelength.

Placement techniques for large circuits encompass *global placement*, *detailed placement* and legalization. Global placement often neglects specific shapes and sizes of placeable objects and does not attempt to align their locations with valid grid rows and columns. Some overlaps are allowed between placed objects, as the emphasis is on judicious global positioning and overall density distribution. Legalization is performed before or during detailed placement. It seeks to align placeable objects with rows and columns, and remove overlap, while trying to minimize displacements from global placement locations as well as impacts on interconnect length and circuit delay. Detailed placement incrementally improves the location of each standard cell by local operations (e.g., swapping two objects) or shifting several objects in a row to create room for another object. Global and detailed placement typically have comparable runtimes, but global placement often requires much more memory and is more difficult to parallelize.

Performance-driven optimizations can be applied during both global placement and detailed placement. However, timing estimation (Chap. 8) can be inaccurate during early stages of global placement. Thus, it is more common to initiate performance optimizations during later stages of, or after, global placement. Legalization is often performed so as to minimize impact on performance, and detailed placement can directly improve performance because of its fine control over individual locations.

4.2 4.2 Optimization Objectives

Placement must produce a layout wherein all nets of the design can be routed simultaneously, i.e., the placement must be *routable*. In addition, electrical effects such as signal delay or crosstalk must be taken into consideration. As detailed routing information is not available during placement, the placer optimizes estimates of routing quality metrics, such as total weighted wirelength, cut size, wire congestion (density), or maximum signal delay (Fig. 4.2).

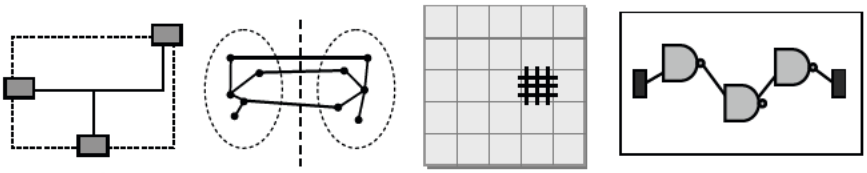


Fig. 4.2 Examples of routing quality metrics optimized during placement.

Subject to maintaining routability, the primary goal of placement is to optimize delays along signal paths. Since the delay of a net directly correlates with the net's length, placers often minimize *total wirelength*.[1] As illustrated in Fig. 4.3, the placement of a design strongly affects the net lengths as well as the wire density. Further placement optimizations, beyond the scope of this chapter, include

- Placement and assignment of I/O pads with respect to both the logic gates connected to them and the package connection points, e.g., bump locations.
- Temperature- and reliability-driven optimizations, such as the placement of actively switching (and heat-generating) circuit elements, to achieve uniform temperature across the chip.

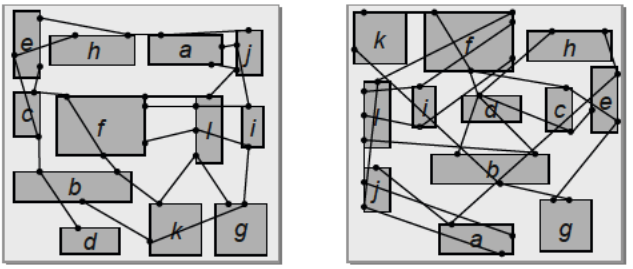


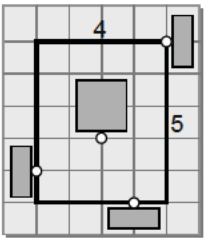
Fig. 4.3 Two placements of the same design with better (left) and worse (right) total wirelength.

Wirelength estimation for a given placement. An important consideration for wirelength estimation during placement is the speed at which a given wirelength estimate can be computed. Moreover, the estimation must be applicable to both *two-pin* and *multi-pin nets*. For two-pin nets, most placement tools use the Manhattan distance d_M between two points (pins) P_1 (x_1,y_1) and P_2 (x_2,y_2)

$$d_M(P_1,P_2) = |x_2 - x_1| + |y_2 - y_1|$$

For multi-pin nets, the following estimation techniques are used [4.23].

The *half-perimeter wirelength* (*HPWL*) model is commonly used because it is reasonably accurate and efficiently calculated. The *bounding box* of a net with p pins is the smallest rectangle that encloses the pin locations. The wirelength is estimated as half the perimeter of the bounding box. For two- and three-pin nets (70-80% of all nets in most modern designs), this is exactly the same as the rectilinear Steiner minimum tree (RSMT) cost (discussed later in this section). When $p \geq 4$, HPWL underestimates the RSMT cost by an average factor that grows asymptotically as $\sqrt{p}$.



[1] Timing-driven placement techniques are discussed in Sec. 8.3.

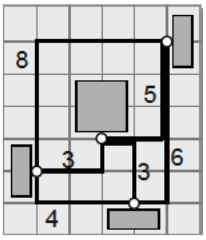
The *complete graph* (*clique*) model of a p-pin net *net* has

$$\binom{p}{2} = \frac{p!}{2!(p-2)!} = \frac{p(p-1)}{2}$$

edges, i.e., each pin is directly connected to every other pin. Since a spanning tree over the net's pins will have $p - 1$ edges, a correction factor of $2 / p$ is applied. The total edge length of *net* according to the clique model is

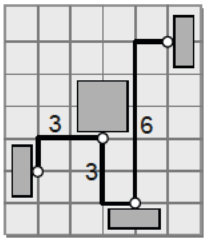
$$L(net) = \frac{2}{p} \sum_{e \in clique} d_M(e)$$

where e is an edge in the clique, and $d_M(e)$ is the Manhattan distance between the endpoints of e.



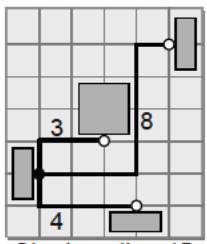
Clique Length = 14.5

The *monotone chain* model connects the pins of a net using a chain topology. Both end pins have degree one; each intermediate pin has degree two. Finding a minimum-length path connecting the pin locations corresponds to the NP-hard Hamiltonian path problem. Thus, the monotone chain model sorts pins by either x- or y-coordinate and connects them accordingly. Though simple, this method often overestimates the actual wirelength. Another disadvantage is that the chain topology changes with the placement.



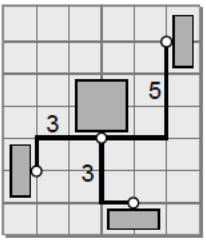
Chain Length = 12

The *star* model considers one pin as the *source* node and all other pins as *sink* nodes; there is an edge from the source to each sink. This is especially useful for timing optimization, since it captures the direction of signal flow from an output pin to one or more input pins. The star model uses only $p - 1$ edges; this sparsity can be advantageous in modeling high pin-count nets. On the other hand, the star model overestimates wirelength.



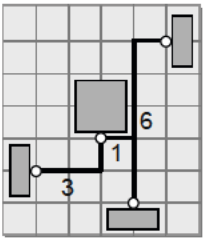
Star Length = 15

The *rectilinear minimum spanning tree* (*RMST*) model decomposes the p-pin net into two-pin connections and connects the p pins with $p - 1$ connections. Several algorithms (e.g., *Kruskal's Algorithm* [4.19]) exist for constructing minimum spanning trees. RMST algorithms can exploit the Manhattan geometry to achieve $O(p \log p)$ runtime complexity (Sec. 5.6.1).



RMST Length = 11

The *rectilinear Steiner minimum tree* (*RSMT*) model connects all p pins of the net and as many as $p - 2$ additional *Steiner* (branching) *points* (Sec. 5.6.1). Finding an optimal set of Steiner points for an arbitrary point set is NP-hard. For nets with a bounded number of pins, computing an RSMT takes constant time. If the Steiner points are known, an RSMT can be found by constructing an RMST over the union of the original point set and the set of added Steiner points.

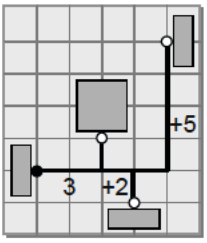


RSMT Length = 10

The *rectilinear Steiner arborescence* (*RSA*) model of a p-pin net is also a tree where a single source node s_0 is connected to $p - 1$ sink nodes. In an RSA, the path length from s_0 to any sink s_i, $1 \leq i < p - 1$, must be equal to the $s_0 \sim s_i$ Manhattan (rectilinear) distance. That is, for all sinks s_i

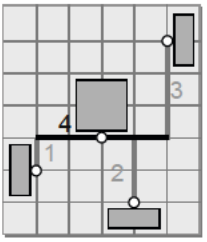
$$L(s_0, s_i) = d_M(s_0, s_i)$$

where $L(s_0, s_i)$ is the path length from s_0 to s_i in the tree. Computing a minimum-length RSA is NP-hard.



RSA Length = 10

The *single-trunk Steiner tree* (*STST*) model consists of one vertical (horizontal) segment, i.e., *trunk*, and connects all pins to this trunk using horizontal (vertical) segments, i.e., *branches*. STSTs are commonly used for estimation due to their ease of construction. RSAs are more timing-relevant but somewhat more complex to construct than STSTs. For practical purposes, both RSAs and STSTs are constructed in $O(p \log p)$-time, where p is the number of pins.



STST Length = 10

Total wirelength with net weights (weighted wirelength). *Net weights* can be used to prioritize certain nets over others. For example, a net *net* with weight $w(net) = 2$ is equivalent to two nets i and j with weights $w(i) = w(j) = 1$. For a placement P, an estimate of total weighted wirelength is

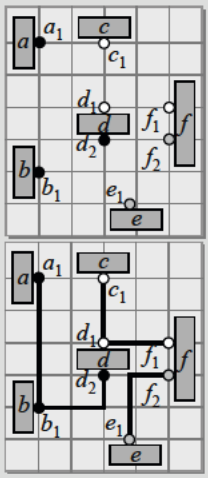
$$L(P) = \sum_{net \in P} w(net) \cdot L(net)$$

where $w(net)$ is the weight of *net*, and $L(net)$ is the estimated wirelength of *net*.

Example: Total Weighted Wirelength of a Placement
Given: (1) placement P of blocks *a-f* and their pins (right) and (2) nets N_1-N_3 and their net weights.

$$N_1 = (a_1, b_1, d_2) \qquad w(N_1) = 2$$
$$N_2 = (c_1, d_1, f_1) \qquad w(N_2) = 4$$
$$N_3 = (e_1, f_2) \qquad w(N_3) = 1$$

Task: estimate the total weighted wirelength of P using the RMST model.



Solution:

$L(N_1) = d_M(a_1,b_1) + d_M(b_1,d_2) = 4 + 3 = 7.$
$L(N_2) = d_M(c_1,d_1) + d_M(d_1,f_1) = 2 + 2 = 4.$
$L(N_3) = d_M(e_1,f_2) = 3.$

$$L(P) = w(N_1) \cdot L(N_1) + w(N_2) \cdot L(N_2) + w(N_3) \cdot L(N_3)$$
$$= 2 \cdot 7 + 4 \cdot 4 + 1 \cdot 3 = 14 + 16 + 3 = 33.$$

Maximum cut size. In Fig. 4.4, a (global) vertical cutline divides the region into a left region L and a right region R. With respect to the cutline, a net can be classified as either *uncut* or *cut*. An uncut net has pins in L or R, but not both, i.e., it is completely left or right of the cutline; a cut net has at least one pin in each L and R.

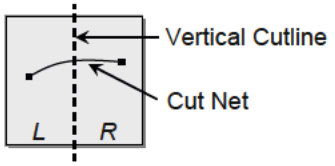


Fig. 4.4 Layout with a vertical cutline and a cut net.

Given a placement P, let

- V_P and H_P be the set of global vertical and horizontal cutlines of P, respectively
- $\Psi_P(cut)$ be the set of nets cut by a cutline *cut*
- $\psi_P(cut)$ be the size of $\Psi_P(cut)$, i.e., $\psi_P(cut) = |\Psi_P(cut)|$

Then, define $X(P)$ to be the maximum of $\psi_P(v)$ over all vertical cutlines $v \in V_P$.

$$X(P) = \max_{v \in V_P} \left(\psi_P(v) \right)$$

Similarly, define $Y(P)$ as the maximum of $\psi_P(h)$ over all horizontal cutlines $h \in H_P$.

$$Y(P) = \max_{h \in H_P} \left(\psi_P(h) \right)$$

$X(P)$ and $Y(P)$ are lower bounds on routing capacity needed in the horizontal (x-) and vertical (y-) directions, respectively. For example, if $X(P) = 10$, then some global vertical cutline x crosses 10 horizontal net segments. Likewise, if $Y(P) = 15$,

some global horizontal cutline y crosses 15 vertical net segments. A necessary but not sufficient condition for routability is that there exist at least 10 (15) horizontal (vertical) routing tracks at x (y). Thus, $X(P)$ and $Y(P)$ can be used to assess routability of P.

For some circuit layout styles, e.g., gate arrays, the *capacity* (maximum number) of horizontal and vertical tracks is pre-set. An optimization constraint in placement, necessary but not sufficient for routability, is to ensure that $X(P)$ and $Y(P)$ are within these capacities. In the context of standard-cell design, $X(P)$ gives a lower bound on the demand for horizontal routing tracks, and $Y(P)$ gives a lower bound on the demand for vertical routing tracks.

To improve routability of P, $X(P)$ and $Y(P)$ must be minimized. To improve total wirelength of P, separately calculate the number of crossings of global vertical and horizontal cutlines, and minimize

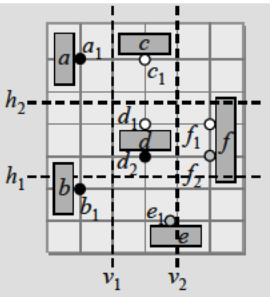
$$L(P) = \sum_{v \in V_P} \psi_P(v) + \sum_{h \in H_P} \psi_P(h)$$

Example: Cut Sizes of a Placement

Given: (1) placement P of blocks *a-f* and their pins (right), (2) nets N_1-N_3, (3) global vertical cutlines v_1 and v_2, and (4) global horizontal cutlines h_1 and h_2.

$N_1 = (a_1, b_1, d_2)$ $N_2 = (c_1, d_1, f_1)$ $N_3 = (e_1, f_2)$

Task: determine the cut sizes $X(P)$ and $Y(P)$ of placement P according to the RMST model.



Solution:

Find the cut values for each global cutline.

$\psi_P(v_1) = 1$ $\psi_P(v_2) = 2$

$\psi_P(h_1) = 3$ $\psi_P(h_2) = 2$

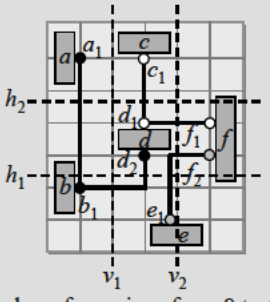
Find the total number of crossings in P.

$\psi_P(v_1) + \psi_P(v_2) + \psi_P(h_1) + \psi_P(h_2) = 1 + 2 + 3 + 2 = 8$

Find the cut sizes.

$X(P) = \max(\psi_P(v_1), \psi_P(v_2)) = \max(1,2) = 2$

$Y(P) = \max(\psi_P(h_1), \psi_P(h_2)) = \max(3,2) = 3$



Observe that moving block b from (0,0) to (0,1) reduces the number of crossings from 8 to 6. This also decreases $\psi_P(h_1)$ from 3 to 1, which makes $Y(P) = 2$, thereby reducing the local congestion.

Routing congestion. The routing congestion of a placement P can be thought of in terms of *density*, namely, the ratio of *demand* for routing tracks to the *supply* of available routing tracks. For example, a *routing channel* has available horizontal routing tracks, while a *switchbox* has available vertical and horizontal routing tracks

(Fig. 4.5). For gate-array designs, this concept of congestion is particularly important, as the supply of routing tracks is fixed (Sec. 5.3).

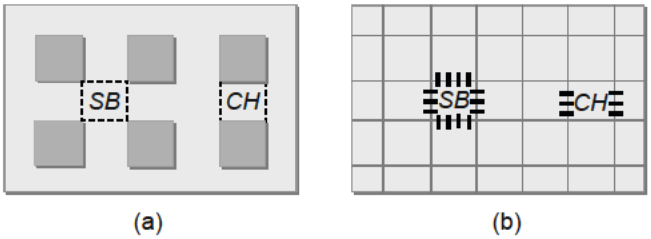


Fig. 4.5 (a) A switchbox *SB* and a channel *CH*. (b) *SB* and *CH* represented by grid cells with wire capacities on the grid cell edges.

For a given placement P, congestion can also be estimated by the number of nets that pass through the boundaries of individual routing regions. Formally, the *local wire density* $\varphi_P(e)$ of an edge e between two neighboring grid cells[2] is

$$\varphi_P(e) = \frac{\eta_P(e)}{\sigma_P(e)}$$

where $\eta_P(e)$ is the estimated number of nets that cross e and $\sigma_P(e)$ is the maximum number of nets that can cross e. If $\varphi_P(e) > 1$, then too many nets are estimated to cross e, making P more likely to be unroutable. The *wire density* of P is

$$\Phi(P) = \max_{e \in E}\left(\varphi_P(e)\right)$$

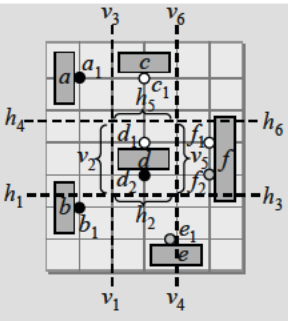
where E is the set of all edges. If $\Phi(P) \leq 1$, then the design is estimated to be fully routable. If $\Phi(P) > 1$, then routing will need to detour some nets through less-congested edges – but in some cases, this may be impossible. Therefore, congestion-driven placement often seeks to minimize $\Phi(P)$.

Example: Wire Density of a Placement

Given: (1) placement P of blocks *a*-*f* and their pins (right), (2) nets N_1-N_3, (3) local vertical cutlines v_1-v_6, (4) local horizontal cutlines h_1-h_6, and (5) $\sigma_P(e) = 3$ for all local cutlines $e \in E$.

$N_1 = (a_1, b_1, d_2)$ $N_2 = (c_1, d_1, f_1)$ $N_3 = (e_1, f_2)$

Task: find the wire density $\Phi(P)$ and determine the routability of P based on the RMST model.

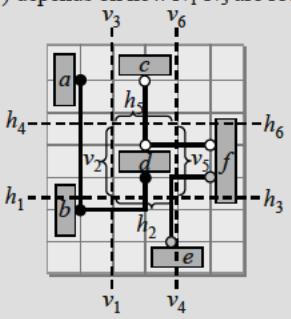


[2] Such an edge represents the border between two regions (e.g., switchboxes or channels) or between two cutlines.

Solution:
The following is one of many possible solutions, as $\Phi(P)$ depends on how N_1-N_3 are routed.

Horizontal edges:
$\eta_P(h_1) = 1$
$\eta_P(h_2) = 2$
$\eta_P(h_3) = 0$
$\eta_P(h_4) = 1$
$\eta_P(h_5) = 1$
$\eta_P(h_6) = 0$

Vertical Edges:
$\eta_P(v_1) = 1$
$\eta_P(v_2) = 0$
$\eta_P(v_3) = 0$
$\eta_P(v_4) = 0$
$\eta_P(v_5) = 2$
$\eta_P(v_6) = 0$



Maximum $\eta_P(e) = 2$.

$\Phi(P) = \eta_P(e) / \sigma_P(e) = 2/3$. Since $\Phi(P) \leq 1$, P is estimated to be routable.

Signal delays. The total wirelength of the placement affects the maximum clock frequency for a given design, as it depends on net (wire) delays and gate delays. In earlier process technologies, gate delays accounted for the majority of circuit delay. With modern processes, however, due to technology scaling, wire delays contribute a significant portion of overall path delay.

Circuit timing is usually verified using *static timing analysis* (*STA*) (Sec. 8.2.1), based on estimated net and gate delays. Common terminology includes *actual arrival time* (*AAT*) and *required arrival time* (*RAT*), which can be estimated for every node v in the circuit. $AAT(v)$ represents the latest transition time at a given node v measured from the beginning of the clock cycle. $RAT(v)$ represents the time by which the latest transition at v must complete in order for the circuit to operate correctly within a given clock cycle. For correct operation of the chip with respect to setup (maximum path delay) constraints, it is required that $AAT(v) \leq RAT(v)$.

4.3 Global Placement

4.3

Techniques for circuit placement are summarized as follows (Fig. 4.6). In *partitioning-based algorithms*, the netlist and the layout are divided into smaller sub-netlists and sub-regions, respectively, according to cut-based cost functions. This process is repeated until each sub-netlist and sub-region is small enough to be handled optimally. An example of this approach is *min-cut placement* (Sec. 4.3.1).

Analytic techniques model the placement problem using an objective (cost) function, which can be maximized or minimized via mathematical analysis. The objective can be quadratic or otherwise non-convex. Examples of analytic techniques include *quadratic placement* and *force-directed placement* (Sec. 4.3.2).

In *stochastic algorithms*, randomized moves are used to optimize the cost function. An example of this approach is *simulated annealing* (Sec. 4.3.3).

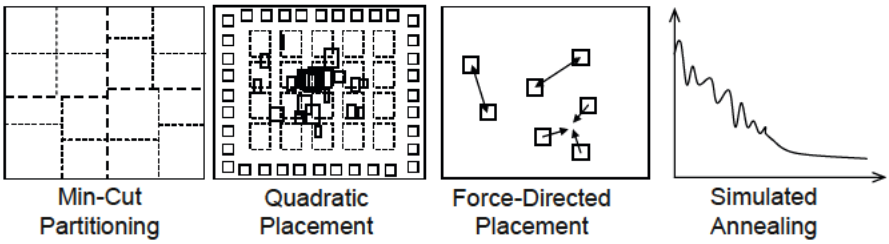


Fig. 4.6 Common techniques for global placement.

▶ 4.3.1 Min-Cut Placement

In the late 1970s, *M. A. Breuer* [4.1] studied *min-cut placement*, which uses partitioning algorithms to divide (1) the netlist and (2) the layout region into smaller, sub-netlists and sub-regions, respectively. The sub-netlists and sub-regions are repeatedly divided into even smaller partitions until each sub-region contains only a small number of cells. Conceptually, each sub-region is assigned a portion of the original netlist. However, when implementing min-cut placement, the netlist should be divided such that each sub-region has access to its own unique (induced) sub-netlist.

Each cut heuristically minimizes the number of cut nets (Sec. 4.2). Standard algorithms used to minimize the number of cut nets are the Kernighan-Lin (KL) algorithm (Sec. 2.4.1) and the Fiduccia-Mattheyses (FM) algorithm (Sec. 2.4.3).

Min-Cut Algorithm
Input: netlist *Netlist*, layout area *LA*, minimum number of cells per region *cells_min*
Output: placement *P*

```
1.  P = Ø
2.  regions = ASSIGN(Netlist,LA)          // assign netlist to layout area
3.  while (regions != Ø)                  // while regions still not placed
4.    region = FIRST_ELEMENT(regions)     // first element in regions
5.    REMOVE(regions, region)             // remove first element of regions
6.    if (region contains more than cell_min cells)
7.      (sr1,sr2) = BISECT(region)        // divide region into two subregions
                                          //   sr1 and sr2, obtaining the sub-
                                          //   netlists and sub-areas
8.      ADD_TO_END(regions,sr1)           // add sr1 to the end of regions
9.      ADD_TO_END(regions,sr2)           // add sr2 to the end of regions
10.   else
11.     PLACE(region)                     // place region
12.     ADD(P,region)                     // add region to P
```

Min-cut optimization is performed *iteratively*, one cutline at a time. That is, heuristic minimum cuts are found for the current sub-netlist, based on the current sub-regions. Ideally, the algorithm should directly optimize the placement figures of merit $X(P)$,

$Y(P)$ and $L(P)$ (Sec. 4.2). However, this is computationally infeasible, especially for large designs. Moreover, even if every cut were optimal, the final solution would not necessarily be optimal. Therefore, the algorithm iteratively minimizes cut size, where the cut size is minimized on the first (horizontal) cut cut_1, then minimized on the second (vertical) cut cut_2, and so on.

$$\text{minimize}(\psi_P(cut_1)) \rightarrow \text{minimize}(\psi_P(cut_2)) \rightarrow \ldots \rightarrow \text{minimize}(\psi_P(cut_{|Cuts|}))$$

Let *Cuts* be the set of all cutlines made in the layout region. The sequence cut_1, cut_2, … , $cut_{|Cuts|}$ denotes the order in which the cuts are made. Possible approaches to dividing the layout include using alternating and repeating cutline directions.

With *alternating cutline directions*, the algorithm divides the layout by switching between sets of vertical and horizontal cutlines. In Fig. 4.7(a), the horizontal cutline cut_1 is made first to bisect the region. Then, two vertical cutlines cut_{2a} and cut_{2b} are made, one (cut_{2a}) in the top half and the other (cut_{2b}) in the bottom half. Each newly formed region is divided in half again with four horizontal cutlines cut_{3a}-cut_{3d}, followed by eight vertical cutlines cut_{4a}-cut_{4h}. This approach is suitable for standard-cell designs with high wire density in the center of the layout region.

With *repeating cutline directions*, the layout is divided using only vertical (horizontal) cutlines until each column (row) is the width (height) of a standard cell. Then, the layout is divided using the orthogonal set of cutlines, e.g., if horizontal cutlines are first used to generate rows, then vertical lines are used to divide each row into columns. In Fig. 4.7(b), the horizontal cut cut_1 is made first to divide the region in half. Then, cut_{2a} and cut_{2b} divided the region into four rows. Next, four vertical cuts cut_{3a}- cut_{3d} are made, followed by eight vertical cuts cut_{4a}- cut_{4h}. This approach often results in greater wirelength because the aspect ratios of the sub-regions can be very far from one. For example, when bisecting the sub-netlist with cut_{3a}, very little information is available regarding the x-locations of adjacent standard cells in the other sub-regions.

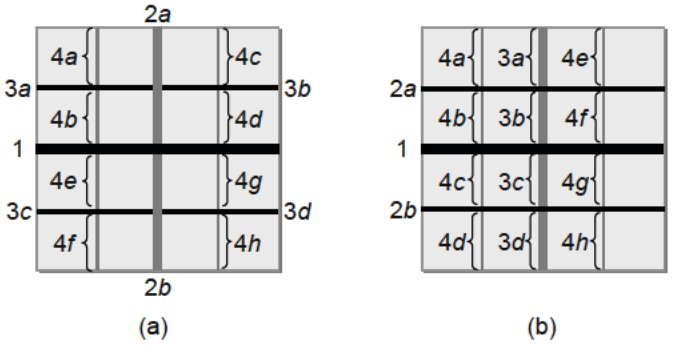
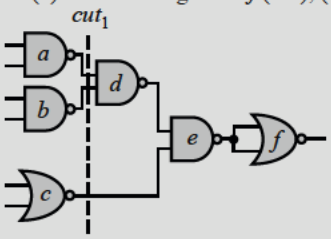


Fig. 4.7 Partitioning a region using different cutline approaches. (a) Alternating cutline directions. (b) Repeating cutline directions.

Example: Min-Cut Placement Using the KL Algorithm

Given: (1) circuit with gates *a-f* (left), (2) 2 × 4 layout (right), and (3) initial vertical cut cut_1.

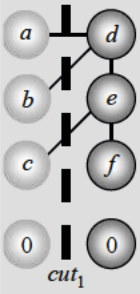




Task: find a placement with minimum wirelength using alternating cutline directions and the KL algorithm.

Solution:

After vertical cut cut_1, $L = \{a,b,c\}$ and $R = \{d,e,f\}$. Partition using the KL algorithm.

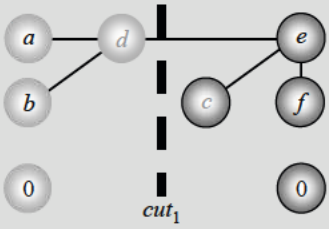


$D(a) = 1 \quad D(d) = 1$

$D(b) = 1 \quad D(e) = -1$

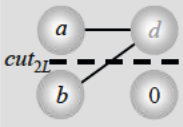
$D(c) = 1 \quad D(f) = -1$

$D(0) = 0 \quad D(0) = 0$



$\Delta g_1 = D(c) + D(d) - 2c(c,d) = 1 + 1 - 2(0) = 2$. Swap nodes *c* and *d*.

After horizontal cut cut_{2L}, $T = \{a,d\}$, $B = \{b\}$.

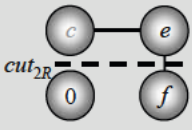


$D(a) = -1 \quad D(d) = 0$

$D(b) = 1 \quad D(0) = 0$

No swapping because no $\Delta g > 0$.

After horizontal cut cut_{2R}, $T = \{c,e\}$, $B = \{f\}$.



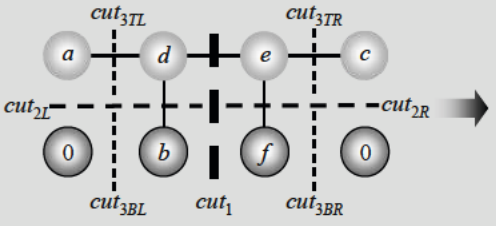
$D(c) = -1 \quad D(e) = 0$

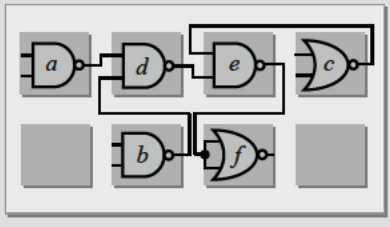
$D(0) = 0 \quad D(f) = 1$

No swapping because no $\Delta g > 0$.

Make four vertical cuts cut_{3TL}, cut_{3TR}, cut_{3BL} and cut_{3BR}. Each region has only one node, so terminate the algorithm.

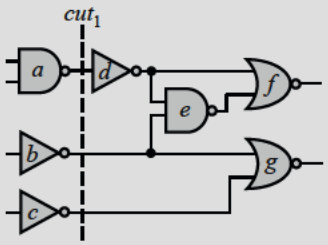
After external pin consideration (not shown), the final placement is





Example: Min-Cut Placement Using the FM Algorithm

Given: (1) circuit with gates *a*-*g* (left), (2) gate areas, (3) ratio factor $r = 0.5$, (4) initial partitioning with vertical cut cut_1, and (5) a 2×4 layout (right).



area(INV) = 1, *area*(NAND) = 2, *area*(NOR) = 2



Task: find a placement with minimum wirelength using alternating cutline directions and using the FM algorithm.

Solution:

Initial vertical cut cut_1: $L = \{a,b,c\}$, $R = \{d,e,f,g\}$.

Balance criterion: $0.5 \cdot 11 - 2 \le area(A) \le 0.5 \cdot 11 + 2 = 3.5 \le area(A) \le 7.5$.

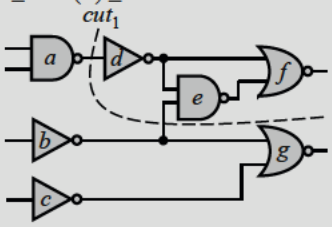
Iteration 1:

Gates *a*, *b*, *c* and *g* have maximum gain $\Delta g_1 = 1$.

Balance criterion for gates *a*, *b* and *c* is violated: $area(A) < 3.5$.

Balance criterion for gate *g* is met: $area(A) = 6$.

Move gate *g*.



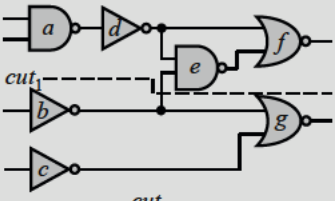
Iteration 2:

Gate *a* has maximum gain $\Delta g_2(a) = 1$, $area(A) = 4$, balance criterion is met.

Move gate *a* (further selection steps result in negative Δg and are omitted).

Maximum positive gain $G_2 = \Delta g_1 + \Delta g_2 = 2$.

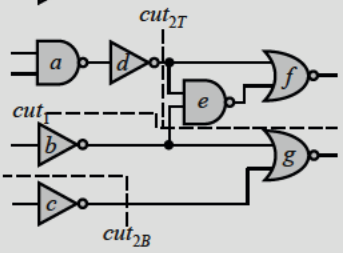
After cut cut_1, $L = \{a,d,e,f\}$, $R = \{b,c,g\}$, cut cost = 1.



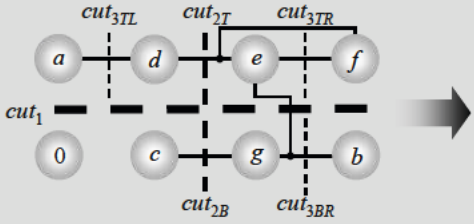
After cut cut_{2T}, $T = \{a,d\}$, $B = \{e,f\}$, cut cost = 1.

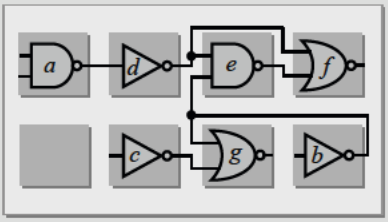
After cut cut_{2B}, $T = \{c\}$, $B = \{b,g\}$, cut cost = 1.

Make three more cuts cut_{3TL}, cut_{3TR} and cut_{3BR} such that every sub-region has one gate.



After external pin consideration (not shown), the final placement is





Naive min-cut algorithms do not consider the locations of connection pins within partitions that have already been visited. Likewise, the locations of fixed, external pin connections (pads) are also ignored. However, as illustrated in Fig. 4.8, cell *a* should properly be placed as close as possible to where the terminal *p'* is located.

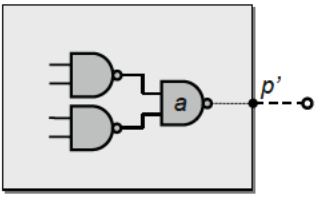


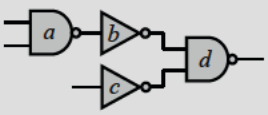
Fig. 4.8 Placement of cell *a* is close to the terminal *p'*, which represents a connection to a neighboring partition.

Formally developed by *A. E. Dunlop* and *B. W. Kernighan* [4.8], *terminal propagation* considers external pin locations during partitioning-based placement. During min-cut placement, external connections are represented by artificial connection points on the cutline and are dummy nodes in hypergraphs. The locations of the connection points affect placement cost functions and hence cell placements.

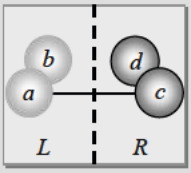
Min-cut placement with external connections assumes that the cells are placed in the centers of their respective partitions. If the related connections (dummy nodes) are *close* to the next partition cutline, these nodes are not considered when making the next cut. *Dunlop* and *Kernighan* define close as *projecting onto the middle third of the region boundary* that is being cut [4.8].

Example: Min-Cut Placement With External Connections

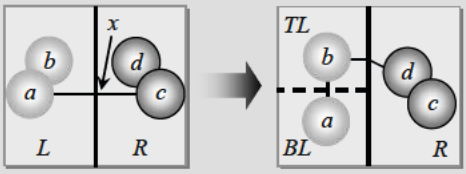
Given: gates *a-d* of a circuit (right).
Task: place the gates in a 2 × 2 grid.



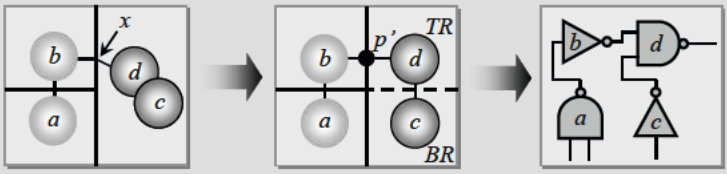
Solution:
Let gates *a-d* be represented by nodes *a-d*. Partition into *L* and *R*. Partition cost = 1.



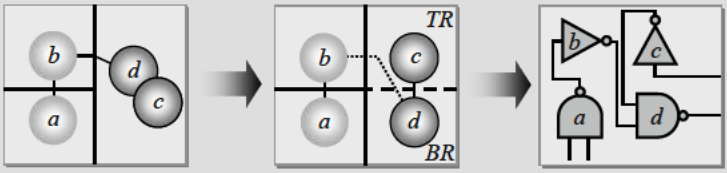
Partition *L* into *TL* and *BL* *independent* of *R*, since the connection *x* is close to the new horizontal cutline (assume that nodes are placed in the centers of their respective partitions).



Partition *R* into *TR* and *BR depending* on *L*, since the connection *x* is not close to the cutline. The addition of the dummy node *p'* means that node *d* moves to grid *TR* and node *c* moves to grid *BR* so that the partition cost of the *TR*-*BR* partition cost = 1.



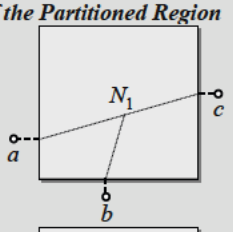
Note that without *p'*, the cost of placing *c* in *TR* and node *d* in *BR* is still 1, but the total cost with consideration of *L* becomes 2.



If a net *net* crosses the partitioned region, all its pins, including those that lie outside the region, must be considered. A rectilinear Steiner minimum tree (RSMT) (Sec. 4.2) is constructed that includes each pin outside the region. Each point at which this tree crosses the region's boundary induces a dummy node *p'* for *net*. If *p'* is close to a cutline, then it is ignored during partitioning. Otherwise, it is considered during partitioning along with cells that are contained in the region.

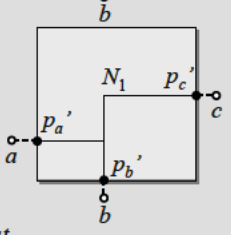
Example: Min-Cut Placement Considering Pins Outside of the Partitioned Region

Given: (1) layout region (right) and (2) net N_1 with three pins *a*-*c* not in the partitioned region.
Task: account for pins *a*-*c*.

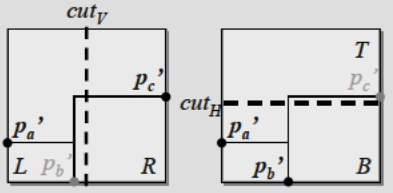


Solution:

For pins *a*-*c*, construct a rectilinear Steiner minimum tree (RSMT). Label all points of that tree that cross the layout region with dummy nodes p_a'-p_c'.



If a vertical cut cut_V is made, then p_b' is ignored because it is *close* to the cutline, while p_a' and p_c' are considered as real nodes. If a horizontal cut cut_H is made, then p_c' is ignored, and p_a' and p_b' are considered as real nodes.



► 4.3.2 Analytic Placement

Analytic placement minimizes a given objective, such as wirelength or circuit delay, using mathematical techniques such as numerical analysis or linear programming. Such methods often require certain assumptions, such as the differentiability of the objective function or the treatment of placeable objects as dimensionless points. For example, to facilitate the calculation of partial derivatives, it is common to optimize *quadratic*, rather than linear wirelength. When such algorithms place cells too close, i.e., creating overlaps, the cell locations must be spread further apart by dedicated post-processing techniques, so as to remove overlap.

Quadratic placement. The *squared* Euclidean distance

$$L(P) = \sum_{i=1,j=1}^{n} c(i,j)\left((x_i - x_j)^2 + (y_i - y_j)^2\right)$$

is used as the cost function, n is the total number of cells, and $c(i,j)$ is the connection cost between cells i and j. If cells i and j are not connected, then $c(i,j) = 0$. The terms $(x_i - x_j)^2$ and $(y_i - y_j)^2$ respectively give the squared horizontal and vertical distances between the centers of i and j. This formulation implicitly decomposes all nets into two-pin subnets. The quadratic form emphasizes the minimization of long connections, which tend to have negative impacts on timing.

Quadratic placement consists of two stages. During *global placement* (first stage), cells are placed so as to minimize the quadratic function with respect to the cell centers. Note that this placement is not legal. Usually, cells appear in large clusters with many cell overlaps. During *detailed placement* (second stage), these large clusters are broken up and all cells are placed such that no overlap occurs. That is, detailed placement legalizes all the cell locations and produces a high-quality, non-overlapping placement.

During *global placement*, each dimension can be considered independently. Therefore, the cost function $L(P)$ can be separated into x- and y-components

$$L_x(P) = \sum_{i=1,j=1}^{n} c(i,j)(x_i - x_j)^2 \quad \text{and} \quad L_y(P) = \sum_{i=1,j=1}^{n} c(i,j)(y_i - y_j)^2$$

With these cost functions, the placement problem becomes a convex quadratic optimization problem. Convexity implies that any local minimum solution is also a global minimum. Hence, the optimal x- and y-coordinates can be found by setting the partial derivatives of $L_x(P)$ and $L_y(P)$ to zero, i.e.,

$$\frac{\partial L_x(P)}{\partial X} = AX - b_x = 0 \text{ and } \frac{\partial L_y(P)}{\partial Y} = AY - b_y = 0$$

where A is a matrix with $A[i][j] = -c(i,j)$ when $i \neq j$, and $A[i][i]$ = the sum of incident connection weights of cell i. X is a vector of all the x-coordinates of the non-fixed cells, and b_x is a vector with $b_x[i]$ = the sum of x-coordinates of all fixed cells attached to i. Y is a vector of all the y-coordinates of the non-fixed cells, and b_y is a vector with $b_y[i]$ = the sum of y-coordinates of all fixed cells attached to i.

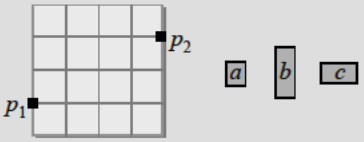
This is a system of linear equations for which iterative numerical methods can be used to find a solution. Known methods include the *conjugate gradient* (*CG*) method [4.27] and the *successive over-relaxation* (*SOR*) method.

During *detailed placement*, cells are spread out to remove all overlaps. Known methods include those described for min-cut placement (Sec. 4.3.1) [4.33] and force-directed placement [4.9].

Example: Quadratic Placement

Given: (1) placement P with two fixed points p_1 (100,175) and p_2 (200,225), (3) three free blocks a-c and (4) nets N_1-N_4.

N_1 (P_1,a) N_2 (a,b) N_3 (b,c) N_4 (c,P_2)



Task: find the coordinates of blocks (x_a, y_a), (x_b, y_b) and (x_c, y_c).

Solution:

Solve for x-coordinates.

$$L_x(P) = (100 - x_a)^2 + (x_a - x_b)^2 + (x_b - x_c)^2 + (x_c - 200)^2$$

$$\frac{\partial L_x(P)}{x_a} = -2(100 - x_a) + 2(x_a - x_b) = 4x_a - 2x_b - 200 = 0$$

$$\frac{\partial L_x(P)}{x_b} = -2(x_a - x_b) + 2(x_b - x_c) = -2x_a + 4x_b - 2x_c = 0$$

$$\frac{\partial L_x(P)}{x_c} = -2(x_b - x_c) + 2(x_c - 200) = -2x_b + 4x_c - 400 = 0$$

Put in matrix form $AX = b_x$.

$$\begin{bmatrix} 4 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 4 \end{bmatrix} \begin{bmatrix} x_a \\ x_b \\ x_c \end{bmatrix} = \begin{bmatrix} 200 \\ 0 \\ 400 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_a \\ x_b \\ x_c \end{bmatrix} = \begin{bmatrix} 100 \\ 0 \\ 200 \end{bmatrix}$$

Solve for X: $x_a = 125$, $x_b = 150$, $x_c = 175$.

Solve for y-coordinates.

$$L_y(P) = (175 - y_a)^2 + (y_a - y_b)^2 + (y_b - y_c)^2 + (y_c - 225)^2$$

$$\frac{\partial L_y(P)}{y_a} = -2(175 - y_a) + 2(y_a - y_b) = 4y_a - 2y_b - 350 = 0$$

$$\frac{\partial L_y(P)}{y_b} = -2(y_a - y_b) + 2(y_b - y_c) = -2y_a + 4y_b - 2y_c = 0$$

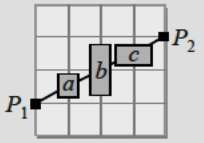
$$\frac{\partial L_y(P)}{y_c} = -2(y_b - y_c) + 2(y_c - 225) = -2y_b + 4y_c - 450 = 0$$

Put in matrix form $AY = b_y$.

$$\begin{bmatrix} 4 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 4 \end{bmatrix} \begin{bmatrix} y_a \\ y_b \\ y_c \end{bmatrix} = \begin{bmatrix} 350 \\ 0 \\ 450 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} y_a \\ y_b \\ y_c \end{bmatrix} = \begin{bmatrix} 175 \\ 0 \\ 225 \end{bmatrix}$$

Solve for Y: $y_a = 187.5$, $y_b = 200$, $y_c = 212.5$.

Final solution: a (125,187.5), b (150,200) and c (175,212.5).



Force-directed placement. During force-directed placement, cells and wires are modeled using the mechanical analogy of a mass-spring system, i.e., masses connected to *Hooke's-Law* springs. Each cell exercises attraction toward other cells, where the attraction force is directly proportional to distance. If free movement is granted to all elements of the mass-spring system, then all cells will eventually settle in a configuration that achieves force-equilibrium. Relating this back to circuit placement, the wirelength will be minimized if all cells reach their equilibrium locations. Thus, the goal is to have all cells settle to a placement with force equilibrium.

Force-directed placement, developed in the late 1970s by *N. R. Quinn* [4.24], is a special case of quadratic placement. The potential energy of a stretched Hooke's-Law spring between cells a and b is directly proportional to the squared Euclidean distance between a and b. Furthermore, the spring force exerted on a given cell i is the partial derivative of potential energy with respect to i's position. Thus, determining the energy-minimal positions of cells is equivalent to minimizing the sum of squared Euclidean distances.

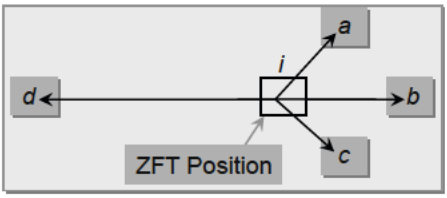
Given two connected cells a and b, the attraction force $\overrightarrow{F_{ab}}$ exerted on a by b is

$$\overrightarrow{F_{ab}} = c(a,b) \cdot (\vec{b} - \vec{a})$$

where $c(a,b)$ is the connection weight (priority) between cells a and b, and $(\vec{b} - \vec{a})$ is the vector difference of the positions of a and b in the Euclidean plane. The sum of forces exerted on a cell i connected to other cells j is (Fig. 4.9)

$$\overrightarrow{F_i} = \sum_{c(i,j)\neq 0} \overrightarrow{F_{ij}}$$

The position that minimizes this sum of forces is known as the *zero-force target* (*ZFT*).



$$\min \vec{F}_i = c(i,a)\cdot(\vec{a}-\vec{i}) + c(i,b)\cdot(\vec{b}-\vec{i}) + c(i,c)\cdot(\vec{c}-\vec{i}) + c(i,d)\cdot(\vec{d}-\vec{i})$$

Fig. 4.9 ZFT position of cell *i*, which is connected to four other cells *a*-*d*.

Using this formulation, two possible extensions are as follows. First, in addition to the attraction force, a repulsion force can be added for unconnected cells to avoid overlap and take into account the overall layout (for a balanced placement). These forces form a system of linear equations that can be solved efficiently using analytic techniques, as with quadratic placement. Second, for each cell, there is an ideal minimum-energy (ZFT) position. By iteratively moving each cell to this position (or nearby, if that position is already occupied) or by cell swapping, gradual improvements can be made. In the end, all cells will be in the position with minimum-force equilibrium configuration.

Basic force-directed placement algorithms. Force-directed placement algorithms iteratively move all cells to their respective ZFT positions. In order to find a cell *i*'s ZFT position (x_i^0, y_i^0), all forces that affect *i* must be taken into account. Since the ZFT position minimizes the force, both the *x*- and *y*- direction forces are set to zero.

$$\sum_{c(i,j)\neq 0} c(i,j)\cdot(x_j^0 - x_i^0) = 0 \quad \text{and} \quad \sum_{c(i,j)\neq 0} c(i,j)\cdot(y_j^0 - y_i^0) = 0$$

Rearranging the variables to solve for x_i^0 and y_i^0 yields

$$x_i^0 = \frac{\sum_{c(i,j)\neq 0} c(i,j)\cdot x_j^0}{\sum_{c(i,j)\neq 0} c(i,j)} \quad \text{and} \quad y_i^0 = \frac{\sum_{c(i,j)\neq 0} c(i,j)\cdot y_j^0}{\sum_{c(i,j)\neq 0} c(i,j)}$$

Using these equations, the ZFT position of a cell *i* that is connected to other cells *j* can be computed.

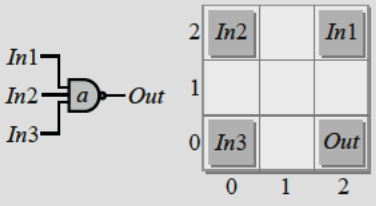
Example: ZFT Position

Given: (1) circuit with NAND gate (cell) *a* (left), (2) four I/O pads – *In*1-*In*3 and *Out* – and their positions, (3) a 3 × 3 layout grid (right), and (4) weighted connections.

*In*1 (2,2) *In*2 (0,2) *In*3 (0,0) *Out* (2,0)

$c(a,In1) = 8$ $c(a,In2) = 10$

$c(a,In3) = 2$ $c(a,Out) = 2$



Task: find the ZFT position of cell *a*.

Solution:

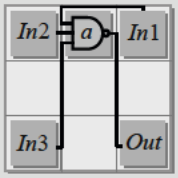
$$x_a^0 = \frac{\sum_{c(i,j)\neq 0} c(a,j)\cdot x_j^0}{\sum_{c(i,j)\neq 0} c(a,j)} = \frac{c(a,In1)\cdot x_{In1} + c(a,In2)\cdot x_{In2} + c(a,In3)\cdot x_{In3} + c(a,Out)\cdot x_{Out}}{c(a,In1)+c(a,In2)+c(a,In3)+c(a,Out)}$$

$$= \frac{8\cdot 2+10\cdot 0+2\cdot 0+2\cdot 2}{8+10+2+2} = \frac{20}{22} \approx 0.9$$

$$y_a^0 = \frac{\sum_{c(i,j)\neq 0} c(a,j)\cdot y_j^0}{\sum_{c(i,j)\neq 0} c(a,j)} = \frac{c(a,In1)\cdot y_{In1} + c(a,In2)\cdot y_{In2} + c(a,In3)\cdot y_{In3} + c(a,Out)\cdot y_{Out}}{c(a,In1)+c(a,In2)+c(a,In3)+c(a,Out)}$$

$$= \frac{8\cdot 2+10\cdot 2+2\cdot 0+2\cdot 0}{8+10+2+2} = \frac{36}{22} \approx 1.6$$

ZFT position of cell *a* is at (1,2).



The following gives a high-level sketch of the force-directed placement approach.

Force-Directed Placement Algorithm
Input: set of all cells *V*
Output: placement *P*

```
1.  P = PLACE(V)                    // arbitrary initial placement
2.  loc = LOCATIONS(P)              // set coordinates for each cell in P
3.  foreach (cell c ∈ V)
4.     status[c] = UNMOVED
5.  while (ALL_MOVED(V) || !STOP())  // continue until all cells have been
                                    //    moved or some stopping
                                    //    criterion is reached
6.     c = MAX_DEGREE(V,status)     // unmoved cell that has largest
                                    //    number of connections
7.     ZFT_pos = ZFT_POSITION(c)    // ZFT position of c
8.     if (loc[ZFT_pos] == Ø)       // if position is unoccupied,
9.        loc[ZFT_pos] = c          //    move c to its ZFT position
10.    else
11.       RELOCATE(c,loc)           // use methods discussed below
12.    status[c] = MOVED            // mark c as moved
```

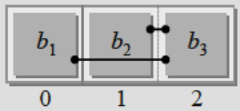
Starting with an initial placement (line 1), the algorithm selects a cell *c* that has the largest number of connections and has not been moved (line 6). It then calculates the ZFT position of *c* (line 7). If that position is unoccupied, then *c* is moved there (lines 8-9). Otherwise, *c* can be moved according to several methods discussed next (lines 10-11). This process continues until some stopping criterion is reached or until all cells have been considered (lines 5-12).

Finding a valid location for a cell with an occupied ZFT position. Let p be an incoming cell and let q be the cell that is currently in p's ZFT position. Following are four options for how to rearrange p and q.

- If possible, move p to a cell position close to q.
- Compute the cost difference if p and q were to be swapped. If the total cost reduces, i.e., the weighted connection length $L(P)$ is smaller, then swap p and q.
- *Chain move*: cell p is moved to cells q's location. Cell q, in turn, is shifted to the next position. If a cell r is occupying this space, cell r is shifted to the next position. This continues until all affected cells are placed.
- *Ripple move*: cell p is moved to cell q's location. A new ZFT position for cell q is computed (discussed below). This ripple effect continues until all cells are placed.

Example: Force-Directed Placement

Given: (1) placement of blocks b_1-b_3 (right) and (2) weighted nets N_1 and N_2.



$N_1 = (b_1,b_3)$ $N_2 = (b_2,b_3)$ $c(N_1) = 2$ $c(N_2) = 1$

Task: use force-directed placement to find a heuristic minimum-wirelength placement.

Solution:

Consider cell b_3.

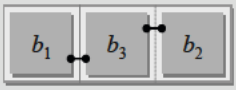
ZFT position for $b_3 = x^0_{b_3} = \dfrac{\sum_{c(b_3,j)\neq 0} c(b_3,j)\cdot x^0_j}{\sum_{c(b_3,j)\neq 0} c(b_3,j)} = \dfrac{2\cdot 0+1\cdot 1}{2+1} \approx 0$, which is occupied by cell b_1.

$L(P)$ before move = 3, $L(P)$ after move = 3. Cells b_3 and b_1 should not swap.

Consider cell b_2.

ZFT position for $b_2 = x^0_{b_2} = \dfrac{\sum_{c(b_2,j)\neq 0} c(b_2,j)\cdot x^0_j}{\sum_{c(b_2,j)\neq 0} c(b_2,j)} = \dfrac{1\cdot 2}{1} = 2$, which is occupied by cell b_3.

$L(P)$ before move = 3, $L(P)$ after move = 2. Cells b_2 and b_3 should swap.



Force-directed placement with ripple moves. In force-directed placement with ripple moves [4.30], the cells are sorted in descending order according to their connection degrees (lines 1-3). During each iteration (line 4), the ZFT position for the next cell in the list (*seed*) is computed (line 9). If the position is free, *seed* is moved there (lines 12-17). If the position is occupied, then its current inhabitant is moved next, assuming it has not already been moved. In order to avoid infinite loops,

once a cell has been moved, it is marked as *LOCKED* and can no longer be moved until the next iteration (lines 18-40).

```
Force-Directed Placement with Ripple Moves Algorithm
Input: set of all cells V
Output: positions of each cell pos
1.  foreach (cell c ∈ V)
2.    degree[c] = CONNECTION_DEGREE(c)          // compute connection degree
3.  L = SORT(V,degree)                          // descending order by degree
4.  while (iteration_count < iteration_limit)
5.    end_ripple_move = false
6.    seed = NEXT_ELEMENT(L)                    // next cell in L
7.    pos_type[seed] = VACANT                   // position type = VACANT
8.    while (end_ripple_move == false)
9.      curr_pos = ZFT_POSITION(seed)
10.     curr_pos_type = ZFT_POSITION_TYPE(seed)
11.     switch (curr_pos_type)
12.       case VACANT:
13.         pos[seed] = MOVE(seed,curr_pos)     // move seed to curr_pos
14.         pos_type[seed] = LOCKED
15.         end_ripple_move = true
16.         abort_count = 0
17.       break
18.       case LOCKED:
19.         pos[seed] = MOVE(seed,NEXT_FREE_POS())
20.         pos_type[seed] = LOCKED
21.         end_ripple_move = true
22.         abort_count = abort_count + 1
23.         if (abort_count > abort_limit)
24.           foreach (pos_type[c] == LOCKED)
25.             RESET(pos_type[c])
26.           iteration_count = iteration_count + 1
27.       break
28.       case SAME_AS_PRESENT_LOCATION:
29.         pos[seed] = MOVE(seed,curr_pos)
30.         pos_type[seed] = LOCKED
31.         end_ripple_move = true
32.         abort_count = 0
33.       break
34.       case OCCUPIED:                        // occupied but not locked
35.         prev_cell = CELL(curr_pos)          // cell in ZFT position
36.         pos[seed] = MOVE(seed,curr_pos)
37.         pos_type[seed] = LOCKED
38.         seed = prev_cell
39.         end_ripple_move = false
40.         abort_count = 0
41.       break
```

In each iteration, an upper bound of fixed cells (*abort_limit*) is imposed. If the number of fixed cells exceeds this limit, then all fixed cells are released and the cell

with the next highest connectivity is considered in the new iteration (lines 23-26). Each cell has one of four options (lines 11-41).

- *VACANT*: ZFT position is free. Place the cell there and proceed with the next iteration (lines 12-17).
- *LOCKED*: ZFT position is occupied and fixed. Place the cell at the next free position and increment *abort_count*. If *abort_count* > *abort_limit*, stop the ripple moves and go on with the next cell in list L (lines 18-27).
- *SAME_AS_PRESENT_LOCATION*: ZFT position is the same as the current position of the cell. Place cell here and consider the next cell in L (lines 28-33).
- *OCCUPIED*: ZFT position is occupied but not by a locked cell. Place the cell here and move the cell that had occupied the ZFT position (lines 34-41).

The inner while loop (cell movement) is called only when *end_ripple_move* is **false**. Note that this flag is set to **true** when the seed cell's ZFT position is *VACANT*, *LOCKED* or *SAME_AS_PRESENT_LOCATION*. In these cases, the algorithm can move on with the next cell in the list. The outer while loop goes through list L and finds the best ZFT position of each cell until *iteration_limit* is reached.

► 4.3.3 Simulated Annealing

Simulated annealing (SA) (Sec. 3.5.3) is the basis of some of the most well-known placement algorithms.

Simulated Annealing Algorithm for Placement
Input: set of all cells V
Output: placement P

```
1.   T = T0                                   // set initial temperature
2.   P = PLACE(V)                             // arbitrary initial placement
3.   while (T > Tmin)
4.     while (!STOP())                        // not yet in equilibrium at T
5.       new_P = PERTURB(P)
6.       Δcost = COST(new_P) – COST(P)
7.       if (Δcost < 0)                       // cost improvement
8.         P = new_P                          // accept new placement
9.       else                                 // no cost improvement
10.        r = RANDOM(0,1)                    // random number [0,1)
11.        if (r < e^(-Δcost/T))              // probabilistically accept
12.          P = new P
13.    T = α · T                              // reduce T, 0 < α < 1
```

From an initial placement (line 2), the PERTURB function generates a new placement *new_P* by perturbing the current placement (line 5). $\Delta cost$ records the cost difference between the previous placement P and *new_P* (line 6). If the cost improves, i.e., $\Delta cost < 0$, then *new_P* is accepted (lines 7-8). Otherwise, *new_P* is

probabilistically accepted (lines 9-12). Once the annealing process has "reached equilibrium" (e.g., after a prescribed number of move attempts) at the current temperature, the temperature is decreased. This process continues until $T < T_{min}$.

TimberWolf. One of the earliest academic packages, *TimberWolf*, was developed at the *University of California, Berkeley* by *C. Sechen* and later commercialized [4.28] [4.29]. From the widths of all cells in the netlist, the algorithm produces an initial placement with all cells in rows, along with a target row length. TimberWolf allots additional area around any macro cells to allow sufficient interconnect area, and uses a specialized pin assignment algorithm to optimize any macro-cell pins that are not fixed to specific locations. The macro cells are then placed simultaneously along with the standard cells. The original TimberWolf (v.3.2) only optimizes the placement of standard cells while I/Os and macros stay in their original locations.

The placement algorithm consists of the following three stages.

1. Place standard cells with minimum total wirelength using simulated annealing.
2. Globally route (Chap. 5) the placement by introducing routing channels, when necessary. Recompute and minimize the total wirelength.
3. Locally optimize the placement with the goal of minimizing channel height.

The remainder of this section discusses the first (placement) stage. For further reading on TimberWolf, see [4.29].

Perturb. The PERTURB function generates a new placement from an existing placement using one of the following actions.

- MOVE: Shift a cell to a new position (another row)
- SWAP: Exchange two cells
- MIRROR: Reflect the cell orientation around the y-axis, used only if MOVE and SWAP are infeasible

The scope of PERTURB is limited to a small window of size $w_T \times h_T$ (Fig. 4.10). For MOVE, a cell can only be moved within this window.

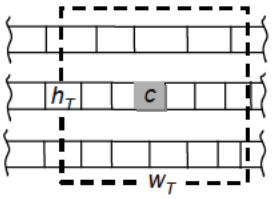


Fig. 4.10 Window with the dimensions $w_T \times h_T$ around a standard cell c.

For SWAP, two cells a (x_a,y_a) and b (x_b,y_b) can be exchanged only if

$$|x_a - x_b| \leq w_T \quad \text{and} \quad |y_a - y_b| \leq h_T$$

The window size (w_T,h_T) depends on the current temperature T, and decreases as temperature reduces. The window size for the next iteration is based on the current temperature T_{curr} and the next iteration's temperature T_{next} using

$$w_{T_{next}} = w_{T_{curr}} \frac{\log(T_{next})}{\log(T_{curr})} \quad \text{and} \quad h_{T_{next}} = h_{T_{curr}} \frac{\log(T_{next})}{\log(T_{curr})}$$

Cost. The COST function in TimberWolf (v3.2) is defined as $\Gamma = \Gamma_1 + \Gamma_2 + \Gamma_3$, the sum of three parameters – (1) total estimated wirelength Γ_1, (2) amount of overlap Γ_2, and (3) row inequality length Γ_3.

Γ_1 is computed as the summation of each net's *half-perimeter wirelength* (*HPWL*), which is defined as its horizontal span plus its vertical span. Weights for each direction, horizontal weight w_H and vertical weight w_V, can also be applied. Given a priority weight γ_1, Γ_1 is defined as the sum of the total wirelength over all nets $net \in Netlist$, where *Netlist* is the set of all nets.

$$\Gamma_1 = \gamma_1 \cdot \sum_{net \in Netlist} w_H(net) \cdot x_{net} + w_V(net) \cdot y_{net}$$

A higher weight value for *net* gives higher emphasis on reducing *net*'s wirelength. Weights can also be used for direction control – giving preference to a certain wiring direction. During standard-cell placement where feedthrough cells are limited, low horizontal weights $w_H(net)$ encourage the usage of horizontal channels rather than the vertical connections.

Γ_2 represents the total cell overlap of the placement. Let $o(i,j)$ represent the area of overlap between cells i and j. Given a priority weight γ_2, Γ_2 is defined as the sum of the square of all cell overlaps between cells i and j, where $i \in V, j \in V, i \neq j$, with V being the set of all cells.

$$\Gamma_2 = \gamma_2 \cdot \sum_{i \in V, j \in V, i \neq j} o(i,j)^2$$

Larger overlaps, which require more effort to correct, are penalized more heavily due to the quadratic form.

Γ_3 represents the cost of all row lengths $L(row)$ that deviate from the goal length $L_{opt}(row)$ during placement. Cell movement can often lead to row length variation, where the resulting rows lengths deviate from the goal length. In practice, uneven rows can waste area and induce uneven wire distributions. Both phenomena can lead to increased total wirelength and total congestion. Given a priority factor γ_3, Γ_3 is defined as the sum of row length deviation for all rows $row \in Rows$, where *Rows* is the set of all rows.

$$\Gamma_3 = \gamma_3 \cdot \sum_{row \in Rows} \left| L(row) - L_{opt}(row) \right|$$

Temperature Reduction. The temperature T is reduced by a cooling factor α. This value is empirically chosen and often depends on the temperature range. The annealing process starts at a high temperature, such as $4 \cdot 10^6$ (units do not play a role). Initially, the temperature is reduced quickly ($\alpha \approx 0.8$). After a certain number of iterations, the temperature reduces at a slower rate ($\alpha \approx 0.95$), when the placement is being fine-tuned. Toward the end, the temperature is again reduced at a fast pace ($\alpha \approx 0.8$), corresponding to a "quenching" step. TimberWolf finishes when $T < T_{min}$ where $T_{min} = 1$.

Number of Times Through the Inner Loop. At each temperature, a number of calls are made to PERTURB to generate new placements. This number is intended to achieve *equilibrium* at the given temperature, and depends on the size of the design. The authors of [4.29] experimentally determined that designs with ~200 cells require 100 iterations per cell, or roughly $2 \cdot 10^4$ runs per temperature step. Other simulated annealing approaches use acceptance ratio as an equilibrium criterion, e.g., *Lam* [4.20] shows that a target acceptance ratio of 44% produces competitive results.

► 4.3.4 Modern Placement Algorithms

Algorithms for global placement have been studied by many researchers since the late 1980s, and the prevailing paradigm has changed several times to address new challenges arising in commercial chip designs [4.6][4.22]. This section reviews modern algorithms for global placement, while the next section covers legalization and detailed placement, as well as the need for such a separation of concerns. Timing-driven placement is discussed in Sec. 8.3.

The global placement algorithms in use today can handle extremely large netlists using analytic techniques, i.e., by modeling interconnect length with mathematical functions and optimizing these functions with numerical methods. Dimensions and sizes of standard cells are initially ignored to quickly find a seed placement, but are then gradually factored into the placement optimization so as to avoid uneven densities or routing congestion. Two common paradigms are based on *quadratic* and *force-directed placement*, and on *nonlinear optimization*. The former was introduced earlier and seeks to approximate wirelength by quadratic functions, which can be minimized by solving linear systems of equations (Sec. 4.3.2). The latter relies on more sophisticated functions to approximate interconnect length, and requires more sophisticated numerical optimization algorithms [4.4][4.16][4.17].

Of the two types, quadratic methods are easier to implement and appear to be more scalable in terms of runtime. Nonlinear methods require careful tuning to achieve numerical stability and often run much slower than quadratic methods. However, nonlinear methods can better account for the shapes and sizes of standard cells and

especially macro blocks, whereas quadratic placement requires dedicated *spreading* techniques. Both placement techniques are often combined with netlist clustering to reduce runtime [4.4][4.16][4.17][4.34], in a manner that is conceptually similar to multilevel partitioning (Chap. 2). However, the use of clustering in placement often leads to losses in solution quality. Thus, it is an open question whether the multilevel approach can outperform the flat approach in terms of runtime and solution quality for placement, as is the case for partitioning [4.5].

The aspects of quadratic placement that appear most impactful in practice are (1) the representation of multi-pin nets by sets of graph edges (net modeling), (2) the choice of algorithms for spreading, and (3) the strategy for interleaving spreading with quadratic optimization. Two common net models include *cliques*, where every pair of pins is connected by an edge with a small weight, and *stars*, where every pin is connected to a "star-point" that represents the net (or hyperedge) itself [4.12]. Edges representing a net are given fractional weights that add up to the net's weight (or to unity). For nets with fewer pins, cliques are preferred because they do not introduce new variables. For larger nets, stars are useful because they entail only a linear number of graph edges [4.34]. The star-point can be movable or placed in the centroid (average location or *barycenter*) of its neighbors. The latter option is preferred in practice because (1) it corresponds to the optimal location of the star-point in quadratic placement, and (2) it saves two (x,y) variables. Some placers additionally use a *linearization* technique that assigns a constant weight

$$w(i,j) = \frac{1}{x_i - x_j}$$

to each quadratic term $(x_i - x_j)^2$ within the objective function. The weight $w(i,j)$ has the effect of turning each such *squared wirelength term* into a *linear wirelength term*, and can therefore be truer to an underlying linear wirelength objective. These weights are treated as constants, and then updated between rounds of quadratic optimization. A more accurate placement-dependent net model is proposed in [4.32].

Spreading is based on estimates of cell density in different regions of the chip. These estimates are computed by allocating movable objects into bins of a regular grid, and comparing their total area to available capacity per bin. Spreading can be performed after quadratic optimization using a combination of sorting by location and geometric scaling [4.32]. For example, cells in a dense region may be sorted by their x-coordinates and then re-placed in this order, so as to avoid overlaps. An implicit spreading method to reduce overlap is to enclose a set of cells in a rectangular region and then perform linear scaling [4.18].

Spreading can also be integrated directly into quadratic optimization by adding *spreading forces* that push movable objects away from dense regions. These additional forces are modeled by imaginary fixed pins (anchor points) and imaginary wires pulling individual standard cells toward fixed pins [4.11]. This integration allows conventional quadratic placement to trade interconnect minimization for

smaller overlaps between modules. *FastPlace* [4.34] first performs simple geometric scaling and then uses the resulting locations as anchor points during quadratic optimization. These steps of spreading and quadratic optimization are interleaved in FastPlace to encourage spreading that does not conflict with interconnect optimization. Researchers have also sought to develop spreading algorithms that are sufficiently accurate to be invoked only once after quadratic optimization [4.35].

Analytic placement can be extended to optimize not only interconnect length, but also routing congestion [4.31]. This requires wiring congestion estimation, which is similar to density estimation and is also maintained on a regular grid. Congestion information can be used in the same ways as density estimation to perform spreading. Some researchers have developed post-processors to improve congestion properties of a given placement [4.21].

Several modern placers are available free of charge for research purposes. As of 2010, the most accessible placers are *APlace* [4.16][4.17], *Capo* [4.2][4.26], *FastPlace 3.0* [4.34], *mPL6* [4.4], and *simPL* [4.18]. All except simPL[3] are equipped with legalizers and detailed placers so as to produce legal and highly optimized solutions. mPL6 is significantly slower than FastPlace, but finds solutions with smaller total interconnect length. Capo, a min-cut placer, is available in C++ source code. Its runtime is between that of mPL6 and FastPlace, but in many cases it produces solutions that are inferior to FastPlace solutions in terms of total interconnect length. However, for designs where achieving routability is difficult, Capo offers a better chance to produce a routable placement. It is also competitive on smaller designs (below 50,000 movable objects), especially those with high density and many fixed obstacles.

4.4 Legalization and Detailed Placement

Global placement assigns locations to standard cells and larger circuit modules, e.g., macro blocks. However, these locations typically do not align with power rails, and may have continuous (real) coordinates rather than discrete coordinates. Therefore, the global placement must be *legalized*. The allowed legal locations are equally spaced within pre-defined rows, and the point-locations from global placement should be snapped to the closest possible legal locations (Fig. 4.11).

Legalization is necessary not only *after global placement*, but also *after incremental changes* such as cell resizing and buffer insertion during physical synthesis (Sec. 8.5). Legalization seeks to find legal, non-overlapping placements for all placeable modules so as to minimize any adverse impact on wirelength, timing and other design objectives. Unlike algorithms for "cell spreading" during global placement (Sec. 4.3), legalization typically assumes that the cells are distributed fairly well

[3] simPL uses FastPlace-DP [4.23] for both legalization and detailed placement.

throughout the layout region and have relatively small mutual overlap. Once a legal placement is available, it can be improved with respect to a given objective by means of *detailed placement* techniques, such as swapping neighboring cells to reduce total wirelength, or sliding cells to one side of the row when unused space is available. Some detailed placers target routability, given that route topologies can be determined once a legal placement is available.

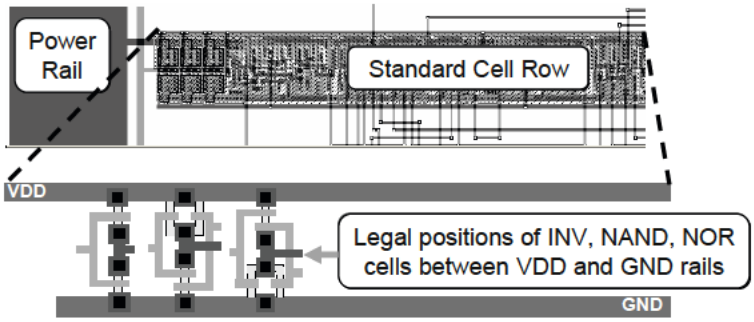


Fig. 4.11 An instance of detailed placement. Between each *VDD* and *GND* rail, cells must be placed in non-overlapping site-aligned locations.

One of the simplest and fastest techniques for legalization is *Tetris* by *D. Hill* [4.10]. It sorts cells by x-coordinate and processes them greedily. Each cell is placed at the closest available legal location that does not exceed the row capacity. In its purest form, the Tetris algorithm has several known drawbacks, one being its ignorance of the netlist. Another drawback is that in the presence of large amounts of whitespace, the Tetris algorithm can place cells onto one side of the layout, and thus considerably increase wirelength or circuit delay if some I/O pads are fixed on the opposite side of the layout.

Several variations of the Tetris algorithm exist. One variant subdivides the layout into regions and performs Tetris legalization in each region: this prevents the large displacements observed with the original version. Other variants try to account for wirelength while finding the best legal location for a given cell [4.10]. There are also versions that inject unused space between legalized cells, so as to avoid routing congestion [4.21].

Some algorithms for legalization and placement are co-developed with global placement algorithms. For instance, in the context of min-cut placement, detailed placement can be performed by optimal partitioners and end-case placers [4.1][4.3] invoked in very small (end-case) bins that are produced after the netlist is repeatedly partitioned. Given that these end-case bins contain a very small number of cells (four to six), optimal locations can be found by exhaustive enumeration or *branch-and-bound*. For larger bins, partitioning can be performed optimally (with up to ~35 cells in a bin). Some analytic algorithms perform legalization in iterations [4.32]. At each iteration, cells closest to legal sites are identified and snapped to legal sites, then considered fixed thereafter. After a round of analytic placement,

another group of cells is snapped to legal sites, and the process continues until all cells have been given legal locations.

A common problem with simple and fast legalization algorithms is that some cells may travel a long distance, thus significantly increasing the length and, hence, delay of incident nets. This phenomenon can be mitigated by detailed placement. For example, optimal branch-and-bound placers [4.3] can reorder groups of neighboring cells in a row. Such groups of cells are often located in a sliding window; the optimal placer reorders cells in a given window so as to improve total wirelength (accounting for connections to cells with fixed locations outside the window).

A more scalable optimization splits the cells in a given window into left and right halves, and optimally interleaves the two groups while preserving the relative order of cells from each group [4.12]. Up to 20 cells per window can be interleaved efficiently during detailed placement, whereas branch-and-bound placement can typically handle only up to eight cells [4.3]. These two optimizations can be combined for greater impact.

Sometimes, wirelength can be improved by reordering cells that are not adjacent. For example, pairs of non-adjacent cells connected by a net can be swapped [4.23], and sets of three such cells can be cycled. Yet another detailed placement optimization is possible when unused space is available between cells placed in a row. These cells can be shifted to either side, or to intermediate locations. Optimal locations to minimize wirelength can be found by a polynomial-time algorithm [4.15], which is practical in many applications.

Software implementations of legalization and detailed placement are often bundled, but are sometimes independent of global placement. One example is *FastPlace-DP* [4.23] (binary available from the authors). FastPlace-DP works best when the input placement is almost legal or requires only a small number of local changes. FastPlace-DP performs a series of simple but efficient incremental optimizations which typically decrease interconnect length by several percent. On the other end of the spectrum is *ECO-System* [4.25]. It is integrated with the *Capo* placer [4.2][4.26] and uses more sophisticated yet slower optimizations. ECO-System first analyzes a given placement and identifies regions where cells overlap so much that they need to be re-placed. The Capo algorithm is then applied simultaneously to each region so as to ensure consistency. Capo integrates legalization and detailed placement into global min-cut placement. Therefore, ECO-System will produce a legal placement even if the initial placement requires significant changes.

Other strategies, such as the use of *linear programming* [4.7] and *dynamic programming* [4.14], have been integrated into legalization and detailed placement with promising results. The legalization of mixed-size netlists that contain large movable blocks is particularly challenging [4.14].

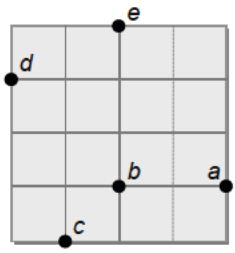
Chapter 4 Exercises

Exercise 1: Estimating Total Wirelength

Consider the five-pin net with pins *a*-*e* (right). Each grid edge has unit length.

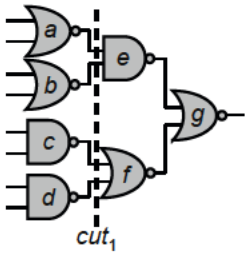
(a) Draw a rectilinear minimum-length chain, a rectilinear minimum spanning tree (RMST), and a rectilinear Steiner minimum tree (RSMT) to connect all pins.

(b) Find the weighted total wirelength using each estimation technique from (a) if each grid edge has weight = 2.



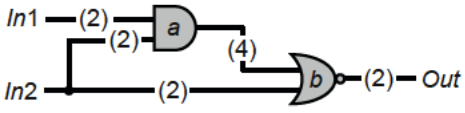
Exercise 2: Min-Cut Placement

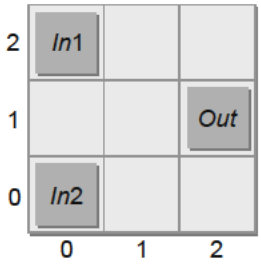
Perform min-cut placement to place gates *a*-*g* on a 2 × 4 grid. Use the Kernighan-Lin algorithm for partitioning. Use alternating (horizontal and vertical) cutlines. The cutline cut_1 represents the initial vertical cut. Each edge on the grid has capacity $\sigma_P(e) = 2$. Estimate whether the placement is routable.



Exercise 3: Force-Directed Placement

A circuit with two gates *a* and *b* and three I/O pads *In*1 (0,2), *In*2 (0,0) and *Out* (2,1) is given (left). The weights of the connections are shown below. Calculate the ZFT positions of the two gates. Place the circuit on a 3 × 3 grid (right).





Exercise 4: Global and Detailed Placement

What are the main differences between global and detailed placement? Explain why the global and detailed placement steps are performed separately. Explain why detailed placement follows global placement.

Chapter 4 References

[4.1] M. Breuer, "Min-Cut Placement", *J. Design Autom. and Fault-Tolerant Computing* 10 (1977), pp. 343-382.

[4.2] A. E. Caldwell, A. B. Kahng and I. L. Markov, "Can Recursive Bisection Alone Produce Routable Placements?", *Proc. Design Autom. Conf.*, 2000, pp. 477-482.

[4.3] A. E. Caldwell, A. B. Kahng and I. L. Markov, "Optimal Partitioners and End-Case Placers for Standard-Cell Layout", *IEEE Trans. on CAD* 19(11) (2000), pp. 1304-1313.

[4.4] T. F. Chan, J. Cong, J. R. Shinnerl, K. Sze and M. Xie, "mPL6: Enhanced Multilevel Mixed-Size Placement", *Proc. Intl. Symp. on Phys. Design*, 2006, pp. 212-221.

[4.5] H. Chen, C.-K. Cheng, N.-C. Chou, A. B. Kahng, J. F. MacDonald, P. Suaris, B. Yao and Z. Zhu, "An Algebraic Multigrid Solver for Analytical Placement with Layout Based Clustering", *Proc. Design Autom. Conf.*, 2003, pp. 794-799.

[4.6] J. Cong, J. R. Shinnerl, M. Xie, T. Kong and X. Yuan, "Large-Scale Circuit Placement", *ACM Trans. on Design Autom. of Electronic Sys.* 10(2) (2005), pp. 389-430.

[4.7] J. Cong and M. Xie, "A Robust Detailed Placement for Mixed-Size IC Designs", *Proc. Asia and South Pacific Design Autom. Conf.*, 2006, pp. 188-194.

[4.8] A. E. Dunlop and B. W. Kernighan, "A Procedure for Placement of Standard-Cell VLSI Circuits", *IEEE Trans. on CAD* 4(1) (1985), pp. 92-98.

[4.9] H. Eisenmann and F. Johannes, "Generic Global Placement and Floorplanning", *Proc. Design Autom. Conf.*, 1998, pp. 269-274.

[4.10] D. Hill, *Method and System for High Speed Detailed Placement of Cells Within an Integrated Circuit Design*, U.S. Patent 6370673, 2001.

[4.11] B. Hu, Y. Zeng and M. Marek-Sadowska, "mFAR: Fixed-Points-Addition-Based VLSI Placement Algorithm", *Proc. Intl. Symp. on Phys. Design*, 2005, pp. 239-241.

[4.12] T. C. Hu and K. Moerder, "Multiterminal Flows in a Hypergraph", in *VLSI Circuit Layout: Theory and Design* (T. C. Hu and E. S. Kuh, eds.), IEEE, 1985.

[4.13] S. W. Hur and J. Lillis, "Mongrel: Hybrid Techniques for Standard Cell Placement", *Proc. Intl. Conf. on CAD*, 2000, pp. 165-170.

[4.14] A. B. Kahng, I. L. Markov and S. Reda, "On Legalization of Row-Based Placements", *Proc. Great Lakes Symp. on VLSI*, 2004, pp. 214-219.

[4.15] A. B. Kahng, P. Tucker and A. Zelikovsky, "Optimization of Linear Placements for Wirelength Minimization with Free Sites", *Proc. Asia and South Pacific Design Autom. Conf.*, 1999, pp. 241-244.

[4.16] A. B. Kahng and Q. Wang, "Implementation and Extensibility of an Analytic Placer", *IEEE Trans. on CAD* 24(5) (2005), pp. 734-747.

[4.17] A. B. Kahng and Q. Wang, "A Faster Implementation of APlace", *Proc. Intl. Symp. on Phys. Design*, 2006, pp. 218-220.

[4.18] M.-C. Kim, D.-J. Lee and I. L. Markov, "simPL: An Effective Placement Algorithm", *Proc. Intl. Conf. on CAD*, 2010.

[4.19] J. B. Kruskal, "On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem", *Proc. Amer. Math. Soc.* 7(1) (1956), pp. 8-50.

[4.20] J. K. Lam, *An Efficient Simulated Annealing Schedule* (*Doctoral Dissertation*), Yale University, 1988.

[4.21] C. Li, M. Xie, C.-K. Koh, J. Cong and P. H. Madden, "Routability-Driven Placement and White Space Allocation", *IEEE Trans. on CAD* 26(5) (2007), pp. 858-871.

[4.22] G.-J. Nam and J. Cong, eds., *Modern Circuit Placement: Best Practices and Results*, Springer, 2007.

[4.23] M. Pan, N. Viswanathan and C. Chu, "An Efficient and Effective Detailed Placement Algorithm", *Proc. Intl. Conf. on CAD*, 2005, pp. 48-55.

[4.24] N. R. Quinn, "The Placement Problem as Viewed from the Physics of Classical Mechanics", *Proc. Design Autom. Conf.*, 1975, pp. 173-178.

[4.25] J. A. Roy and I. L. Markov, "ECO-System: Embracing the Change in Placement," *IEEE Trans. on CAD* 26(12) (2007), pp. 2173-2185.

[4.26] J. A. Roy, D. A. Papa, S. N. Adya, H. H. Chan, A. N. Ng, J. F. Lu and I. L. Markov, "Capo: Robust and Scalable Open-Source Min-Cut Floorplacer", *Proc. Intl. Symp. on Phys. Design*, 2005, pp. 224-226, vlsicad.eecs.umich.edu/BK/PDtools/.

[4.27] Y. Saad, *Iterative Methods for Sparse Linear Systems*, Soc. of Industrial and App. Math., 2003.

[4.28] C. Sechen, "Chip-Planning, Placement and Global Routing of Macro/Custom Cell Integrated Circuits Using Simulated Annealing", *Proc. Design Autom. Conf.*, 1988, pp. 73-80.

[4.29] C. Sechen and A. Sangiovanni-Vincentelli, "TimberWolf 3.2: A New Standard Cell Placement and Global Routing Package", *Proc. Design Autom. Conf.*, 1986, pp. 432-439.

[4.30] K. Shahookar and P. Mazumder, "VLSI Cell Placement Techniques", *ACM Computing Surveys* 23(2) (1991), pp. 143-220.

[4.31] P. Spindler and F. M. Johannes, "Fast and Accurate Routing Demand Estimation for Efficient Routability-Driven Placement", *Proc. Design, Autom. and Test in Europe*, 2007, pp. 1226-1231.

[4.32] P. Spindler, U. Schlichtmann and F. M. Johannes, "Kraftwerk2 – A Fast Force-Directed Quadratic Placement Approach Using an Accurate Net Model", *IEEE Trans. on CAD* 27(8) (2008), pp. 1398-1411.

[4.33] R.-S. Tsay, E. S. Kuh and C.-P. Hsu, "PROUD: A Sea-of-Gates Placement Algorithm", *IEEE Design and Test* 5(6) (1988), pp. 44-56.

[4.34] N. Viswanathan, M. Pan and C. Chu, "FastPlace 3.0: A Fast Multi-level Quadratic Placement Algorithm with Placement Congestion Control", *Proc. Asia and South Pacific Design Autom. Conf.*, 2007, pp. 135-140.

[4.35] Z. Xiu and R. A. Rutenbar, "Mixed-Size Placement With Fixed Macrocells Using Grid-Warping", *Proc. Intl. Symp. on Phys. Design*, 2007, pp. 103-110.

Chapter 5

Global Routing

5

5

5 Global Routing 131
5.1 Introduction 131
5.2 Terminology and Definitions 133
5.3 Optimization Goals 136
5.4 Representations of Routing Regions 138
5.5 The Global Routing Flow 140
5.6 Single-Net Routing 141
5.6.1 Rectilinear Routing 141
5.6.2 Global Routing in a Connectivity Graph 146
5.6.3 Finding Shortest Paths with Dijkstra's Algorithm 149
5.6.4 Finding Shortest Paths with A* Search 154
5.7 Full-Netlist Routing 155
5.7.1 Routing by Integer Linear Programming 155
5.7.2 Rip-Up and Reroute (RRR) 158
5.8 Modern Global Routing 160
5.8.1 Pattern Routing 161
5.8.2 Negotiated Congestion Routing 162
Chapter 5 Exercises 164
Chapter 5 References 165

5 Global Routing

During *global routing*, pins with the same electric potential are connected using wire segments. Specifically, after *placement* (Chap. 4), the layout area is represented as *routing regions* (Sec. 5.4) and all nets in the *netlist* are routed in a systematic manner (Sec. 5.5). To minimize total routed length, or optimize other objectives (Sec. 5.3), the route of each net should be short (Sec. 5.6). However, these routes often compete for the same set of limited resources. Such conflicts can be resolved by concurrent routing of all nets (Sec. 5.7), e.g., *integer linear programming* (*ILP*), or by sequential routing techniques, e.g., *rip-up and reroute*. Several algorithmic techniques enable scalability of modern global routers (Sec. 5.8).

5.1 Introduction

A *net* is a set of two or more *pins* that have the same electric potential. In the final chip design, they must be connected. A typical p-pin net connects one output pin of a gate and $p - 1$ input pins of other gates; its *fanout* is equal to $p - 1$. The term *netlist* refers collectively to all nets.

Given a placement and a netlist, determine the necessary wiring, e.g., net topologies and specific routing segments, to connect these cells while respecting constraints, e.g., design rules and routing resource capacities, and optimizing routing objectives, e.g., minimizing total wirelength and maximizing timing slack.

In area-limited designs, standard cells may be packed densely without unused space. This often leads to routing *congestion*, where the shortest routes of several nets are incompatible because they traverse the same tracks. Congestion forces some routes to detour; thus, in congested regions, it can be difficult to predict the eventual length of wire segments. However, the total wirelength cannot exceed the available routing resources, and in some cases the chip area must be increased to ensure successful routing. *Fixed-die routing*, where the chip outline and all routing resources are fixed, is distinguished from *variable-die routing*, where new routing tracks can be added as needed. For the fixed-die routing problem,[1] 100% routing completion is not always possible *a priori*, but may be possible after changes to placement. On the other hand, in older standard-cell circuits with two or three metal layers, new tracks can be inserted as needed, resulting in the classical variable-die channel routing problem for which 100% routing completion is always possible. Fig. 5.1 outlines the major categories of routing algorithms discussed in this book.

[1] The *fixed-die* routing problem is so named because the layout bounding box and the number of routing tracks are predetermined due to the fixed floorplan and power-ground distribution.

A. B. Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure,*
DOI 10.1007/978-90-481-9591-6_5, © Springer Science+Business Media B.V. 2011

MULTI-STAGE ROUTING OF SIGNAL NETS				
GLOBAL ROUTING	**DETAILED ROUTING**	**TIMING-DRIVEN ROUTING**	**LARGE SINGLE-NET ROUTING**	**GEOMETRIC TECHNIQUES**
Coarse-grain assignment of routes to routing regions (Chap. 5)	Fine-grain assignment of routes to routing tracks (Chap. 6)	Net topology optimization and resource allocation to critical nets (Chap. 8)	Power (*VDD*) and Ground (*GND*) routing (Chap. 3)	Non-Manhattan and clock routing (Chap. 7)

Fig. 5.1 Routing problem types and the chapters in which they are discussed.

With the scale of modern designs at millions of nets, *global routing* has become a major computational challenge. Full-chip routing is usually performed in three steps: (high-level) *global routing*, (low-level) *detailed routing*, and *timing-driven routing*. The first two steps are illustrated in Fig. 5.2; the last step is discussed in Sec. 8.4.

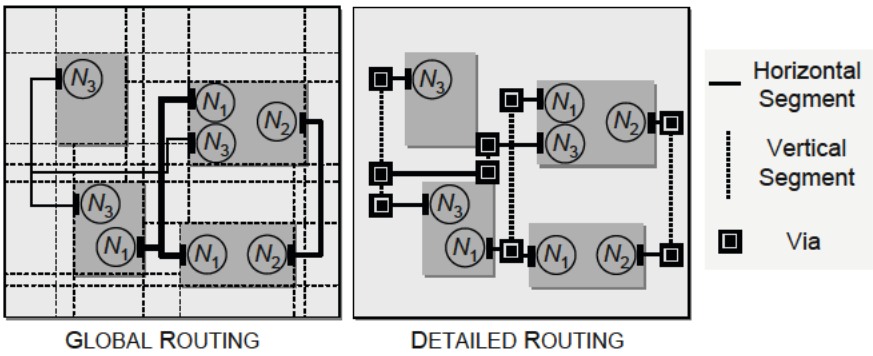


Fig. 5.2 Graphical representation of nets N_1-N_3 that are (coarsely) globally routed using *routing regions* (left), and then (finely) detailed routed using rectilinear paths (right). This example assumes two-layer routing, with horizontal and vertical segments routed on separate layers.

During global routing, the wire segments used by net topologies are tentatively assigned (embedded) within the chip layout. The chip area is represented by a coarse routing grid, and available routing resources are represented by edges with capacities in a grid graph. Nets are then assigned to these routing resources.

During detailed routing, the wire segments are assigned to specific routing tracks. This process involves a number of intermediate tasks and decisions such as *net ordering*, i.e., which nets should be routed first, and *pin ordering*, i.e., within a net, in what order should the pins be connected. These two issues are the major challenges in *sequential* routing, where nets are routed one at a time. The net and pin orderings can have dramatic impacts on final solution quality. Detailed routing seeks to refine global routes and typically does not alter the configuration of nets determined by global routing. Hence, if the global routing solution is poor, the quality of the detailed routing solution will likewise suffer.

To determine the net ordering, each net is given a numerical indicator of importance (priority), known as a *net weight*. High priority can be given to nets that are timing-critical, connect to numerous pins, or carry specific functions such as delivering clock signals. High-priority nets should avoid unnecessary detours, even at the cost of detouring other nets. Pin ordering is typically performed using either tree-based algorithms (Sec. 5.6.1) or geometric criteria based on pin locations.

Specializing routing into global and detailed stages is common for digital circuits. For analog circuits, multi-chip modules (MCMs), and printed circuit boards (PCBs), global routing is sometimes unnecessary due to the smaller number of nets involved, and only detailed routing is performed.

5.2 Terminology and Definitions

The following terms are relevant to global routing in general. Terms pertaining to specific algorithms and techniques will be introduced in their respective sections.

A *routing track* (*column*) is an available horizontal (vertical) wiring path. A signal net often uses a sequence of alternating horizontal tracks and vertical columns, where adjacent tracks and columns are connected by inter-layer vias.

A *routing region* is a region that contains routing tracks and/or columns.

A *uniform routing region* is formed by evenly spaced horizontal and vertical grid lines that induce a uniform grid over the chip area. This grid is sometimes referred to as a *ggrid* (global grid); it is composed of unit *gcells* (global cells). Grid lines are typically spaced seven to 40 routing tracks [5.18] apart to balance the complexities of the chip-scale global routing and gcell-scale detailed routing problems.

A *non-uniform routing region* is formed by horizontal and vertical boundaries that are aligned to external pin connections or macro-cell boundaries. This results in *channels* and *switchboxes* – routing regions that have differing sizes. During global routing, nets are assigned to these routing regions. During detailed routing, the nets within each routing region are assigned to specific wiring paths.

A *channel* is a rectangular routing region with pins on two opposite (usually the longer) sides and no pins on the other (usually the shorter) sides. There are two types of channels – *horizontal* and *vertical*.

A *horizontal channel* is a channel with pins on the top and bottom boundaries (Fig. 5.3).

A *vertical channel* is a channel with pins on the left and right boundaries (Fig. 5.4).

The pins of a net are connected to the routing channel by columns, which in turn are connected to other columns by tracks. In older, variable-die routing contexts, such as two-layer standard-cell routing, the channel height is flexible, i.e., its capacity can be adjusted to accommodate the necessary amount of wiring. However, due to the increased number of routing layers in modern designs, this traditional channel model has largely lost its relevance. Instead, *over-the-cell* (*OTC*) routing is used, as discussed further in Sec. 6.7.

The *channel capacity* represents the number of available routing tracks or columns. For single-layer routing, the capacity is the height h of the channel divided by the pitch d_{pitch}, where d_{pitch} is the minimum distance between two wiring paths in the relevant (vertical or horizontal) direction (Fig. 5.3). For multilayer routing, the capacity σ is the sum of the capacities of all layers.

$$\sigma(Layers) = \sum_{layer \in Layers} \left\lfloor \frac{h}{d_{pitch}(layer)} \right\rfloor$$

Here, *Layers* is the set of all layers, and $d_{pitch}(layer)$ is the routing pitch for *layer*.

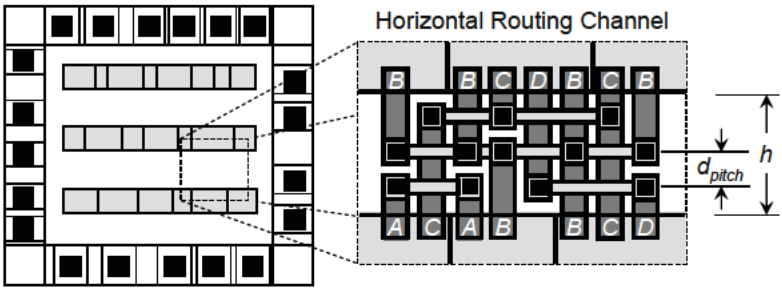


Fig. 5.3 Channel routing for a horizontal channel.

A *switchbox* (Fig. 5.4) is the intersection of horizontal and vertical channels. Due to fixed dimensions, switchbox routing exhibits less flexibility and is more difficult than channel routing. Note that since the entry points of wires are fixed, the problem is one of finding routes inside the switchbox.

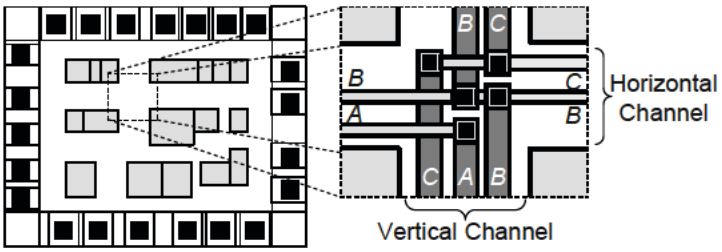


Fig. 5.4 Switchbox routing between horizontal and vertical channels of a macro-cell circuit.

A *2D switchbox* (Fig. 5.5) is a switchbox with terminals on four boundaries (top, bottom, left, and right). This model is primarily used with two-layer routing, where interlayer connections (vias) are relatively insignificant.

A *3D switchbox* (Fig. 5.5) is a switchbox with terminals on all six boundaries (top, bottom, left, right, up, and down), allowing paths to travel between routing layers. Four of the sides are the same as those of the 2D switchbox, one goes to the layer above, and one goes to the layer below.

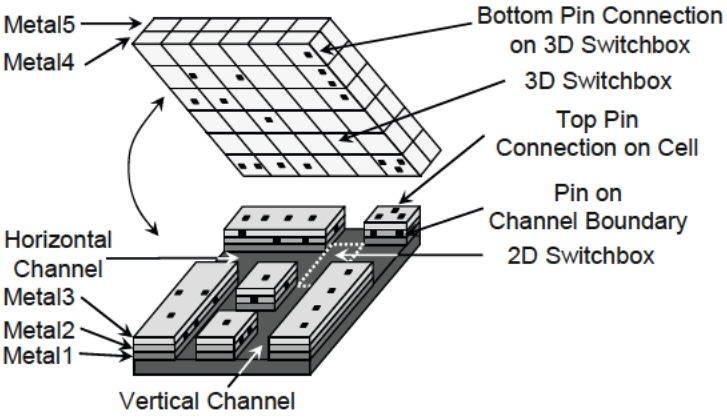


Fig. 5.5 2D and 3D switchboxes in a five-layer process. The cells are internally routed using layers up to Metal3. 2D switchboxes typically exist on layers Metal1, Metal2 and Metal3. Layers Metal4 and Metal5 are connected by a 3D switchbox.

A *T-junction* occurs when a vertical channel meets a horizontal channel, e.g., in a macro-cell placement. In the example of Fig. 5.6, both (1) the height of the vertical channel and (2) the locations of the fixed pin connections for the horizontal channel are determined only after the vertical channel has been routed. Therefore, the vertical channel must be routed before the horizontal channel.

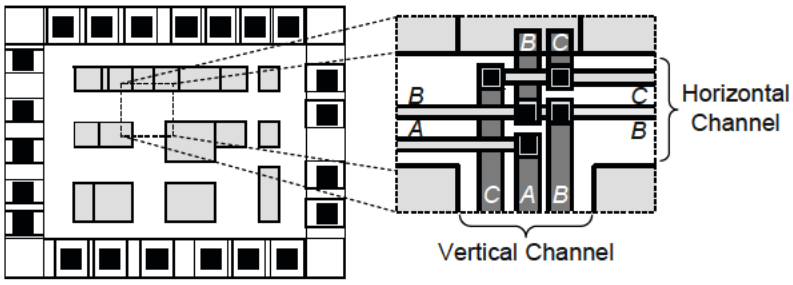


Fig. 5.6 Routing for a T-junction in a macro-cell placement.